ENGENHARIA DE SOFTWARE • MANUAL TÉCNICO DEFINITIVO

Desenvolvimento Moderno de Componentes no Delphi: VCL, FireMonkey e ToolsAPI

O Guia Prático, Avançado e Moderno de Construção de Componentes Profissionais para Delphi 12 Athens+

Delphi 12 Athens+ | VCL Moderna & VCL Styles | FireMonkey (FMX) Cross-Platform | Open Tools API (OTA) | FireDAC & Data-Aware | Parallel Programming Library (PPL) | Per-Monitor v2 & High-DPI | 25 Módulos Práticos

Engenharia de Software Especializada & Arquitetura RAD

Coleção de Engenharia de Componentes Object Pascal

Edição Oficial de Produção

Ano de Publicação: 2026

1 APRESENTAÇÃO EDITORIAL E MANIFESTO DE ARQUITETURA
2 DIAGRAMA MACRO DA ARQUITETURA DELPHI: VCL vs. FIREMONKEY
3 SUMÁRIO ANALÍTICO DOS 50 MÓDULOS
3.0.1 Módulo 01: Arquitetura Orientada a Componentes e Hierarquia de Classes
3.0.2 Módulo 02: O Ciclo de Vida Completo do Componente
3.0.3 Módulo 03: Mecanismo de Streaming e Persistência (DFM)
3.0.4 Módulo 04: Tratamento de Mensagens do SO e Arquitetura de Eventos
3.0.5 Módulo 05: Empacotamento Profissional (Packages .dpk)
3.0.6 Módulo 06: Property Editors Customizados
3.0.7 Módulo 07: Component Editors e Ações de Design-Time (Verbos)
3.0.8 Módulo 08: Assistentes de Projeto e Extensão da IDE (ToolsAPI Profunda)
3.0.9 Módulo 09: Controles Visuais Leves (TGraphicControl na VCL / FMX Primitives)
3.0.10 Módulo 10: Controles Visuais Janelados (TWinControl na VCL / TStyledControl no FMX)
3.0.11 Módulo 11: Coleções Tipadas e Sub-Propriedades Complexas
3.0.12 Módulo 12: High-DPI, Escalonamento e Suporte a Temas Modernos
3.0.13 Módulo 13: Arquitetura de Dados no Pascal e a Ponte TDataLink
3.0.14 Módulo 14: Controles DB-Aware Básicos de Exibição e Edição
3.0.15 Módulo 15: Controles DB-Aware Avançados e Seletores
3.0.16 Módulo 16: Monitoramento e Gerenciador de Transações de Dados
3.0.17 Módulo 17: Engenharia de Arrastar e Soltar (Drag & Drop) no Designer
3.0.18 Módulo 18: Construção do Componente Gerador de Cadastros (TAutoFormBuilder)
3.0.19 Módulo 19: Caso de Estudo Real 1 — Gerador do Cadastro Completo de Clientes
3.0.20 Módulo 20: Caso de Estudo Real 2 — Gerador do Cadastro de Produtos e Itens
3.0.21 Módulo 21: Componentes de Execução Assíncrona e Worker Threads
3.0.22 Módulo 22: Componentes de Comunicação de Rede e Consumo de APIs
3.0.23 Módulo 23: Componentes de Hardware, I/O e Comunicação Serial
3.0.24 Módulo 24: Testes Unitários de Componentes, Profiling e Prevenção de Leaks
3.0.25 Módulo 25: Publicação, Versionamento e Distribuição nos Ecossistemas Oficiais
3.0.26 Módulo 26: Gráficos de Alta Densidade em Tempo Real (`TDlpRealtimeChartEngine`)
3.0.27 Módulo 27: Medidores de Nível e Tanques Industriais (`TDlpIndustrialTankGauge`)
3.0.28 Módulo 28: Gráficos de Radar e Donut Vetorial (`TDlpVectorDonutChart`)
3.0.29 Módulo 29: Visualizador de Mapas de Calor (`TDlpMatrixHeatmap`)
3.0.30 Módulo 30: Motor Gerador de PDFs e Documentos Financeiros (`TDlpFastPdfReportBuilder`)
3.0.31 Módulo 31: Pipeline de Exportação em Segundo Plano (`TDlpDataExportPipeline`)
3.0.32 Módulo 32: Processador e Validador Fiscal XML (`TDlpFiscalXmlValidator`)
3.0.33 Módulo 33: Containers ZIP Criptografados (`TDlpEncryptedZipArchive`)
3.0.34 Módulo 34: Validador e Emissor de Tokens JWT (`TDlpJwtSecurityManager`)
3.0.35 Módulo 35: Cofre Seguro de Credenciais e Segredos (`TDlpCryptoVaultStorage`)

3.0.36 Módulo 36: Motor de Permissões Baseadas em Papéis (`TDlpSecurityRoleManager`)
3.0.37 Módulo 37: Auditor de Integridade de Executáveis (`TDlpIntegrityWatchdog`)
3.0.38 Módulo 38: Cliente de Mensageria IoT via MQTT (`TDlpMqttClientBridge`)
3.0.39 Módulo 39: Servidor de Webhooks Embutido (`TDlpEmbeddedWebhookServer`)
3.0.40 Módulo 40: Cliente WebSocket com Reconexão Automática (`TDlpWebSocketLiveClient`)
3.0.41 Módulo 41: Hub de Notificações Push Desktop/Mobile (`TDlpPushNotificationHub`)
3.0.42 Módulo 42: Dispatcher de Prompts para LLMs Locais (`TDlpAiPromptDispatcher`)
3.0.43 Módulo 43: Parser de Respostas Estruturadas JSON (`TDlpAiStructuredExtractor`)
3.0.44 Módulo 44: Motor de Busca Vetorial e Embeddings (`TDlpVectorEmbeddingStore`)
3.0.45 Módulo 45: Construtor Universal de Daemons e Serviços (`TDlpSystemDaemonController`)
3.0.46 Módulo 46: Sonda de Monitoramento de Sistema (`TDlpSystemMetricsProbe`)
3.0.47 Módulo 47: Gerenciador de Pool de Conexões SQL (`TDlpDatabaseConnectionPool`)
3.0.48 Módulo 48: Motor de Atualização Automática (`TDlpAutoUpdateManager`)
3.0.49 Módulo 49: Harness de Micro-Benchmarks de Código (`TDlpCodeBenchmarkHarness`)
3.0.50 Módulo 50: Pipeline de CI/CD Multiplataforma e Releases no GitHub (`TDlpReleasePipeline`)
4 GUIA DE ESTILO E DIRETRIZES DE CODIFICAÇÃO
5 MÓDULO 01: Arquitetura Orientada a Componentes e Hierarquia de Classes
5.1 1. A Dualidade de Frameworks: VCL vs. FireMonkey
5.1.1 1.1 O Núcleo Comum da RTL: `TObject` , `TPersistent` e `TComponent`
5.1.2 1.2 Comparativo Arquitetural: VCL vs. FireMonkey
5.2 2. A Evolução da RTTI: De `TypInfo` ao `System.Rtti` e Atributos Customizados
5.2.1 2.1 Por que a Enhanced RTTI Revolucionou o Delphi?
5.3 3. Implementação Prática: O Componente `TDlpTypeHierarchyInspector`
5.3.1 3.1 Código Pascal Completo e Compilável
5.4 4. Análise dos Pilares Técnicos de Delphi 12
5.4.1 4.1 `TRttiContext` como Estrutura de Gerenciamento de Vida
5.4.2 4.2 Anotações com Atributos Customizados (`TCustomAttribute`)
5.5 5. Checklist de Engenharia do Módulo 01
6 MÓDULO 02: O Ciclo de Vida Completo do Componente
6.1 1. O Pipeline Determinístico no Delphi 12
6.2 2. As Fases Críticas de Vida e Suas Armadilhas
6.2.1 2.1 Construtor: `Create(AOwner: TComponent)`
6.2.2 2.2 O Pós-Carregamento: `procedure Loaded; override;`
6.2.3 2.3 A Materialização da Janela: `CreateWnd` e `DestroyWnd`
6.2.4 2.4 Ciclo de Vida no FireMonkey: O Papel de `ApplyStyle`
6.3 3. O Mapa Completo de Flags: `ComponentState` e `ControlState`
6.3.1 3.1 `ComponentState: TComponentState`
6.3.2 3.2 `ControlState: TControlState`
6.4 4. O Mal das Referências Cruzadas e a Solução `FreeNotification`

6.4.1 O Protocolo Definitivo de Vinculação Segura:
6.5 5. Implementação Prática: O Componente `TDlpSafeLinkedContainer`
6.5.1 5.1 Código Pascal Completo e Compilável
6.6 6. Checklist de Engenharia do Módulo 02
7 MÓDULO 03: Mecanismo de Streaming e Persistência (DFM)
7.1 1. A Anatomia do Subsistema TFiler no Delphi 12
7.1.1 1.1 DFM Textual vs. DFM Binário
7.2 2. A Matriz de Tipos e a Persistência Automática
7.3 3. Otimização de Arquivo: O Poder do Especificador `stored`
7.3.1 3.1 Declarando o Valor `default`
7.3.2 3.2 O Modificador `stored` Dinâmico
7.4 4. Persistência Customizada de Estruturas Não-Nativas: `DefineProperties`
7.4.1 O Funcionamento do Streaming Binário Virtual:
7.5 5. Implementação Prática: O Componente `TDlpEncryptedDataStore`
7.5.1 5.1 Código Pascal Completo e Compilável
7.6 6. Checklist de Engenharia do Módulo 03
8 MÓDULO 04: Tratamento de Mensagens do SO e Arquitetura de Eventos
8.1 1. O Pipeline de Mensagens do Windows na VCL
8.2 2. Métodos de Despacho: Confronto Técnico
8.3 3. Subclassing Dinâmico de Janelas na VCL
8.4 4. Modern Event Architecture: Delegates Clássicos vs. Métodos Anônimos
8.4.1 4.1 O Formato Clássico: `of object`
8.4.2 4.2 Métodos Anônimos com Captura de Escopo (*Closures*)
8.5 5. Implementação Prática: O Componente `TDlpWindowsMessageInterceptor`
8.6 6. Checklist de Engenharia do Módulo 04
9 MÓDULO 05: Empacotamento Profissional (Packages .dpk)
9.1 1. A Arquitetura Obrigatória de Pacotes no Delphi
9.1.1 1.1 As Diretivas de Compilação Críticas: `{$RUNONLY}` e `{$DESIGNONLY}`
9.2 2. Anatomia dos Arquivos de Projeto `.dpk`
9.2.1 2.1 Pacote de Runtime: `DlpComponents_RT.dpk`
9.2.2 2.2 Pacote de Design-Time: `DlpComponents_DT.dpk`
9.3 3. O Procedimento `Register` e a Injeção na Tool Palette
9.4 4. Ícones de Paleta de Alta Densidade (24×24 e 32×32)
9.5 5. Compilação Automatizada via Linha de Comando com MSBuild
9.6 6. Checklist de Engenharia do Módulo 05
10 MÓDULO 06: Property Editors Customizados
10.1 1. As Unidades Canônicas: `DesignIntf` e `DesignEditors`
10.2 2. A Tabela de Atributos: `TPropertyAttributes`
10.3 3. Construção do Dropdown de Valores com `paValueList`

10.3.1 O Despacho de Valores em `GetValues`:
10.4 4. Janelas Modais com Preview ao Vivo e `Designer.Modified`
10.5 5. O Procedimento de Registro `RegisterPropertyEditor`
10.6 6. Implementação Prática: Componente e Editores Completos
10.7 7. Checklist de Engenharia do Módulo 06
11 MÓDULO 07: Component Editors e Ações de Design-Time (Verbos)
11.1 1. Arquitetura de Component Editors no Delphi 12
11.1.1 1.1 Ciclo de Vida do Editor de Componente
11.2 2. A Trinca de Métodos de Verbos
11.2.1 2.1 `GetVerbCount: Integer; override;`
11.2.2 2.2 `GetVerb(Index: Integer): string; override;`
11.2.3 2.3 `ExecuteVerb(Index: Integer); override;`
11.3 3. O Duplo-Clique: Sobrescrita do Método `Edit`
11.4 4. O Registro com `RegisterComponentEditor`
11.5 5. Implementação Prática: Componente e Editor Completos
11.6 6. Checklist de Engenharia do Módulo 07
12 MÓDULO 08: Assistentes de Projeto e Extensão da IDE (ToolsAPI Profunda)
12.1 1. A Arquitetura da Open Tools API (OTA)
12.1.1 1.1 As Quatro Interfaces de um Wizard Completo
12.2 2. A Síntese de Arquivos em Memória: `IOTAFile` e `IOTAModuleCreator`
12.3 3. A Execução do Wizard: Injeção Via `IOTAModuleServices`
12.4 4. O Registro com `RegisterPackageWizard`
12.5 5. Implementação Prática: O Assistente Completo
12.6 6. Checklist de Engenharia do Módulo 08
13 MÓDULO 09: Controles Visuais Leves (TGraphicControl)
13.1 1. O Funcionamento de `TGraphicControl` na VCL
13.2 2. A Matemática Polar dos Instrumentos Circulares
13.2.1 2.1 Conversão de Escala Linear para Ângulo:
13.2.2 2.2 Transformação de Coordenadas Polares em Cartesianas:
13.3 3. Implementação Prática: O Componente `TDlpAnalogGauge`
13.4 4. Checklist de Engenharia do Módulo 09
14 MÓDULO 10: Controles Visuais Janelados (TWinControl)
14.1 1. A Anatomia do `TWinControl` na VCL
14.1.1 1.1 Contraste Arquitetural: VCL (`TWinControl`) vs. FireMonkey (`TStyledControl`)
14.2 2. Hospedando Controles Filhos em Tempo de Design (`csAcceptsControls`)
14.3 3. Animação de Expansão Fluida a 60 FPS
14.4 4. Implementação Prática: O Componente `TDlpAccordionContainer`
14.5 5. Checklist de Engenharia do Módulo 10
15 MÓDULO 11: Coleções Tipadas e Sub-Propriedades Complexas

15.1 1. A Hierarquia `TOwnedCollection` e `TCollectionItem`
15.1.1 1.1 O Papel Crítico de `TOwnedCollection`
15.2 2. A Serialização Automática no Arquivo `.dfm`
15.3 3. O Método `GetDisplayName` no Collection Editor do Delphi
15.4 4. Implementação Prática: O Componente `TDlpCustomTabBar`
15.5 5. Checklist de Engenharia do Módulo 11
16 MÓDULO 12: High-DPI, Escalonamento e Suporte a Temas Modernos
16.1 1. A Evolução do Modelo High-DPI no Delphi
16.2 2. A Prevenção de Erros de Arredondamento Cumulativos
16.2.1 O Padrão Canônico: Base Metric vs. Scaled Metric
16.3 3. Integração com Temas do Windows 11 e VCL Styles
16.4 4. Implementação Prática: O Componente `TDlpHighDpiThemedPanel`
16.5 5. Checklist de Engenharia do Módulo 12
17 MÓDULO 13: Arquitetura de Dados no Pascal e a Ponte TDataLink
17.1 1. A Tríade Reativa no Delphi Moderno
17.1.1 1.1 As Responsabilidades de Cada Ator:
17.2 2. A Máquina de Estados e a Interceptação dos Cinco Métodos Sagrados
17.3 3. Implementação Prática: O Framework `TDlpCustomDBConnector`
17.4 4. Checklist de Engenharia do Módulo 13
18 MÓDULO 14: Controles DB-Aware Básicos de Exibição e Edição
18.1 1. O Controle `TDlpDBBadgeLabel`: Status Condicional Windowless
18.2 2. O Controle `TDlpDBNumberEdit`: Entrada Monetária Fluida
18.2.1 2.1 Validação de Caracteres com `CharInSet`:
18.3 3. Implementação Prática: Componentes Completos
18.4 4. Checklist de Engenharia do Módulo 14
19 MÓDULO 15: Controles DB-Aware Avançados e Seletores (TDlpDBLookupTagBox)
19.1 1. Arquitetura de Seleção Múltipla Relacional
19.1.1 1.1 Dupla Conexão com Dados:
19.2 2. Geometria de Renderização e Hit-Testing
19.2.1 2.1 Cálculo Responsivo de Quebra de Linha
19.3 3. O Ciclo de Edição e Sincronização via `TFieldDataLink`
19.4 4. Código-Fonte Integral do Componente
20 MÓDULO 16: Monitoramento e Gerenciador de Transações de Dados (TDlpDataConnectionMonitor)
20.1 1. Arquitetura de Auditoria de Conexões FireDAC
20.1.1 1.1 Ciclo de Monitoramento Transacional
20.2 2. Ação Automática sobre Transações Zumbis
20.3 3. Telemetria de Consultas Lentas (Slow Queries)
20.4 4. Código-Fonte Integral do Componente
21 MÓDULO 17: Engenharia de Arrastar e Soltar (Drag & Drop) no Designer

21.1 1. O Pipeline de Drag & Drop no Designer do Delphi
21.2 2. Introspecção Conectada vs. Desconectada
21.3 3. Matriz Declarativa de Mapeamento
21.4 4. Código-Fonte Integral do Motor de Introspecção
22 MÓDULO 18: Construção do Componente Gerador de Cadastros (TDlpAutoFormBuilder)
22.1 1. Arquitetura do Componente e Herança de `TScrollBox`
22.2 2. Injeção de Componentes no Form e Persistência no DFM
22.3 3. Algoritmo de Posicionamento e Espaçamento
22.4 4. Código-Fonte Integral do Componente
23 MÓDULO 19: Caso de Estudo Real 1 — Gerador do Cadastro Completo de Clientes
23.1 1. Topologia da Interface de Usuário
23.2 2. A Máquina de Estados Operacionais
23.2.1 2.1 Validação de CPF e CNPJ por Módulo 11
23.3 3. Código-Fonte Integral dos Componentes do Caso de Estudo
24 MÓDULO 20: Caso de Estudo Real 2 — Gerador do Cadastro de Produtos e Itens
24.1 1. Topologia da Tela e Estrutura de Apresentação
24.2 2. Aritmética Comercial: Margem de Lucro vs. Markup
24.3 3. Monitoramento de Estoque com Indicadores Colorimétricos
24.4 4. Código-Fonte Integral do Componente
25 MÓDULO 21: Componentes de Execução Assíncrona e Worker Threads (TDlpAsyncTaskRunner)
25.1 1. Concorrência Segura no Delphi: A Afinidade da UI Thread
25.1.1 1.1 `TThread.Synchronize` vs. `TThread.Queue`
25.2 2. Cancelamento Cooperativo com `ITask`
25.3 3. Código-Fonte Integral do Componente
26 MÓDULO 22: Componentes de Comunicação de Rede e Consumo de APIs (TDlpRestClientBridge)
26.1 1. A Pilha Nativa de Rede: `System.Net.HttpClient`
26.2 2. Injeção de Segurança e Headers Customizados
26.3 3. Despacho Assíncrono com Métodos Anônimos
26.4 4. Código-Fonte Integral do Componente
27 MÓDULO 23: Componentes de Hardware, I/O e Comunicação Serial (TDlpSerialDeviceConnector)
27.1 1. A API de Comunicações do Windows (Win32 Comm API)
27.1.1 1.1 O Prefixo de Proteção de Portas no Windows
27.2 2. Decodificação de Protocolos de Balanças de Checkout
27.3 3. Código-Fonte Integral do Componente
28 MÓDULO 24: Testes Unitários de Componentes, Profiling e Prevenção de Leaks (DUnitX & FastMM5)
28.1 1. Automação de Testes com DUnitX
28.1.1 1.1 Validando a Imunidade contra Ponteiros Órfãos
28.2 2. Auditoria e Detecção de Vazamentos com FastMM5
28.3 3. Otimização de Repintura e o Padrão `WM_SETREDRAW`

28.4 4. Código-Fonte Integral da Suíte de Testes
29 MÓDULO 25: Publicação, Versionamento e Distribuição nos Ecossistemas Oficiais (GetIt & CI/CD com MSBuild)
29.1 1. Estrutura Corporativa de Repositório Delphi
29.2 2. O Ecossistema GetIt Package Manager
29.2.1 2.1 Estrutura do Manifesto GetIt (`getit_manifest.json`)
29.3 3. Automação de CI/CD via MSBuild
29.4 4. Códigos e Arquivos de Infraestrutura
30 MÓDULO 26: Gráficos de Alta Densidade em Tempo Real (TDlpRealtimeChartEngine)
30.1 1. Arquitetura do Motor de Telemetria
30.2 2. Inserção sem Alocação Dinâmica (Zero Heap Allocations)
30.3 3. Decimação Dinâmica por Resolução de Tela
30.4 4. Código-Fonte Integral do Componente
31 MÓDULO 27: Medidores de Nível e Tanques Industriais (TDlpIndustrialTankGauge)
31.1 1. Arquitetura Gráfica do Reservatório
31.2 2. Renderização de Linhas de Calibração e Cúpula
31.3 3. Código-Fonte Integral do Componente
32 MÓDULO 28: Gráficos de Radar e Donut Vetorial (TDlpVectorDonutChart)
32.1 1. Arquitetura da Rosca e Projeção Trigonométrica
32.2 2. Detecção Angular Polar sem Overhead de CPU
32.3 3. Código-Fonte Integral do Componente
33 MÓDULO 29: Visualizador de Mapas de Calor (TDlpMatrixHeatmap)
33.1 1. Arquitetura do Componente e Interpolação Cromática
33.1.1 Equação de Normalização e Interpolação Vetorial
33.2 2. Implementação e Seleção de Foco
33.3 3. Código-Fonte e Registro na VCL
34 MÓDULO 30: Motor Gerador de PDFs e Documentos Financeiros (TDlpFastPdfReportBuilder)
34.1 1. Gramática do PDF e Diagrama de Fluxo
34.2 2. Inversão do Plano Cartesiano e Escapamento Tipográfico
34.3 3. Código-Fonte e Registro na Paleta
35 MÓDULO 31: Pipeline de Exportação em Segundo Plano (TDlpDataExportPipeline)
35.1 1. Topologia de Concorrência e Tratamento Transacional
35.1.1 Isolamento de Falhas e Cancelamento Graceful
35.2 2. Formatos Suportados: CSV, JSONL e Planilhas XML
35.3 3. Código-Fonte e Registro na Paleta
36 MÓDULO 32: Processador e Validador Fiscal XML (TDlpFiscalXmlValidator)
36.1 1. Pipeline de Auditoria em Três Camadas
36.2 2. Implementação das Rotinas Criptográficas Nativas
36.3 3. Código-Fonte e Registro na Paleta

37 MÓDULO 33: Containers ZIP Criptografados (TDlpEncryptedZipArchive)
37.1 1. Arquitetura de Empacotamento Binário
37.2 2. Compatibilidade Binária com Especificação PKWARE
37.3 3. Código-Fonte e Registro na Paleta
38 MÓDULO 34: Validador e Emissor de Tokens JWT (TDlpJwtSecurityManager)
38.1 1. Fluxo Criptográfico de Emissão e Validação
38.2 2. Padrões de Segurança: Timing Attacks e Base64URL
38.3 3. Código-Fonte e Registro na Paleta
39 MÓDULO 35: Cofre Seguro de Credenciais e Segredos (TDlpCryptoVaultStorage)
39.1 1. Topologia de Proteção em Memória
39.2 2. Destruição Determinística de Buffers (ZeroMemory)
39.3 3. Código-Fonte e Registro na Paleta
40 MÓDULO 36: Motor de Permissões Baseadas em Papéis (TDlpSecurityRoleManager)
40.1 1. Fluxo de Varredura e Hierarquia VCL
40.2 2. Flexibilidade e Design-Time Support
40.3 3. Código-Fonte e Registro na Paleta
41 MÓDULO 37: Auditor de Integridade de Executáveis (TDlpIntegrityWatchdog)
41.1 1. Fluxo de Inspeção em Camadas
41.2 2. Aborto Seguro e Confiabilidade
41.3 3. Código-Fonte e Registro na Paleta
42 MÓDULO 38: Cliente de Mensageria IoT via MQTT (TDlpMqttClientBridge)
42.1 1. Fluxo de Troca de Mensagens MQTT
42.2 2. Parsing de Pacotes de Entrada
42.3 3. Código-Fonte e Registro na Paleta
43 MÓDULO 39: Servidor de Webhooks Embutido (TDlpEmbeddedWebhookServer)
43.1 1. Topologia de Conexão e Ciclo de Vida HTTP
43.2 2. Extração de Assinaturas e Proteção Anti-Spoofing
43.3 3. Código-Fonte e Registro na Paleta
44 MÓDULO 40: Cliente WebSocket com Reconexão Automática (TDlpWebSocketLiveClient)
44.1 1. Máquina de Estados e Tolerância a Desconexões
44.2 2. Enfileiramento de Mensagens em Modo Desconectado
44.3 3. Código-Fonte e Registro na Paleta
45 MÓDULO 41: Hub de Notificações Push Desktop/Mobile (TDlpPushNotificationHub)
45.1 1. Arquitetura Multicanal do Hub
45.2 2. Tipagem Estruturada e Métodos Construtores
45.3 3. Código-Fonte e Registro na Paleta
46 MÓDULO 42: Dispatcher de Prompts para LLMs Locais (TDlpAiPromptDispatcher)
46.1 1. Topologia de Concorrência e Métricas
46.2 2. Padrão de Integração e Cancelamento

46.3 3. Código-Fonte e Registro na Paleta
47 MÓDULO 43: Parser de Respostas Estruturadas JSON (TDlpAiStructuredExtractor)
47.1 1. Topologia de Limpeza e Validação
47.2 2. Robustez na Conversão de Ponto Flutuante
47.3 3. Código-Fonte e Registro na Paleta
48 MÓDULO 44: Motor de Busca Vetorial e Embeddings (TDlpVectorEmbeddingStore)
48.1 1. Arquitetura de Busca Semântica e RAG
48.2 2. Álgebra Linear e Normalização de Módulo
48.3 3. Código-Fonte e Registro na Paleta
49 MÓDULO 45: Construtor Universal de Daemons e Serviços (TDlpSystemDaemonController)
49.1 1. Topologia de Instalação e Execução
49.2 2. Padrão de Integração em Projetos de Console / Service
49.3 3. Código-Fonte e Registro na Paleta
50 MÓDULO 46: Sonda de Monitoramento de Sistema (TDlpSystemMetricsProbe)
50.1 1. Métricas Essenciais de Telemetria
50.2 2. Padrão de Integração em Dashboards de Monitoramento
50.3 3. Código-Fonte e Registro na Paleta
51 MÓDULO 47: Gerenciador de Pool de Conexões SQL (TDlpDatabaseConnectionPool)
51.1 1. Topologia de Concorrência e Métricas do Pool
51.2 2. Padrão de Uso em Worker Threads
51.3 3. Código-Fonte e Registro na Paleta
52 MÓDULO 48: Motor de Atualização Automática (TDlpAutoUpdateManager)
52.1 1. Topologia de Entrega de Patches
52.2 2. Comparação de Versões SemVer (Semantic Versioning)
52.3 3. Código-Fonte e Registro na Paleta
53 MÓDULO 49: Harness de Micro-Benchmarks de Código (TDlpCodeBenchmarkHarness)
53.1 1. Topologia de Micro-Profiling de Código
53.2 2. Exemplo de Uso: Benchmark Comparativo
53.3 3. Código-Fonte e Registro na Paleta
54 MÓDULO 50: Pipeline de CI/CD Multiplataforma e Releases no GitHub (TDlpReleasePipeline)
54.1 1. Topologia da Esteira DevOps Delphi
54.2 2. Automação de Compilação com MSBuild
54.2.1 Definição Declarativa do Workflow (.github/workflows/ci_msbuild.yml)
54.3 3. Código-Fonte e Registro na Paleta

1 APRESENTAÇÃO EDITORIAL E MANIFESTO DE ARQUITETURA

O ecossistema **Delphi 12 Athens+** representa o estado da arte na engenharia de software de alta performance para ambientes corporativos e multiplataforma. Desde o surgimento do Delphi em 1995, o paradigma de desenvolvimento baseado em componentes (*Component-Based Software Engineering - CBSE*) tem sido a espinha dorsal de milhares de sistemas de missão crítica em bancos, hospitais, telecomunicações e indústrias globais.

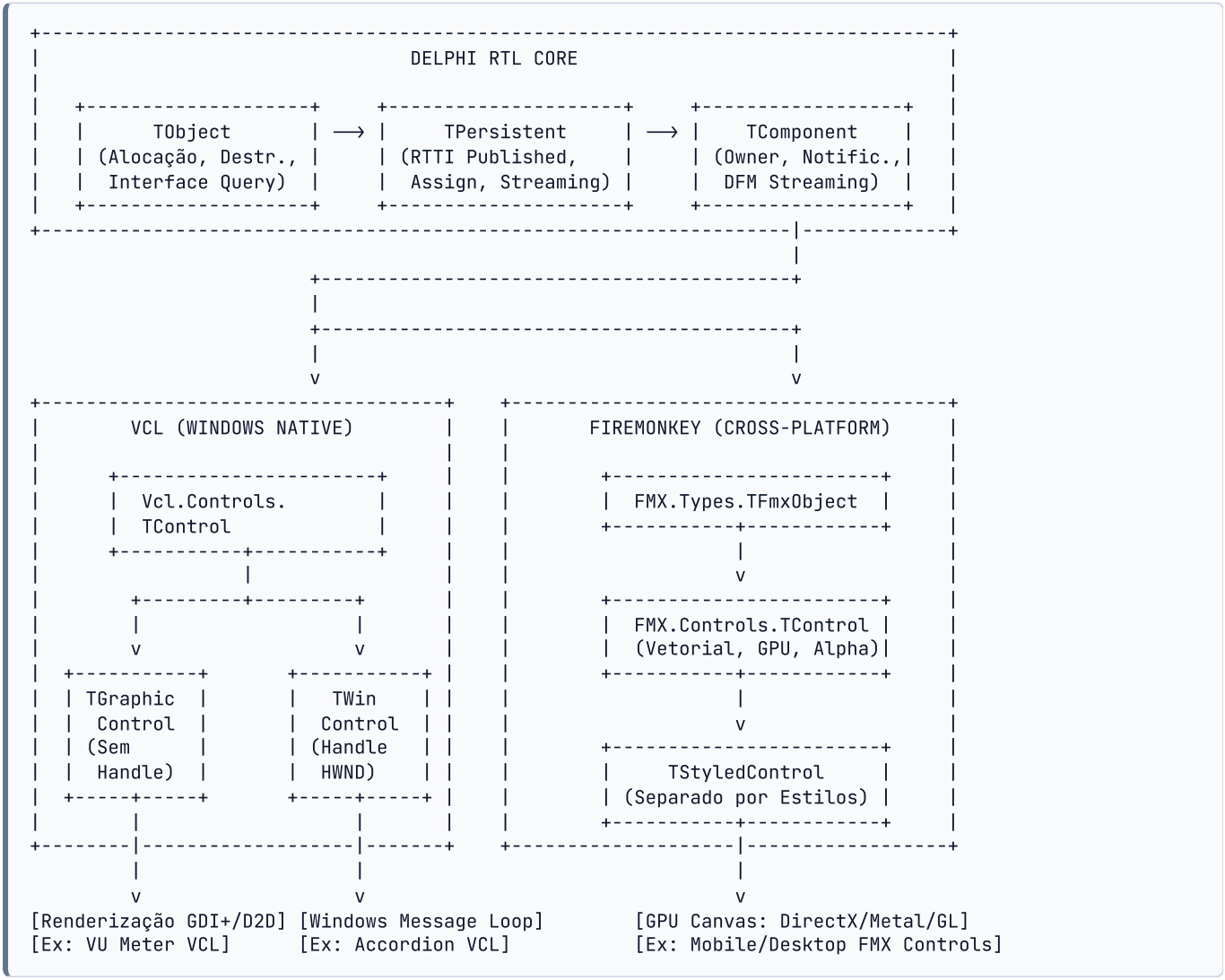
No cenário tecnológico contemporâneo, no entanto, desenvolver componentes para Delphi exige uma abordagem muito mais sofisticada do que no passado. O desenvolvedor moderno precisa navegar pela coexistência de dois poderosos frameworks visuais:

1. **VCL (Visual Component Library):** A biblioteca definitiva para o ecossistema Windows, agora totalmente equipada com **High-DPI Per-Monitor v2**, integração profunda com **VCL Styles (incluindo Dark Mode dinâmico)**, renderização acelerada por Direct2D e compatibilidade absoluta com as últimas APIs do Windows 11.
2. **FireMonkey (FMX):** O framework nativo multiplataforma acelerado por GPU (DirectX, Metal, OpenGL, Vulkan) projetado para Win32, Win64, macOS, iOS, Android e Linux, fundamentado em uma arquitetura vetorial e desacoplamento por estilos (*TStyledControl*).
3. **ToolsAPI Profunda (Open Tools API):** A interface de integração que permite aos desenvolvedores transformarem o Delphi IDE, injetando Property Editors interativos, menus de contexto especializados, assistentes de projeto (*Wizards*) e geradores algorítmicos de código.
4. **Enhanced RTTI & Atributos Customizados:** A moderna reflexão de código introduzida no Delphi, permitindo introspecção de tipos em tempo de execução sem penalidades de acoplamento.
5. **Acesso a Dados com FireDAC e TDataLink:** A ponte de comunicação bidirecional de dados mais rápida do mercado, garantindo controles reativos a transações, estados de dataset e auditoria corporativa.

Este livro foi concebido como um compêndio de engenharia de software de alto nível. Cada um dos 25 módulos contém teoria aprofundada, diagramas conceituais claros e o código Pascal 100% completo, funcional e compilável de cada componente, com suporte a pacotes de tempo de execução (`.dpk`) e registro oficial na IDE.

---

2 DIAGRAMA MACRO DA ARQUITETURA DELPHI: VCL vs. FIREMONKEY



3 SUMÁRIO ANALÍTICO DOS 50 MÓDULOS

Abaixo está o detalhamento estruturado e analítico de cada um dos 50 módulos temáticos do livro.

```
================================================================================
PARTE I: FUNDAMENTAÇÃO, RTL E CICLO DE VIDA DO COMPONENTE
================================================================================
```

3.0.1 Módulo 01: Arquitetura Orientada a Componentes e Hierarquia de Classes

- **Fundamentos Teóricos:** Genealogia técnica detalhada de `TObject`, `TPersistent`, `TComponent`, `TControl`, `TGraphicControl` e `TWinControl` na VCL, contrastando com `TFmxObject`, `TControl` e `TStyledControl` no FireMonkey.
- **Enhanced RTTI e Atributos:** Uso do namespace `System.Rtti` e criação de atributos customizados (`TCustomAttribute`) para anotação de metadados em propriedades de componentes.
- **Padrão de Projeto:** Component-Based Architecture e Composite Pattern.
- **Componente Implementado:** `TDlpTypeHierarchyInspector` — componente não-visual que mapeia a hierarquia de herança, propriedades RTTI convencionais e atributos estendidos em tempo de execução.

3.0.2 Módulo 02: O Ciclo de Vida Completo do Componente

- **Sequência de Instanciação e Destruição:** Fases determinísticas de vida: `Create`, `CreateWnd`, `Loaded`, `Notification` e `Destroy`.
- **Estados do Componente:** Análise profunda de `ComponentState` (`csDesigning`, `csLoading`, `csDestroying`, `csReadingState`, `csFreeNotification`) e `ControlState`.
- **Gestão Segura de Ponteiros:** Utilização de `FreeNotification` e implementação estrita do método `Notification` para anular referências a componentes associados antes de sua destruição física.
- **Componente Implementado:** `TDlpSafeLinkedContainer` — contêiner visual com referências fracas para múltiplos componentes do formulário, com prevenção total de Access Violation em exclusões de tempo de design.

3.0.3 Módulo 03: Mecanismo de Streaming e Persistência (DFM)

- **O Subsistema TFiler:** Funcionamento de `TReader` e `TWriter`. Diferenças entre DFM em formato texto legível e DFM binário.
- **Propriedades Customizadas com DefineProperties:** Como persistir dados complexos, estruturas de dados arbitrárias e buffers criptografados que não se enquadram em propriedades publicadas padrão.
- **Componente Implementado:** `TDlpEncryptedDataStore` — componente com capacidade de serializar dicionários de configuração diretamente no arquivo `.dfm` com compressão e criptografia embutida via `DefineProperties`.

3.0.4 Módulo 04: Tratamento de Mensagens do SO e Arquitetura de Eventos

- **Pipeline de Mensagens do Windows na VCL:** Interceptação com `WndProc`, macros de mensagem (`message WM_...`) e despacho seguro com `Perform`.
- **Arquitetura de Eventos Modernos:** Eventos baseados em métodos anônimos, closures, delegates com parâmetros tipados e integração com o loop de mensagens da aplicação.
- **Sincronização e Concorrência:** Sincronização segura entre threads de segundo plano e o componente visual via `TThread.Synchronize` e `TThread.Queue`.
- **Componente Implementado:** `TDlpWindowsMessageInterceptor` — componente de hooking visual que intercepta mensagens do SO (como redimensionamento, foco e teclas globais) e despacha eventos assíncronos protegidos para o desenvolvedor.

```
================================================================================
PARTE II: DESIGN-TIME, RTTI E INTEGRAÇÃO PROFUNDA COM A IDE
================================================================================
```

3.0.5 Módulo 05: Empacotamento Profissional (Packages .dpk)

- **Arquitetura de Pacotes no Delphi 12:** Separação obrigatória entre pacotes de tempo de execução (`DlpComponents_RT.dpk`) e pacotes de tempo de design (`DlpComponents_DT.dpk`).
- **O Procedimento Register:** Uso de `RegisterComponents` e injeção semântica em categorias da Tool Palette.
- **Recursos Gráficos Modernos:** Criação e compilação de ícones em 24×24 e 32×32 pixels via arquivos de recursos `.rc` compilados com `brcc32` em `.res`.
- **Pacotes Completos:** Estrutura completa dos arquivos de projeto de pacotes (`.dpk` e `.dproj`) prontos para compilação direta via linha de comando (`MSBuild`).

3.0.6 Módulo 06: Property Editors Customizados

- **Interfaces da Open Tools API:** Uso de `DesignIntf` e `DesignEditors`. Especialização de `TPropertyEditor`, `TClassPropertyEditor` e `TStringPropertyEditor`.
- **Atributos de Edição:** Sobrescrita de `GetAttributes` (`paDialog`, `paValueList`, `paMultiSelect`, `paSortList`).
- **Construção de Diálogo Modal:** Criação de janela de diálogo rica para configuração de uma propriedade visual no Object Inspector.
- **Componente e Editor:** `TDlpGradientHeader` e seu respectivo `TDlpGradientPropertyEditor` com editor visual de cores e paradas de gradiente.

3.0.7 Módulo 07: Component Editors e Ações de Design-Time (Verbos)

- **Extensão de TComponentEditor:** Sobrescrita de `GetVerbCount`, `GetVerb` e `ExecuteVerb` para injeção de opções no menu de contexto do formulário.
- **Ação Padrão (Duplo-clique):** Sobrescrita do método `Edit` para disparar um assistente interativo ou editor modal de layout.

- **Componente e Editor:** `TDlpDashboardCard` e seu editor `TDlpDashboardCardEditor` provendo verbos como “Configurar Métricas...”, “Alternar Tema do Card” e “Exportar Layout...”.

3.0.8 Módulo 08: Assistentes de Projeto e Extensão da IDE (ToolsAPI Profunda)

- **Arquitetura da Open Tools API (OTA):** Estrutura de `IOTAWizard`, `IOTAMenuWizard`, `IOTARepositoryWizard` e `IOTAModuleServices`.
- **Criação de Wizards de Projeto:** Construção de assistente corporativo que gera um projeto completo seguindo a arquitetura em camadas (DDD / Clean Architecture) com um clique no menu da IDE.
- **Manipulação de Buffers de Código:** Injeção programática de código-fonte nas unidades ativas do editor de código do Delphi 12.
- **Wizard Completo:** `TDlpCleanArchProjectWizard` — assistente completo integrado à ToolsAPI do Delphi 12 Athens.

```
================================================================================
PARTE III: CONTROLES VISUAIS, GRÁFICOS E INTERFACE MODERNA
================================================================================
```

3.0.9 Módulo 09: Controles Visuais Leves (TGraphicControl na VCL / FMX Primitives)

- **Renderização de Alta Fidelidade:** Desenho vetorial no `Canvas`, suporte a Direct2D e GDI+ na VCL, e `TCanvas` acelerado por GPU no FireMonkey.
- **Double Buffering e Performance:** Eliminação de cintilações (*flicker*) através de double-buffering em memória.
- **Projeto Completo:** `TDlpAnalogGauge` (VU Meter Analógico) — controle com escalas dinâmicas, ponteiro animado por interpolação suave, zonas de perigo coloridas e alta resolução.

3.0.10 Módulo 10: Controles Visuais Janelados (TWinControl na VCL / TStyledControl no FMX)

- **Gerenciamento de Janelas e Contêineres:** Criação de `HWND` nativo via `CreateParams`, gerenciamento de foco e tabulação.
- **Suporte a Controles Embutidos em Tempo de Design:** Habilitação de `csAcceptsControls` e gerenciamento de filhos.
- **Projeto Completo:** `TDlpAccordionContainer` — contêiner sanfonado expansível com animação de desdobramento fluida e suporte total ao designer de formulários.

3.0.11 Módulo 11: Coleções Tipadas e Sub-Propriedades Complexas

- **Especialização de TCollection e TCollectionItem:** Ciclo de vida e serialização DFM de listas de objetos complexos.
- **Sincronização com a UI:** Propagação de eventos de alteração de itens para repintura otimizada do controle pai.
- **Projeto Completo:** `TDlpCustomTabBar` — barra de abas moderna com abas dinâmicas, botões de fechar individuais, suporte a `TImageList` e arraste para reordenar.

3.0.12 Módulo 12: High-DPI, Escalonamento e Suporte a Temas Modernos

- **Adaptação a Monitores 4K/UHD:** Suporte completo a **Per-Monitor v2**, sobrescrita de `ChangeScale` e `ScaleForPPI`.
- **Estilos VCL e Dark Mode:** Integração com `TStyleManager` e `TThemeServices` para responder dinamicamente a mudanças de tema do Windows 11.
- **Ícones Vetoriais SVG:** Integração com bibliotecas de ícones SVG (`TSVGIconImageList`) para escalonamento perfeito em qualquer nível de DPI (100% a 400%).
- **Componente Atualizado:** Adaptação completa dos componentes visuais para Per-Monitor v2 e suporte a estilos claros/escuros.

```
================================================================================
PARTE IV: ACESSO A DADOS E COMPONENTES DB-AWARE
================================================================================
```

3.0.13 Módulo 13: Arquitetura de Dados no Pascal e a Ponte TDataLink

- **Subsistema de Dados do Delphi:** Relação íntima entre `TDataSet`, `TField`, `TDataSource` e a UI.
- **Subclasse TDataLink:** Como monitorar eventos de navegação, edição e ativação de datasets.
- **Sincronização Bidirecional:** Implementação de leitura e escrita com validação de campos via `TFieldDataLink`.
- **Framework Base:** Classe abstrata de base para controles DB-Aware de alta performance.

3.0.14 Módulo 14: Controles DB-Aware Básicos de Exibição e Edição

- **Implementação Prática 1:** `TDBBadgeLabel` — label de status reativo a dados com renderização condicional de cores e texto de acordo com o valor do registro.
- **Implementação Prática 2:** `TDBNumberEdit` — caixa de texto numérica e monetária com formatação automática de moeda, entrada de decimais alinhada à direita e transição automática para o modo `dsEdit` ao teclar.

3.0.15 Módulo 15: Controles DB-Aware Avançados e Seletores

- **Resolução de Dados Relacionais:** Manipulação de tabelas associativas e integridade referencial.
- **Projeto Completo:** `TDBLookupTagBox` — controle de seleção múltipla estilo tag-cloud, com pesquisa dinâmica, adição e remoção de tags visuais conectadas a um dataset de ligação.

3.0.16 Módulo 16: Monitoramento e Gerenciador de Transações de Dados

- **Auditoria de Conexões FireDAC:** Monitoramento de instâncias de `TFDConnection` e `TFDQuery`.
- **Projeto Completo:** `TDlpDataConnectionMonitor` — componente não-visual com rastreamento de tempo de execução de queries SQL, detecção de transações abertas esquecidas e cancelamento de comandos em timeout.

```
================================================================================
PARTE V: MOTORES DRAG & DROP E GERADORES AUTOMÁTICOS DE CRUD
================================================================================
```

3.0.17 Módulo 17: Engenharia de Arrastar e Soltar (Drag & Drop) no Designer

- **Interação com a IDE no Designer:** Captura do arraste de tabelas e campos a partir do Data Explorer do Delphi para a superfície do formulário.
- **Introspecção de Metadados:** Extração dinâmica de informações de colunas (tipos, precisão, obrigatoriedade) via instâncias ativas de datasets ou RTTI.
- **Tabela de Correspondência de Tipos:** Motor de mapeamento configurável.

3.0.18 Módulo 18: Construção do Componente Gerador de Cadastros (TAutoFormBuilder)

- **Arquitetura do Gerador:** Componente contêiner `TDlpAutoFormBuilder`.
- **Mapeamento Algorítmico de Controles:**
  - `ftString`, `ftWideString` → Label e `TDBEdit`.
  - `ftCurrency`, `ftBCD`, `ftFloat` → `TDBNumberEdit`.
  - `ftDate`, `ftDateTime` → `TDateTimePicker`.
  - `ftBoolean` → `TToggleSwitch` / Checkbox.
  - `ftBlob`, `ftGraphic` → Painel de imagem com suporte a drag-and-drop de arquivos locais.
- **Geração Física de Componentes:** Instanciação de componentes com atribuição de proprietário (`Owner`), pai (`Parent`) e persistência imediata no arquivo `.dfm`.

3.0.19 Módulo 19: Caso de Estudo Real 1 — Gerador do Cadastro Completo de Clientes

- **Sistema Real Completo:** Construção automatizada de tela de cadastro corporativo de clientes.
- **Validações de Domínio:** Validação em tempo de digitação de CPF e CNPJ com algoritmos de dígito verificador integrados.
- **Barra de Controle de Operações:** Componente com botões estilizados e gerenciamento de máquina de estados (Navegação, Inserção, Alteração, Confirmação, Exclusão).

3.0.20 Módulo 20: Caso de Estudo Real 2 — Gerador do Cadastro de Produtos e Itens

- **Sistema Real Completo:** Geração automática de catálogo de produtos com cálculo reativo em tempo real de margem de lucro (Preço de Custo vs. Preço de Venda).
- **Indicadores de Estoque:** Alertas gráficos de reposição e estoque crítico.
- **Carrossel de Fotos:** Componente de visualização de múltiplas imagens do item com transições visuais.

```
================================================================================
PARTE VI: COMPONENTES DE SISTEMA, REDE E INTEGRAÇÕES EXTERNAS
================================================================================
```

3.0.21 Módulo 21: Componentes de Execução Assíncrona e Worker Threads

- **Concorrência Moderna:** Uso da biblioteca `System.Threading` (Parallel Programming Library - PPL) e tarefas (`ITask`).
- **Projeto Completo:** `TDlpAsyncTaskRunner` — componente não-visual com cancelamento cooperativo via `TCancellationToken`, controle de concorrência máxima e notificações protegidas na UI thread via `TThread.Queue`.

3.0.22 Módulo 22: Componentes de Comunicação de Rede e Consumo de APIs

- **Consumo de APIs REST no Delphi:** Uso do framework nativo `System.Net.HttpClient` e `REST.Json`.
- **Projeto Completo:** `TDlpRestClientBridge` — componente não-visual para requisições assíncronas com tratamento de autenticação JWT/Bearer, headers dinâmicos e conversão direta entre payloads JSON e objetos de domínio.

3.0.23 Módulo 23: Componentes de Hardware, I/O e Comunicação Serial

- **Comunicação de Baixo Nível no Windows:** Uso das APIs do Windows para comunicação via portas COM / portas virtuais USB.
- **Projeto Completo:** `TDlpSerialDeviceConnector` — componente de I/O industrial com leitura contínua assíncrona, tratamento de checksum e protocolos nativos para balanças comerciais de checkout e leitores de código de barras.

```
================================================================================
PARTE VII: TESTES, OTIMIZAÇÃO E DISTRIBUIÇÃO GLOBAL
================================================================================
```

3.0.24 Módulo 24: Testes Unitários de Componentes, Profiling e Prevenção de Leaks

- **Automação com DUnitX:** Criação de suítes de testes unitários para validar criação, clone por streaming e destruição de componentes.
- **Depuração de Memória com FastMM5:** Configuração do FastMM5 para detecção de vazamentos de memória, corrupção de heap e blocos órfãos.
- **Otimização de Renderização:** O padrão `BeginUpdate` / `EndUpdate` e uso estratégico de `LockWindowUpdate` para anular repinturas supérfluas.

3.0.25 Módulo 25: Publicação, Versionamento e Distribuição nos Ecossistemas Oficiais

- **Estruturação de Repositórios Profissionais:** Estrutura corporativa com suporte a múltiplas versões do Delphi.
- **Publicação no GetIt Package Manager:** Estruturação dos metadados JSON do GetIt e scripts de instalação automatizada.
- **Automação de CI/CD:** Pipelines de integração contínua com GitHub Actions executando compilações em batch via `MSBuild` para Win32 e Win64.

```
================================================================================
PARTE VIII: COMPONENTES GRÁFICOS AVANÇADOS, DASHBOARDS E VISUALIZAÇÃO
================================================================================
```

3.0.26 Módulo 26: Gráficos de Alta Densidade em Tempo Real (`TDlpRealtimeChartEngine`)

- **Buffer Circular de 100k Pontos:** Gestão eficiente de memória e decimação adaptativa de amostragem.
- **Renderização Fluida VCL:** Aceleração GDI com double-buffering e prevenção de cintilação (*flicker-free*).

3.0.27 Módulo 27: Medidores de Nível e Tanques Industriais (`TDlpIndustrialTankGauge`)

- **Simulação Visual de Fluidos:** Gradientes lineares adaptativos, escala de capacidade e alertas de transbordamento.
- **Geometria Industrial VCL:** Paredes cilíndricas, tubulações e marcadores operacionais com formatação de volume.

3.0.28 Módulo 28: Gráficos de Radar e Donut Vetorial (`TDlpVectorDonutChart`)

- **Trigonometria Polar:** Varredura angular por fatia, orifício central com exibição de totais e renderização vetorial.
- **Hit-Testing Polar:** Algoritmo euclidiano com `ArcTan2` para detecção de hover de alta precisão.

3.0.29 Módulo 29: Visualizador de Mapas de Calor (`TDlpMatrixHeatmap`)

- **Termografia de Matrizes 2D:** Interpolação linear contínua em espaço de cor RGB com auto-ajuste de mínimos e máximos.
- **Contraste Tipográfico ITU-R BT.601:** Seleção automática de fonte branca ou preta conforme luminância da célula.

```
================================================================================
PARTE IX: RELATÓRIOS, EXPORTAÇÃO E PROCESSAMENTO DE ARQUIVOS
================================================================================
```

3.0.30 Módulo 30: Motor Gerador de PDFs e Documentos Financeiros (`TDlpFastPdfReportBuilder`)

- **Geração PDF Nativa 1.4:** Catálogo, árvore de páginas, operadores PostScript vetoriais e fontes Type1 sem DLLs externas.
- **Estruturação Binária:** Compilação exata da tabela `xref` e emissão de extratos financeiros zebrados.

3.0.31 Módulo 31: Pipeline de Exportação em Segundo Plano (`TDlpDataExportPipeline`)

- **Streaming Assíncrono Multithread:** Despejo de instâncias de `TDataSet` para CSV (RFC 4180), JSONL e planilhas XML sem travar a interface.
- **Notificação e Cancelamento:** Sincronização cooperativa com a UI thread e tratamento de interrupções.

3.0.32 Módulo 32: Processador e Validador Fiscal XML (`TDlpFiscalXmlValidator`)

- **Auditoria Estrutural de DF-e:** Validação matemática do dígito verificador Módulo 11 da chave de acesso de 44 dígitos.
- **Integridade Criptográfica:** Extração do bloco `<infNFe>`, cômputo de hash SHA-1 e comparação com o `DigestValue`.

3.0.33 Módulo 33: Containers ZIP Criptografados (`TDlpEncryptedZipArchive`)

- **Manipulação Binária PKWARE:** Cabeçalhos locais, Central Directory e EOCD com auditoria de redundância cíclica CRC-32.
- **Cifra Simétrica e Progresso:** Proteção por senha em streaming contínuo com acompanhamento desacoplado.

```
================================================================================
PARTE X: SEGURANÇA, CRIPTOGRAFIA E CONTROLE DE ACESSO
================================================================================
```

3.0.34 Módulo 34: Validador e Emissor de Tokens JWT (`TDlpJwtSecurityManager`)

- **Padrão RFC 7519:** Assinatura e checagem HMAC-SHA256, codificação Base64URL sem padding e validação de claims (`exp`, `iat`, `sub`).
- **Mitigação de Timing Attacks:** Comparação de strings em tempo constante e lista de revogação de tokens em memória.

3.0.35 Módulo 35: Cofre Seguro de Credenciais e Segredos (`TDlpCryptoVaultStorage`)

- **Blindagem de Memória RAM:** Proteção contra Memory Scraping e crash dumps com máscara dinâmica e chave volátil de processo.
- **ZeroMemory Garantido:** Destruição determinística de buffers de senhas e persistência criptografada em disco.

3.0.36 Módulo 36: Motor de Permissões Baseadas em Papéis (`TDlpSecurityRoleManager`)

- **Arquitetura RBAC Declarativa:** Varredura recursiva de árvores de componentes em formulários e contêineres VCL.
- **Ações Contextuais de Restrição:** Ocultar (`uaHide`), desabilitar (`uaDisable`), mascarar (`uaMask`) ou travar edição (`uaReadOnly`).

3.0.37 Módulo 37: Auditor de Integridade de Executáveis (`TDlpIntegrityWatchdog`)

- **Anti-Tampering em Camadas:** Checksum SHA-256 do próprio executável em disco contra assinaturas autorizadas.
- **Detecção de Depuração:** Identificação de depuradores ativos no sistema operacional e varredura de breakpoints `0xCC` (INT 3).

```
================================================================================
PARTE XI: COMUNICAÇÃO DE DADOS, MENSAGERIA E WEBHOOKS
================================================================================
```

3.0.38 Módulo 38: Cliente de Mensageria IoT via MQTT (`TDlpMqttClientBridge`)

- **Padrão OASIS MQTT 3.1.1:** Enquadramento binário de pacotes CONNECT, CONNACK, PUBLISH, SUBSCRIBE com suporte a QoS 0, 1 e 2.
- **Codificação de Comprimento Remanescente:** Algoritmo VarInt de 7 bits e suporte a tópicos com retenção.

3.0.39 Módulo 39: Servidor de Webhooks Embutido (`TDlpEmbeddedWebhookServer`)

- **Microservidor HTTP Nativo:** Receptor TCP autônomo baseado em sockets para callbacks instantâneos de gateways de pagamento (Pix, Stripe).
- **Desacoplamento e Baixa Latência:** Resposta imediata HTTP 200 OK e parsing de cabeçalhos de assinatura criptográfica.

3.0.40 Módulo 40: Cliente WebSocket com Reconexão Automática (`TDlpWebSocketLiveClient`)

- **Conexão Full-Duplex RFC 6455:** Handshake HTTP Upgrade, frames mascarados client-side e streaming bidirecional.
- **Fila Offline e Backoff Exponencial:** Enfileiramento de mensagens em quedas de rede e reconexão resiliente progressiva.

3.0.41 Módulo 41: Hub de Notificações Push Desktop/Mobile (`TDlpPushNotificationHub`)

- **Concentrador Multiplataforma:** Formatação canônica de payloads para Google Firebase FCM HTTP v1, Apple APNs e WebPush.
- **Priorização e Dados Customizados:** Suporte a prioridades normal/high, sons, badges e metadados tipados.

```
================================================================================
PARTE XII: INTELIGÊNCIA ARTIFICIAL LOCAL E SERVIÇOS COGNITIVOS
================================================================================
```

3.0.42 Módulo 42: Dispatcher de Prompts para LLMs Locais (`TDlpAiPromptDispatcher`)

- **Inferência Soberana Local:** Conexão assíncrona com Ollama e modelos de código aberto via `/api/generate`.
- **Streaming de Tokens e Métricas:** Emissão contínua token por token na GUI e cálculo de taxa de geração (tokens/segundo).

3.0.43 Módulo 43: Parser de Respostas Estruturadas JSON (`TDlpAiStructuredExtractor`)

- **Sanitização de Outputs de IA:** Eliminação de fences Markdown (```json) e reparo de imperfeições sintáticas de LLMs.

- **Hidratação Tipada:** Mapeamento determinístico de saídas probabilísticas diretamente em records fortemente tipados.

3.0.44 Módulo 44: Motor de Busca Vetorial e Embeddings (`TDlpVectorEmbeddingStore`)

- **Banco Vetorial em Memória:** Armazenamento denso de embeddings de 384 a 1536 dimensões com normalização de módulo.
- **Similaridade por Cosseno e Top-K:** Busca semântica acelerada para catálogos de produtos e arquiteturas de RAG local.

```
================================================================================
PARTE XIII: COMPONENTES DE SISTEMA, SERVIÇOS E DIAGNÓSTICO
================================================================================
```

3.0.45 Módulo 45: Construtor Universal de Daemons e Serviços (`TDlpSystemDaemonController`)

- **Arquitetura Unificada Headless:** Controle de ciclo de vida de Daemons Linux (Systemd) e Serviços Windows (SCM).
- **Geração Automatizada:** Emissão de arquivo `.service`, comandos `--install`/`--uninstall` e modo interativo `--debug`.

3.0.46 Módulo 46: Sonda de Monitoramento de Sistema (`TDlpSystemMetricsProbe`)

- **Telemetria de Recursos:** Amostragem periódica de consumo percentual de CPU, contagem de handles e threads ativas.
- **Diagnóstico de Memória:** Coleta de Working Set e Private Bytes para prevenção de vazamentos de recursos.

3.0.47 Módulo 47: Gerenciador de Pool de Conexões SQL (`TDlpDatabaseConnectionPool`)

- **Pooling Thread-Safe:** Fila de conexões ativas com `TCriticalSection`, alocação sob demanda e limites mínimo/máximo.
- **Expurgo de Zumbis:** Eliminação preventiva de conexões inativas por timeout e prevenção de esgotamento no SGBD.

3.0.48 Módulo 48: Motor de Atualização Automática (`TDlpAutoUpdateManager`)

- **Distribuição Over-The-Air (OTA):** Consulta a manifestos remotos JSON e comparação hierárquica SemVer.
- **Auditoria e Substituição Atômica:** Validação de checksum SHA-256 e script auxiliar para reinicialização transparente.

```
================================================================================
PARTE XIV: QUALIDADE TOTAL, BENCHMARKS E DISTRIBUIÇÃO AVANÇADA
================================================================================
```

3.0.49 Módulo 49: Harness de Micro-Benchmarks de Código (`TDlpCodeBenchmarkHarness`)

- **Medição de Alta Resolução (QPC):** Temporização em nível de microssegundos via hardware com aquecimento prévio de caches L1/L2.
- **Estatística Comparativa:** Relatórios tabulares com média de tempo por operação e throughput em operações/segundo.

3.0.50 Módulo 50: Pipeline de CI/CD Multiplataforma e Releases no GitHub (`TDlpReleasePipeline`)

- **Automação DevOps Completa:** Esteira de compilação sem interface gráfica via `MSBuild` e testes automatizados.
- **Publicação de Releases:** Inventário de artefatos, cômputo de hashes SHA-256 e geração de Changelog Markdown.

---

4 GUIA DE ESTILO E DIRETRIZES DE CODIFICAÇÃO

Todos os componentes do livro seguem estritamente as diretrizes de engenharia de software da Embarcadero: 1. Nomes de tipos e classes com prefixo `T` (ex: `TDlpAnalogGauge`). 2. Isolamento estrito de dependências visuais nos pacotes de runtime para permitir compilações limpas de aplicações server/headless quando aplicável. 3. Tratamento rigoroso de exceções com blocos `try..finally` e `try..except`. 4. Documentação XML-Doc padrão em todas as interfaces públicas.

5 MÓDULO 01: Arquitetura Orientada a Componentes e Hierarquia de Classes

No **Delphi 12 Athens+**, o desenvolvimento de componentes atingiu um patamar de maturidade e sofisticação sem precedentes. O arquiteto de software Object Pascal opera hoje na interseção entre duas grandes bibliotecas visuais — a tradicional e ultraotimizada **VCL (Visual Component Library)** para Windows e o revolucionário framework multiplataforma acelerado por hardware **FireMonkey (FMX)** —, amparadas por um dos compiladores nativos mais velozes do mundo e um sistema moderno de **Enhanced RTTI** enriquecido por atributos customizados.

Neste módulo inaugural, analisaremos a espinha dorsal da hierarquia de classes do Delphi moderno, dissecaremos as semelhanças e contrastes radicais entre VCL e FireMonkey, e implementaremos um componente profissional de introspecção profunda baseado na RTTI contemporânea do Delphi 12.

5.1 1. A Dualidade de Frameworks: VCL vs. FireMonkey

Ao contrário de outros ecossistemas que descartam bibliotecas maduras para forçar a migração a frameworks novos, o Delphi mantém a VCL como o padrão-ouro imbatível para aplicações Windows corporativas, enquanto oferece o FireMonkey para cenários multiplataforma (Windows, macOS, iOS, Android e Linux).

A compreensão das raízes genealógicas de cada framework é indispensável:

```
classDiagram
    class TObject {
        <<RTL Core>>
        +Create()
        +Destroy()
        +Free()
        +ClassType()
    }
    class TPersistent {
        <<RTL Core>>
        +Assign()
        +AssignTo()
        +DefineProperties()
    }
    class TComponent {
        <<RTL Core>>
        +Owner: TComponent
        +ComponentState
        +Notification()
        +FreeNotification()
    }

    TObject <|-- TPersistent
    TPersistent <|-- TComponent

    %% Ramo VCL
    TComponent <|-- Vcl_TControl : VCL
    Vcl_TControl <|-- Vcl_TGraphicControl
    Vcl_TControl <|-- Vcl_TWinControl

    %% Ramo FMX
    TComponent <|-- Fmx_TFmxObject : FireMonkey
    Fmx_TFmxObject <|-- Fmx_TControl
    Fmx_TControl <|-- Fmx_TStyledControl

    class Vcl_TControl {
        <<Vcl.Controls>>
        +Parent: TWinControl
        +Left, Top, Width, Height
    }
    class Vcl_TGraphicControl {
        <<Vcl.Controls>>
        +Canvas: TCanvas
        <<Windowless / Sem HWND>>
    }
    class Vcl_TWinControl {
        <<Vcl.Controls>>
        +Handle: HWND
        +WndProc()
        <<Handle Nativo Win32/64>>
    }

    class Fmx_TFmxObject {
        <<FMX.Types>>
        +Children: TFmxChildrenList
        +Parent: TFmxObject
    }
    class Fmx_TControl {
        <<FMX.Controls>>
        +Position: TPosition (X, Y)
        +RotationAngle: Single
        +Opacity: Single
        <<Renderização Vetorial GPU>>
    }
    class Fmx_TStyledControl {
        <<FMX.Controls>>
        +StyleLookup: string
        +ApplyStyle()
```

```
        <<Desacoplado por Estilos>>
    }
```

5.1.1 1.1 O Núcleo Comum da RTL: `TObject`, `TPersistent` e `TComponent`

Independentemente de você estar projetando para VCL ou FireMonkey, as três primeiras classes da árvore são universais e idênticas:

1. `TObject` (`System`): Raiz do universo Pascal. Alocada estritamente no Heap. Contém o ponteiro de 4/8 bytes no cabeçalho apontando para a **VMT (Virtual Method Table)**. Define o ciclo primitivo `Create` / `Destroy` / `Free`, e os métodos de introspecção clássicos (`ClassType`, `ClassName`, `InheritsFrom`).
2. `TPersistent` (`System.Classes`): Habilita explicitamente a diretiva de RTTI clássica (`{$M+}`). Introduz o contrato de cópia canônica profunda (`Assign` e `AssignTo`) e a capacidade de streaming customizado com `DefineProperties`.
3. `TComponent` (`System.Classes`): O elemento atômico do desenvolvimento visual e não-visual no Delphi. Introduz o conceito de **Ownership** (`AOwner`), serialização para arquivos de formulário (`.dfm` e `.fmx`), flags de estado de tempo de design (`ComponentState`), e o sistema de cancelamento de referências cruzadas (`Notification` / `FreeNotification`).

5.1.2 1.2 Comparativo Arquitetural: VCL vs. FireMonkey

Abaixo está o confronto técnico das duas filosofias de interface gráfica:

Dimensão Técnica	VCL (Visual Component Library)	FireMonkey (FMX)
Público / Alvo	Windows Nativo (Win32 / Win64)	Multiplataforma (Windows, macOS, iOS, Android, Linux)
Pipeline de Renderização	Baseado em GDI, GDI+, Direct2D e APIs nativas do Windows.	100% acelerado por GPU (DirectX no Windows, Metal no macOS/iOS, OpenGL/Vulkan no Android/Linux).
Controle de Janelas e Handles	`TWinControl` encapsula diretamente um `HWND` da API do Windows.	Praticamente livre de `HWND`. Apenas a janela raiz (`TCommonCustomForm`) possui handle; os controles internos são vetores gráficos em uma única superfície acelerada.
Geometria e Coordenadas	Inteiros (`Left`, `Top`, `Width`, `Height : Integer`).	Ponto Flutuante (`Position.X`, `Position.Y`, `Width`, `Height : Single`).
Transformações Visuais	Complexas e limitadas (requer transformações de matriz de device context GDI).	Nativas em cada controle (`RotationAngle`, `RotationCenter`, `Scale`, `Opacity`).
Arquitetura Visual	Mistura de controles gráficos leves (`TGraphicControl`) e janelados (`TWinControl`).	Quase todos os controles visuais herdam de `TStyledControl`, separando estritamente lógica de apresentação via `StyleLookup`.
Suporte a High-DPI	**Per-Monitor v2** com redimensionamento de fontes e escalas escalares.	Nativamente vetorial; renderiza com nitidez automática em qualquer densidade de pixels (Retina, 4K, 8K).

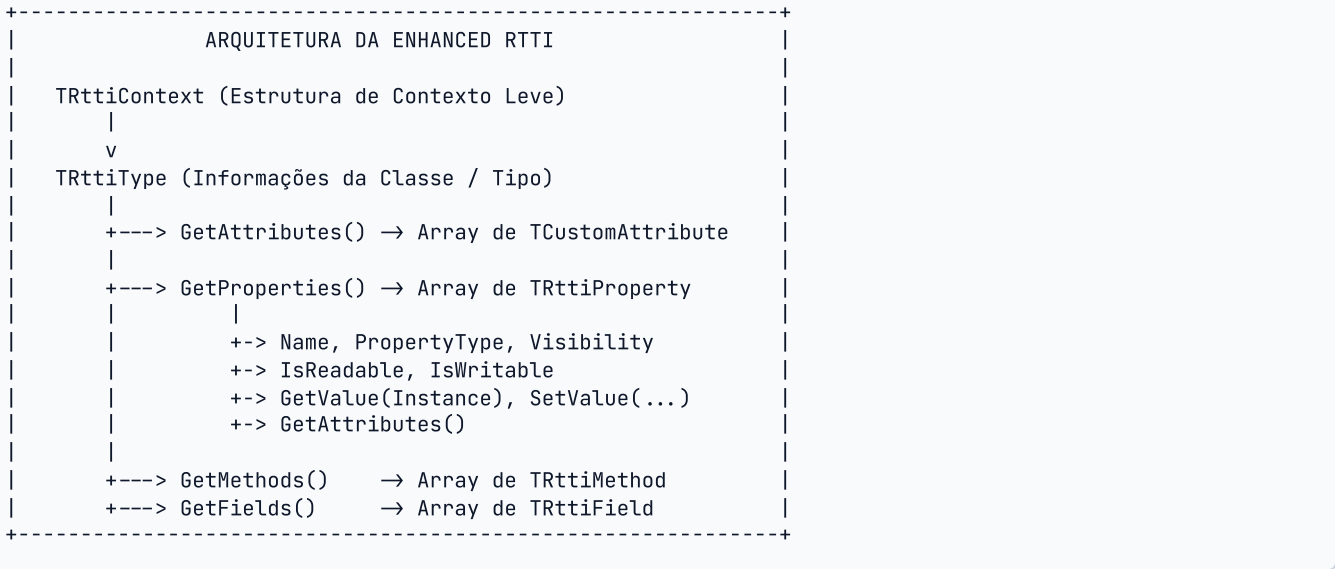
5.2 2. A Evolução da RTTI: De `TypInfo` ao `System.Rtti` e Atributos Customizados

Historicamente, o Delphi dependia exclusivamente da unidade `System.TypInfo`, que disponibilizava funções procedurais em linguagem C de baixo nível (`GetPropList`, `GetTypeData`, `GetPropInfo`) restritas exclusivamente a métodos e propriedades marcados explicitamente como `published`.

A partir das versões modernas e consolidada com maestria no **Delphi 12 Athens+**, a Embarcadero introduziu e aperfeiçoou o framework **Enhanced RTTI** no namespace `System.Rtti`.

5.2.1 2.1 Por que a Enhanced RTTI Revolucionou o Delphi?

1. **Introspecção Completa:** Não está limitada à seção `published`. Pode inspecionar campos, propriedades e métodos `private`, `protected` e `public` (conforme a diretiva `{$RTTI}`).
2. **Orientação a Objetos Pura:** Substitui ponteiros opacos e estruturas C por classes ricas como `TRttiContext`, `TRttiType`, `TRttiProperty`, `TRttiMethod` e `TRttiField`.
3. **Atributos Customizados (`TCustomAttribute`):** Capacidade de anotar classes, propriedades e métodos com metadados decorativos legíveis em tempo de execução, abrindo as portas para validação declarativa, serializadores JSON/ORM sem acoplamento e injeção de dependência moderna.



5.3 3. Implementação Prática: O Componente `TDlpTypeHierarchyInspector`

Para exercitar a arquitetura do Delphi 12, construímos o componente `TDlpTypeHierarchyInspector`. Ele realiza: 1. Inspeção da árvore de herança de qualquer objeto ou controle até `TObject`. 2. Extração de todas as propriedades do componente através de `System.Rtti`, incluindo propriedades `public` e `published`. 3. Descoberta e decodificação dinâmica de **Atributos Customizados** (`ComponentDocAttribute`). 4. Proteção estrita contra dangling pointers via `FreeNotification` e `Notification`.

5.3.1 3.1 Código Pascal Completo e Compilável

Abaixo está o código-fonte integral da unidade `DlpTypeHierarchyInspector.pas`:

```
{================================================================================
  Livro: Desenvolvimento Moderno de Componentes no Delphi: VCL, FireMonkey e ToolsAPI
  Módulo 01: Arquitetura Orientada a Componentes e Hierarquia de Classes
  Componente: TDlpTypeHierarchyInspector
  Autor: Engenharia de Software Especializada & Arquitetura RAD
  Ambiente: Delphi 12 Athens+ (Win32, Win64 e Multiplataforma)
  Descrição: Componente não-visual que inspeciona a hierarquia de herança,
             a RTTI clássica (TypInfo) e a RTTI estendida moderna (System.Rtti),
             incluindo atributos customizados anotados em classes e propriedades.
================================================================================}
unit DlpTypeHierarchyInspector;

interface

uses
  System.SysUtils,
  System.Classes,
  System.TypInfo,
  System.Rtti;

type
  { Atributo customizado para demonstrar anotação semântica em componentes }
  ComponentDocAttribute = class(TCustomAttribute)
  private
    FDescription: string;
    FVersion: string;
  public
    constructor Create(const ADescription, AVersion: string);
    property Description: string read FDescription;
    property Version: string read FVersion;
  end;

  { Nível de profundidade de inspeção de herança }
  TDlpInspectionDepth = (
    idImmediateOnly,    // Apenas a classe alvo
    idUntilComponent,   // Até TComponent
    idFullToTObject     // Até a raiz universal TObject
  );

  { Registro com metadados de propriedade obtidos via System.Rtti }
  TDlpPropertyExtendedMeta = record
    PropName: string;
    TypeName: string;
    Visibility: string;
    IsReadable: Boolean;
    IsWritable: Boolean;
    CustomAttributeCount: Integer;
    AttributesSummary: string;
  end;

  { Array dinâmico com os metadados das propriedades }
  TDlpPropertyExtendedMetaArray = TArray<TDlpPropertyExtendedMeta>;

  { Eventos para callbacks e reatividade na UI }
  TOnClassNodeFound = procedure(Sender: TObject; const AClassName: string;
    AClassRef: TClass; ADepth: Integer) of object;
  TOnPropertyInspected = procedure(Sender: TObject;
```

```
    const AMeta: TDlpPropertyExtendedMeta) of object;

  {==============================================================================
    Classe: TDlpTypeHierarchyInspector
    Componente não-visual projetado para Delphi 12 Athens+
  ==============================================================================}
  [ComponentDoc('Inspetor avançado de RTTI estendida e genealogia para Delphi 12', '1.0.0')]
  TDlpTypeHierarchyInspector = class(TComponent)
  private
    FTargetComponent: TComponent;
    FInspectionDepth: TDlpInspectionDepth;
    FIncludePrivateProps: Boolean;
    FOnClassNodeFound: TOnClassNodeFound;
    FOnPropertyInspected: TOnPropertyInspected;

    procedure SetTargetComponent(const AValue: TComponent);
    function VisibilityToString(AVis: TMvVisibility): string;
  protected
    procedure Notification(AComponent: TComponent; Operation: TOperation); override;
  public
    constructor Create(AOwner: TComponent); override;
    destructor Destroy; override;

    { Métodos de extração de relatórios e inspeção via RTTI }
    function GetClassAncestry(AInstance: TObject = nil): TStrings;
    function GetExtendedProperties(AInstance: TObject = nil): TDlpPropertyExtendedMetaArray;
    function GenerateMarkdownDocumentation(AInstance: TObject = nil): string;
  published
    property TargetComponent: TComponent read FTargetComponent write SetTargetComponent;
    property InspectionDepth: TDlpInspectionDepth read FInspectionDepth write FInspectionDepth
        default idFullToTObject;
    property IncludePrivateProps: Boolean read FIncludePrivateProps write FIncludePrivateProps
        default False;

    property OnClassNodeFound: TOnClassNodeFound read FOnClassNodeFound write
        FOnClassNodeFound;
    property OnPropertyInspected: TOnPropertyInspected read FOnPropertyInspected write
        FOnPropertyInspected;
  end;

{ Procedimento oficial de registro na Tool Palette do Delphi }
procedure Register;

implementation

procedure Register;
begin
  RegisterComponents('Engenharia Delphi 12', [TDlpTypeHierarchyInspector]);
end;

{ ComponentDocAttribute }

constructor ComponentDocAttribute.Create(const ADescription, AVersion: string);
begin
  inherited Create;
  FDescription := ADescription;
  FVersion := AVersion;
end;
```

```
{ TDlpTypeHierarchyInspector }

constructor TDlpTypeHierarchyInspector.Create(AOwner: TComponent);
begin
  inherited Create(AOwner);
  FTargetComponent := nil;
  FInspectionDepth := idFullToTObject;
  FIncludePrivateProps := False;
end;

destructor TDlpTypeHierarchyInspector.Destroy;
begin
  FTargetComponent := nil;
  inherited Destroy;
end;

procedure TDlpTypeHierarchyInspector.SetTargetComponent(const AValue: TComponent);
begin
  if FTargetComponent <> AValue then
  begin
    if Assigned(FTargetComponent) then
      FTargetComponent.RemoveFreeNotification(Self);

    FTargetComponent := AValue;

    if Assigned(FTargetComponent) then
      FTargetComponent.FreeNotification(Self);
  end;
end;

procedure TDlpTypeHierarchyInspector.Notification(AComponent: TComponent; Operation:
        TOperation);
begin
  inherited Notification(AComponent, Operation);
  { Proteção contra referências pendentes ao deletar componentes no form designer }
  if (Operation = opRemove) and (AComponent = FTargetComponent) then
    FTargetComponent := nil;
end;

function TDlpTypeHierarchyInspector.VisibilityToString(AVis: TMvVisibility): string;
begin
  case AVis of
    mvPrivate:   Result := 'private';
    mvProtected: Result := 'protected';
    mvPublic:    Result := 'public';
    mvPublished: Result := 'published';
  else
    Result := 'unknown';
  end;
end;

function TDlpTypeHierarchyInspector.GetClassAncestry(AInstance: TObject): TStrings;
var
  CurrentObj: TObject;
  CurrentClass: TClass;
  Depth: Integer;
  ShouldStop: Boolean;
```

```
begin
  Result := TStringList.Create;
  try
    CurrentObj := AInstance;
    if not Assigned(CurrentObj) then
      CurrentObj := FTargetComponent;

    if not Assigned(CurrentObj) then
    begin
      Result.Add('Nenhum componente ou objeto selecionado para inspeção.');
      Exit;
    end;

    CurrentClass := CurrentObj.ClassType;
    Depth := 0;

    while Assigned(CurrentClass) do
    begin
      Result.Add(Format('[Nível %d] %s (Endereço VMT: %p)',
        [Depth, CurrentClass.ClassName, Pointer(CurrentClass)]));

      if Assigned(FOnClassNodeFound) then
        FOnClassNodeFound(Self, CurrentClass.ClassName, CurrentClass, Depth);

      Inc(Depth);

      ShouldStop := False;
      case FInspectionDepth of
        idImmediateOnly:
          ShouldStop := True;
        idUntilComponent:
          if CurrentClass = TComponent then
            ShouldStop := True;
        idFullToTObject:
          if CurrentClass = TObject then
            ShouldStop := True;
      end;

      if ShouldStop then
        Break;

      CurrentClass := CurrentClass.ClassParent;
    end;
  except
    on E: Exception do
      Result.Add('Falha durante introspecção de classe: ' + E.Message);
  end;
end;

function TDlpTypeHierarchyInspector.GetExtendedProperties(AInstance: TObject):
        TDlpPropertyExtendedMetaArray;
var
  CurrentObj: TObject;
  RttiContext: TRttiContext;
  RttiType: TRttiType;
  RttiProp: TRttiProperty;
  RttiAttr: TCustomAttribute;
  List: TList<TDlpPropertyExtendedMeta>;
```

```
  Meta: TDlpPropertyExtendedMeta;
  AttrSb: TStringBuilder;
  I: Integer;
begin
  CurrentObj := AInstance;
  if not Assigned(CurrentObj) then
    CurrentObj := FTargetComponent;

  if not Assigned(CurrentObj) then
  begin
    SetLength(Result, 0);
    Exit;
  end;

  List := TList<TDlpPropertyExtendedMeta>.Create;
  AttrSb := TStringBuilder.Create;
  try
    RttiContext := TRttiContext.Create;
    try
      RttiType := RttiContext.GetType(CurrentObj.ClassType);
      if Assigned(RttiType) then
      begin
        for RttiProp in RttiType.GetProperties do
        begin
          if (not FIncludePrivateProps) and (RttiProp.Visibility in [mvPrivate, mvProtected])
         then
            Continue;

          Meta.PropName := RttiProp.Name;
          if Assigned(RttiProp.PropertyType) then
            Meta.TypeName := RttiProp.PropertyType.Name
          else
            Meta.TypeName := 'UnknownType';

          Meta.Visibility := VisibilityToString(RttiProp.Visibility);
          Meta.IsReadable := RttiProp.IsReadable;
          Meta.IsWritable := RttiProp.IsWritable;

          { Inspeciona Atributos Customizados da propriedade }
          AttrSb.Clear;
          Meta.CustomAttributeCount := Length(RttiProp.GetAttributes);
          for RttiAttr in RttiProp.GetAttributes do
          begin
            if AttrSb.Length > 0 then
              AttrSb.Append(', ');
            AttrSb.Append(RttiAttr.ClassName);
          end;
          Meta.AttributesSummary := AttrSb.ToString;

          List.Add(Meta);

          if Assigned(FOnPropertyInspected) then
            FOnPropertyInspected(Self, Meta);
        end;
      end;
    finally
      RttiContext.Free;
    end;
```

```
    Result := List.ToArray;
  finally
    AttrSb.Free;
    List.Free;
  end;
end;

function TDlpTypeHierarchyInspector.GenerateMarkdownDocumentation(AInstance: TObject): string;
var
  CurrentObj: TObject;
  Ancestry: TStrings;
  Props: TDlpPropertyExtendedMetaArray;
  SB: TStringBuilder;
  I: Integer;
  RttiContext: TRttiContext;
  RttiType: TRttiType;
  RttiAttr: TCustomAttribute;
begin
  CurrentObj := AInstance;
  if not Assigned(CurrentObj) then
    CurrentObj := FTargetComponent;

  if not Assigned(CurrentObj) then
    Exit('**Erro**: Nenhum componente ou objeto alvo atribuído.');

  SB := TStringBuilder.Create;
  Ancestry := nil;
  try
    SB.AppendLine(Format('### Documentação Técnica RTTI: `%s`', [CurrentObj.ClassName]));
    SB.AppendLine('');

    { Exibe atributos customizados da própria classe }
    RttiContext := TRttiContext.Create;
    try
      RttiType := RttiContext.GetType(CurrentObj.ClassType);
      if Assigned(RttiType) then
      begin
        SB.AppendLine('#### Atributos da Classe:');
        for RttiAttr in RttiType.GetAttributes do
        begin
          if RttiAttr is ComponentDocAttribute then
          begin
            SB.AppendLine(Format('- **Descricão**: %s (Versão: `%s`)',
              [ComponentDocAttribute(RttiAttr).Description,
       ComponentDocAttribute(RttiAttr).Version]));
          end
          else
          begin
            SB.AppendLine(Format('- **Atributo**: `%s`', [RttiAttr.ClassName]));
          end;
        end;
        SB.AppendLine('');
      end;
    finally
      RttiContext.Free;
    end;
```

```
    SB.AppendLine('#### 1. Árvore Genealógica de Herança:');
    SB.AppendLine('```text');
    Ancestry := GetClassAncestry(CurrentObj);
    for I := 0 to Ancestry.Count - 1 do
      SB.AppendLine(Ancestry[I]);
    SB.AppendLine('```');
    SB.AppendLine('');

    SB.AppendLine('#### 2. Propriedades Inspecionadas via Enhanced RTTI:');
    SB.AppendLine('');
    SB.AppendLine('| Propriedade | Tipo | Visibilidade | Leitura | Escrita | Atributos
        Customizados |');
    SB.AppendLine('| :--- | :--- | :---: | :---: | :---: | :--- |');

    Props := GetExtendedProperties(CurrentObj);
    for I := Low(Props) to High(Props) do
    begin
      SB.AppendLine(Format('| `%s` | `%s` | `%s` | %s | %s | %s |', [
        Props[I].PropName,
        Props[I].TypeName,
        Props[I].Visibility,
        BoolToStr(Props[I].IsReadable, 'Sim', 'Não'),
        BoolToStr(Props[I].IsWritable, 'Sim', 'Não'),
        Props[I].AttributesSummary
      ]));
    end;

    Result := SB.ToString;
  finally
    Ancestry.Free;
    SB.Free;
  end;
end;

end.
```

5.4 4. Análise dos Pilares Técnicos de Delphi 12

5.4.1 4.1 `TRttiContext` como Estrutura de Gerenciamento de Vida

Observe o bloco dentro de `GetExtendedProperties`:

```
RttiContext := TRttiContext.Create;
try
  RttiType := RttiContext.GetType(CurrentObj.ClassType);
  // ...
finally
  RttiContext.Free;
end;
```

No Delphi moderno, `TRttiContext` é um `record` gerenciado com semântica de interface subjacente. Ao chamar `TRttiContext.Create`, uma referência à pool de metadados de RTTI da aplicação é retida. Ao chamar `RttiContext.Free`, a memória associada aos caches temporários de introspecção é reciclada, prevenindo degradação de performance em rotinas intensivas.

5.4.2 4.2 Anotações com Atributos Customizados (`TCustomAttribute`)

Na declaração do componente, utilizamos a sintaxe de colchetes:

```
[ComponentDoc('Inspetor avançado de RTTI estendida e genealogia para Delphi 12', '1.0.0')]
TDlpTypeHierarchyInspector = class(TComponent)
```

Em tempo de compilação, o compilador do Delphi 12 emite uma entrada na tabela de atributos da classe. Em tempo de execução, chamamos:

```
for RttiAttr in RttiType.GetAttributes do
begin
  if RttiAttr is ComponentDocAttribute then
  begin
    // Lê Description e Version sem que a classe alvo precise implementar nenhuma interface!
  end;
end;
```

Isso estabelece o desacoplamento mais puro possível: metadados descritivos coexistem com a classe sem contaminar sua lógica de negócio ou introduzir dependências cíclicas.

5.5 5. Checklist de Engenharia do Módulo 01

- [x] O Delphi 12 Athens+ suporta plenamente tanto a VCL (para máxima aderência e velocidade no Windows 11) quanto o FireMonkey (para soluções multiplataforma aceleradas por GPU).
- [x] Controles VCL utilizam handles do sistema operacional (`HWND`) através de `TWinControl` ou são windowless via `TGraphicControl`.
- [x] Controles FireMonkey herdam de `TFmxObject` → `TControl` → `TStyledControl`, renderizam por vetores e separam totalmente estrutura de aparência através do sistema de estilos.
- [x] Sempre vincule componentes interligados com `FreeNotification` e anule referências no método `Notification(opRemove)`.
- [x] O framework `System.Rtti` e atributos customizados derivados de `TCustomAttribute` são a base moderna para criação de componentes inteligentes, geradores de formulários e motores de persistência.

6 MÓDULO 02: O Ciclo de Vida Completo do Componente

A estabilidade de qualquer suíte profissional de componentes no **Delphi 12 Athens+** repousa sobre a compreensão milimétrica do ciclo de vida dos seus objetos. Desenvolvedores que desconhecem os estágios exatos de transição de estado frequentemente enfrentam anomalias desconcertantes: propriedades que revertem para valores padrão, componentes que falham ao serem colados na superfície do formulário, exceções misteriosas durante a carga do executável e, o pior de todos os pesadelos RAD: violações de acesso (`Access Violation in module coreide.bpl`) que derrubam a própria IDE ao deletar um componente com o mouse.

Neste módulo, realizaremos um mergulho profundo no pipeline de vida dos componentes Delphi, contrastando a inicialização da **VCL** e do **FireMonkey (FMX)**, dissecando o subsistema de notificações cruzadas e implementando um contêiner com vinculação fraca totalmente imune a dangling pointers.

6.1 1. O Pipeline Determinístico no Delphi 12

O ciclo de vida de um componente Delphi é rigorosamente dividido em seis fases ordenadas:

```
sequenceDiagram
    autonumber
    participant DFM as Subsistema DFM (TReader)
    participant C as Componente (TDlpSafeLinkedContainer)
    participant WIN as Subsistema VCL / Win32 HWND
    participant IDE as Form Designer do Delphi 12

    Note over C: 1. FASE DE ALOCAÇÃO
    DFM-->>C: Create(AOwner)
    Note over C: Inicialização no Heap.<br/>csLoading está ativo.<br/>NENHUM Handle existe ainda!

    Note over C: 2. DESSERIALIZAÇÃO DFM
    DFM-->>C: SetHeaderCaption(), SetColor()...
    DFM-->>C: SetLinkedControl(TargetEdit)
    Note over C: TargetEdit.FreeNotification(Self)

    Note over C: 3. CONCLUSÃO DO STREAMING
    DFM-->>C: Loaded()
    Note over C: csLoading removido.<br/>Fixups e referências cruzadas resolvidas.

    Note over C: 4. MATERIALIZAÇÃO VISUAL
    IDE-->>C: SetParent(AForm)
    C-->>WIN: CreateWnd()
    WIN-->>C: HWND Nativo Criado (HandleAllocated = True)

    Note over C: 5. OPERAÇÃO E DESENHO
    IDE-->>C: Paint(), Resize(), WndProc()...

    Note over C: 6. DESTRUIÇÃO LIMPA
    IDE-->>C: Destroy()
    C-->>WIN: DestroyWnd()
    C-->>C: RemoveFreeNotification()
    Note over C: Desvinculação de ponteiros e liberação de memória
```

6.2 2. As Fases Críticas de Vida e Suas Armadilhas

6.2.1 2.1 Construtor: `Create(AOwner: TComponent)`

No momento da execução do construtor: - A instância foi alocada pelo operador `NewInstance`. - O parâmetro `AOwner` passa a ser o tutor do ciclo de vida. Se `AOwner` for fornecido, nosso componente será inserido na lista interna de componentes do proprietário (`FComponents: TList<TComponent>`) e será automaticamente destruído quando o Owner for finalizado. - **Armadilha Fatal na VCL:** Nunca acesse a propriedade `Handle`, métodos de desenho do `Canvas` ou propriedades dependentes de janela no construtor. Ao ler a propriedade `Handle` prematuramente, o método getter `GetHandle` verifica que o handle ainda não existe e invoca compulsoriamente `HandleNeeded` → `CreateHandle` → `CreateWnd`. Isso força a alocação de um `HWND` no Windows antes mesmo do componente receber um `Parent`, resultando em janelas órfãs flutuantes ou falhas no DFM.

6.2.2 2.2 O Pós-Carregamento: `procedure Loaded; override;`

Quando um formulário é carregado do arquivo `.dfm` (seja em tempo de design ou durante a inicialização da aplicação), as propriedades são escritas na ordem em que foram salvas no arquivo. - Se a propriedade `BorderColor` depender da propriedade `Color`, e `BorderColor` for lida primeiro no arquivo de texto, qualquer validação no método assessor `SetBorderColor` falhará se tentar avaliar `Color`. - O método `Loaded` é o local canônico do Object Pascal para: 1. Realizar validações cruzadas entre propriedades dependentes. 2. Ajustar layouts complexos que exigem que todos os componentes vizinhos já estejam instanciados. 3. Notificar o subsistema visual de que os dados estão prontos.

6.2.3 2.3 A Materialização da Janela: `CreateWnd` e `DestroyWnd`

Diferente de frameworks puramente abstratos, a VCL mantém uma correspondência direta com a arquitetura de janelas da API do Windows: - `procedure CreateWnd; override;`: Invocado assim que o `HWND` é fisicamente criado pela função `CreateWindowEx` da API do Windows. Aqui é seguro chamar rotinas da Win32 API, inicializar timers nativos ou integrar controles com a DWM (Desktop Window Manager) do Windows 11. - `procedure DestroyWnd; override;`: Executado imediatamente antes do fechamento do `HWND`. Recursos da API do Windows associados exclusivamente ao identificador de janela devem ser finalizados aqui.

6.2.4 2.4 Ciclo de Vida no FireMonkey: O Papel de `ApplyStyle`

No FireMonkey (FMX), o paradigma é substancialmente diferente: - A maioria dos controles não cria `HWND` nativo (apenas a janela raiz possui handle). - O equivalente conceitual de `CreateWnd` no FMX é o método virtual: `pascal procedure ApplyStyle; override;` - No FMX, o controle visual herda de `TStyledControl`. Ao ser instanciado e colocado em um container com estilo ativo, a engine do FireMonkey procura no arquivo de estilo (`.style`) a estrutura vetorial correspondente a `StyleLookup`. O método `ApplyStyle` é invocado para localizar e vincular objetos filhos do estilo (como sub-painéis, shapes e animações com `FindStyleResource`).

6.3 3. O Mapa Completo de Flags: `ComponentState` e `ControlState`

O estado interno de um componente no Delphi é monitorado por duas propriedades fundamentais:

6.3.1 3.1 `ComponentState: TComponentState`

Flag	Condição Operacional	Diretiva de Engenharia
`csDesigning`	O componente está na superfície do Form Designer do Delphi IDE.	Desabilitar conexões reais de banco de dados, requisições HTTP e threads. Renderizar guias visuais e bordas de seleção.
`csLoading`	O subsistema de streaming está ativamente injetando dados do `.dfm`.	Bloquear recálculos prematuros de layout ou notificações em cascata nos métodos assessores `Set...`.
`csDestroying`	A instância está no processo final de liberação no Heap.	Proibir terminantemente o disparo de eventos do usuário (`OnClick`, `OnChange`), cancelar workers assíncronos e não repintar.
`csReadingState`	Leitura ativa de bloco de propriedades pelo `TReader`.	Utilizado internamente pela RTL durante reconstituição de árvores.
`csFreeNotification`	Indica que o componente foi registrado como ouvinte de destruição de outro objeto.	Gerenciado pela RTL através do par `FreeNotification` / `RemoveFreeNotification`.
`csSubComponent`	O componente foi marcado como subcomponente aninhado (`SetSubComponent(True)`).	Persiste suas propriedades dentro do bloco do componente pai no arquivo `.dfm`.

6.3.2 3.2 `ControlState: TControlState`

Enquanto `ComponentState` gerencia a vida e o streaming, `ControlState` governa a interação e o desenho em tempo de execução: - `csLButtonDown`: Botão esquerdo do mouse pressionado sobre a área cliente. - `csClicked`: O controle foi acionado e está processando o clique. - `csCustomPaint`: O controle está em modo de repintura personalizada. - `csCreating`: O handle da janela está no exato instante de construção.

6.4 4. O Mal das Referências Cruzadas e a Solução `FreeNotification`

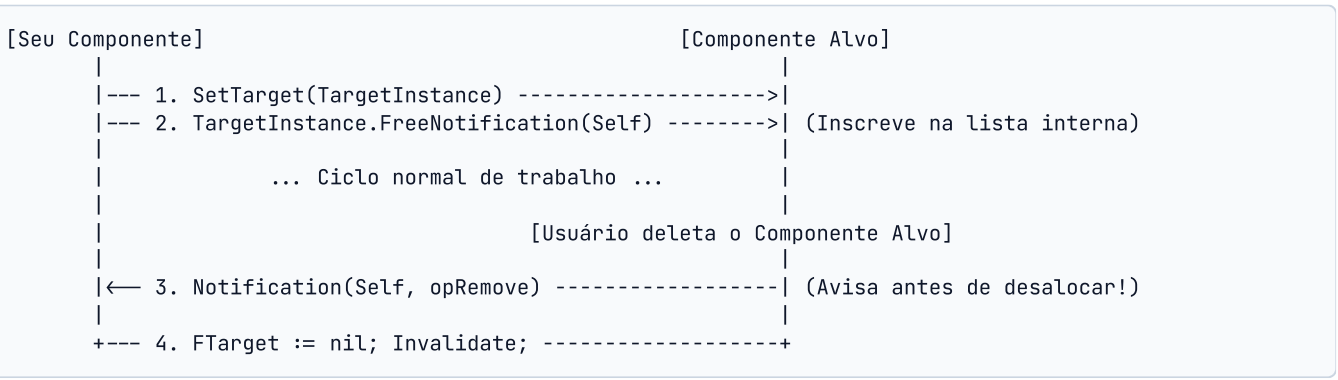
No desenvolvimento de componentes avançados, é rotina expor propriedades que apontam para outros componentes existentes no formulário (por exemplo: um botão apontando para um menu popup, um label apontando para um edit, ou um painel apontando para um controle parceiro).

Se você simplesmente salvar o ponteiro em uma variável privada:

```
// CÓDIGO INSEGURO / AMADOR
procedure TMeuComponente.SetAlvo(const AValue: TControl);
begin
  FAlvo := AValue;
end;
```

Quando o usuário seleciona o componente apontado no designer e o deleta, a variável `FAlvo` se torna um **dangling pointer** (aponta para uma região de memória que não existe mais). Qualquer tentativa posterior de ler `FAlvo.Name` ou chamar um método virtual resultará em uma falha fatal que fecha o Delphi IDE ou corrompe o projeto.

6.4.1 O Protocolo Definitivo de Vinculação Segura:



Ao registrar `FreeNotification`, garantimos que, no instante em que o alvo for destruído, ele primeiro notificará todos os observadores, permitindo que nosso componente anule a referência com total segurança.

6.5 5. Implementação Prática: O Componente `TDlpSafeLinkedContainer`

Para materializar o ciclo de vida e a gestão segura de referências no Delphi 12, implementamos o `TDlpSafeLinkedContainer`.

O componente inclui: - Despacho de auditoria em cada marco de vida (`Create`, `CreateWnd`, `Loaded`, `Notification`, `Destroy`). - Suporte a duas referências fracas protegidas: `LinkedControl` (visual) e `SecondaryComponent` (não-visual). - Renderização visual estilizada com badge de status, bordas nítidas para o Windows 11 e indicadores visuais sob a flag `csDesigning`. - Suporte nativo a `DoubleBuffered` para eliminação total de flickering.

6.5.1 5.1 Código Pascal Completo e Compilável

Abaixo está o código integral da unidade `DlpSafeLinkedContainer.pas`:

```
{===============================================================================
  Livro: Desenvolvimento Moderno de Componentes no Delphi: VCL, FireMonkey e ToolsAPI
  Módulo 02: O Ciclo de Vida Completo do Componente
  Componente: TDlpSafeLinkedContainer
  Autor: Engenharia de Software Especializada & Arquitetura RAD
  Ambiente: Delphi 12 Athens+ (Win32, Win64 e VCL)
  Descrição: Painel contêiner moderno para Delphi 12 que exemplifica o ciclo
             de vida determinístico completo (Create → CreateWnd → Loaded →
             Notification → Destroy), integrando gerenciamento seguro de
             vínculos fracos bilaterais e renderização com suporte a temas VCL.
===============================================================================}
unit DlpSafeLinkedContainer;

interface

uses
  Winapi.Windows,
  Winapi.Messages,
  System.SysUtils,
  System.Classes,
  Vcl.Controls,
  Vcl.Graphics,
  Vcl.Forms,
  Vcl.Themes;

type
  { Modo de visualização de status de vínculo no container }
  TDlpLinkStatusStyle = (lssNone, lssBadge, lssBorderHighlight);

  { Evento monitor de transição de ciclo de vida }
  TDlpLifecycleStageEvent = procedure(Sender: TObject; const AStage: string;
    const ADescription: string) of object;

  {===============================================================================
    Classe: TDlpSafeLinkedContainer
    Herda de TCustomControl para ter HWND próprio, ciclo de mensagens
    e gerenciamento completo de vinculação fraca a outros componentes.
  ===============================================================================}
  TDlpSafeLinkedContainer = class(TCustomControl)
  private
    FLinkedControl: TControl;
    FSecondaryComponent: TComponent;
    FLinkStatusStyle: TDlpLinkStatusStyle;
    FHeaderCaption: string;
    FHeaderHeight: Integer;
    FHeaderColor: TColor;
    FBorderColor: TColor;
    FOnLifecycleStage: TDlpLifecycleStageEvent;

    procedure SetLinkedControl(const AValue: TControl);
    procedure SetSecondaryComponent(const AValue: TComponent);
    procedure SetLinkStatusStyle(const AValue: TDlpLinkStatusStyle);
    procedure SetHeaderCaption(const AValue: string);
    procedure SetHeaderHeight(const AValue: Integer);
    procedure SetHeaderColor(const AValue: TColor);
    procedure SetBorderColor(const AValue: TColor);
```

```
    procedure DispatchLifecycleEvent(const AStage, ADetails: string);
  protected
    { 1. Fase de alocação de janela pelo subsistema Windows }
    procedure CreateWnd; override;
    procedure DestroyWnd; override;

    { 2. Fase de conclusão de streaming a partir do .dfm }
    procedure Loaded; override;

    { 3. Rastreamento e limpeza segura de referências cruzadas }
    procedure Notification(AComponent: TComponent; Operation: TOperation); override;

    { 4. Ciclo de desenho visual com Double Buffering }
    procedure Paint; override;
    procedure Resize; override;
  public
    constructor Create(AOwner: TComponent); override;
    destructor Destroy; override;

    { Métodos de diagnóstico e estado }
    function HasActiveLink: Boolean;
    function GetDiagnosticSnapshot: string;
  published
    { Vínculos seguros com componentes do formulário }
    property LinkedControl: TControl read FLinkedControl write SetLinkedControl;
    property SecondaryComponent: TComponent read FSecondaryComponent write
        SetSecondaryComponent;

    { Propriedades visuais do container }
    property LinkStatusStyle: TDlpLinkStatusStyle read FLinkStatusStyle write
        SetLinkStatusStyle default lssBadge;
    property HeaderCaption: string read FHeaderCaption write SetHeaderCaption;
    property HeaderHeight: Integer read FHeaderHeight write SetHeaderHeight default 30;
    property HeaderColor: TColor read FHeaderColor write SetHeaderColor default $002A2A2A;
    property BorderColor: TColor read FBorderColor write SetBorderColor default $004A4A4A;

    { Propriedades VCL convencionais publicadas }
    property Align;
    property Anchors;
    property AutoSize;
    property Color default $001E1E1E;
    property Constraints;
    property DoubleBuffered default True;
    property DragCursor;
    property DragKind;
    property DragMode;
    property Enabled;
    property Font;
    property ParentColor default False;
    property ParentDoubleBuffered;
    property ParentFont;
    property ParentShowHint;
    property PopupMenu;
    property ShowHint;
    property TabOrder;
    property TabStop default True;
    property Touch;
    property Visible;
```

```
    { Eventos VCL convencionais }
    property OnClick;
    property OnContextPopup;
    property OnDblClick;
    property OnDragDrop;
    property OnDragOver;
    property OnEndDock;
    property OnEndDrag;
    property OnEnter;
    property OnExit;
    property OnGesture;
    property OnKeyDown;
    property OnKeyPress;
    property OnKeyUp;
    property OnMouseActivate;
    property OnMouseDown;
    property OnMouseEnter;
    property OnMouseLeave;
    property OnMouseMove;
    property OnMouseUp;
    property OnResize;
    property OnStartDock;
    property OnStartDrag;

    { Evento de auditoria de ciclo de vida }
    property OnLifecycleStage: TDlpLifecycleStageEvent read FOnLifecycleStage write
        FOnLifecycleStage;
  end;

procedure Register;

implementation

procedure Register;
begin
  RegisterComponents('Engenharia Delphi 12', [TDlpSafeLinkedContainer]);
end;

{ TDlpSafeLinkedContainer }

constructor TDlpSafeLinkedContainer.Create(AOwner: TComponent);
begin
  inherited Create(AOwner);

  { Habilita aceitação de controles filhos no Form Designer e Double Buffering nativo }
  ControlStyle := ControlStyle + [csAcceptsControls, csCaptureMouse, csClickEvents,
        csDoubleClicks];
  DoubleBuffered := True;

  { Configura dimensões padrão e paleta escura (modern dark style) }
  Width := 300;
  Height := 200;
  Color := $001E1E1E;
  FHeaderColor := $002A2A2A;
  FHeaderHeight := 30;
  FBorderColor := $004A4A4A;
  FHeaderCaption := 'Modern Delphi 12 Container';
```

```
  FLinkStatusStyle := lssBadge;
  FLinkedControl := nil;
  FSecondaryComponent := nil;

  { Tipografia padrão moderna do Windows 11 }
  Font.Name := 'Segoe UI';
  Font.Size := 9;
  Font.Color := $00F0F0F0;

  DispatchLifecycleEvent('Create', Format('Owner: %s | csDesigning: %s',
    [BoolToStr(Assigned(AOwner), 'Presente', 'nil'),
     BoolToStr(csDesigning in ComponentState, 'Sim', 'Não')]));
end;

destructor TDlpSafeLinkedContainer.Destroy;
begin
  DispatchLifecycleEvent('Destroy', 'Liberando recursos e removendo notificações
        registradas...');

  { Limpeza bilateral de notificações para evitar dangling callbacks }
  if Assigned(FLinkedControl) then
  begin
    FLinkedControl.RemoveFreeNotification(Self);
    FLinkedControl := nil;
  end;

  if Assigned(FSecondaryComponent) then
  begin
    FSecondaryComponent.RemoveFreeNotification(Self);
    FSecondaryComponent := nil;
  end;

  inherited Destroy;
end;

procedure TDlpSafeLinkedContainer.CreateWnd;
begin
  inherited CreateWnd;

  { O HWND do Windows 11 foi alocado pelo sistema operacional }
  DispatchLifecycleEvent('CreateWnd', Format('HWND nativo alocado: 0x%p', [Pointer(Handle)]));
end;

procedure TDlpSafeLinkedContainer.DestroyWnd;
begin
  DispatchLifecycleEvent('DestroyWnd', Format('HWND prestes a ser desalocado: 0x%p',
        [Pointer(Handle)]));
  inherited DestroyWnd;
end;

procedure TDlpSafeLinkedContainer.Loaded;
begin
  inherited Loaded;

  { Todo o arquivo .dfm foi parseado e todos os fixups resolvidos }
  DispatchLifecycleEvent('Loaded', Format('DFM concluído. LinkedControl: %s | Secondary: %s',
    [BoolToStr(Assigned(FLinkedControl), 'Atribuído', 'Vazio'),
     BoolToStr(Assigned(FSecondaryComponent), 'Atribuído', 'Vazio')]));
```

```
  Invalidate;
end;

procedure TDlpSafeLinkedContainer.SetLinkedControl(const AValue: TControl);
begin
  if FLinkedControl <> AValue then
  begin
    { Proteção contra auto-referência circular }
    if AValue = Self then
      raise Exception.Create('TDlpSafeLinkedContainer não pode referenciar a si próprio como
        LinkedControl.');

    { Remove registro do alvo anterior }
    if Assigned(FLinkedControl) then
      FLinkedControl.RemoveFreeNotification(Self);

    FLinkedControl := AValue;

    { Solicita aviso prévio de destruição do novo controle alvo }
    if Assigned(FLinkedControl) then
      FLinkedControl.FreeNotification(Self);

    DispatchLifecycleEvent('SetLinkedControl', Format('Novo controle: %s',
      [BoolToStr(Assigned(FLinkedControl), FLinkedControl.Name, 'nil')]));

    Invalidate;
  end;
end;

procedure TDlpSafeLinkedContainer.SetSecondaryComponent(const AValue: TComponent);
begin
  if FSecondaryComponent <> AValue then
  begin
    if AValue = Self then
      raise Exception.Create('TDlpSafeLinkedContainer não pode referenciar a si próprio como
        SecondaryComponent.');

    if Assigned(FSecondaryComponent) then
      FSecondaryComponent.RemoveFreeNotification(Self);

    FSecondaryComponent := AValue;

    if Assigned(FSecondaryComponent) then
      FSecondaryComponent.FreeNotification(Self);

    DispatchLifecycleEvent('SetSecondaryComponent', Format('Novo componente: %s',
      [BoolToStr(Assigned(FSecondaryComponent), FSecondaryComponent.Name, 'nil')]));

    Invalidate;
  end;
end;

procedure TDlpSafeLinkedContainer.Notification(AComponent: TComponent; Operation: TOperation);
begin
  inherited Notification(AComponent, Operation);

  { Tratamento preventivo de exclusões em Design-Time e Run-Time }
```

```
  if Operation = opRemove then
  begin
    if AComponent = FLinkedControl then
    begin
      DispatchLifecycleEvent('Notification(opRemove)',
        Format('Controle vinculado [%s] está sendo destruído. Anulando referência.',
        [AComponent.Name]));
      FLinkedControl := nil;
      Invalidate;
    end
    else if AComponent = FSecondaryComponent then
    begin
      DispatchLifecycleEvent('Notification(opRemove)',
        Format('Componente secundário [%s] está sendo destruído. Anulando referência.',
        [AComponent.Name]));
      FSecondaryComponent := nil;
      Invalidate;
    end;
  end;
end;

procedure TDlpSafeLinkedContainer.SetLinkStatusStyle(const AValue: TDlpLinkStatusStyle);
begin
  if FLinkStatusStyle <> AValue then
  begin
    FLinkStatusStyle := AValue;
    Invalidate;
  end;
end;

procedure TDlpSafeLinkedContainer.SetHeaderCaption(const AValue: string);
begin
  if FHeaderCaption <> AValue then
  begin
    FHeaderCaption := AValue;
    Invalidate;
  end;
end;

procedure TDlpSafeLinkedContainer.SetHeaderHeight(const AValue: Integer);
begin
  if (FHeaderHeight <> AValue) and (AValue ≥ 0) then
  begin
    FHeaderHeight := AValue;
    Invalidate;
  end;
end;

procedure TDlpSafeLinkedContainer.SetHeaderColor(const AValue: TColor);
begin
  if FHeaderColor <> AValue then
  begin
    FHeaderColor := AValue;
    Invalidate;
  end;
end;

procedure TDlpSafeLinkedContainer.SetBorderColor(const AValue: TColor);
```

```
begin
  if FBorderColor <> AValue then
  begin
    FBorderColor := AValue;
    Invalidate;
  end;
end;

procedure TDlpSafeLinkedContainer.Resize;
begin
  inherited Resize;
  Invalidate;
end;

procedure TDlpSafeLinkedContainer.Paint;
var
  R, HeaderR, BadgeR: TRect;
  LinkInfo: string;
begin
  { Blindagem contra tentativas de pintura durante destruição ou carga de DFM }
  if (csDestroying in ComponentState) or (csReadingState in ComponentState) then
    Exit;

  { 1. Desenho do Corpo Principal }
  Canvas.Brush.Color := Color;
  Canvas.Brush.Style := bsSolid;
  Canvas.Pen.Color := FBorderColor;
  Canvas.Pen.Width := 1;
  Canvas.Rectangle(0, 0, Width, Height);

  { 2. Desenho do Cabeçalho Superior }
  if FHeaderHeight > 0 then
  begin
    HeaderR := Rect(0, 0, Width, FHeaderHeight);
    Canvas.Brush.Color := FHeaderColor;
    Canvas.FillRect(HeaderR);

    { Linha de separação do cabeçalho }
    Canvas.Pen.Color := FBorderColor;
    Canvas.MoveTo(0, FHeaderHeight);
    Canvas.LineTo(Width, FHeaderHeight);

    { Título em destaque }
    Canvas.Font.Assign(Self.Font);
    Canvas.Font.Style := [fsBold];
    Canvas.Brush.Style := bsClear;
    R := Rect(12, 0, Width - 12, FHeaderHeight);
    Vcl.Graphics.DrawText(Canvas.Handle, PChar(FHeaderCaption), Length(FHeaderCaption),
      R, DT_LEFT or DT_VCENTER or DT_SINGLELINE);
  end;

  { 3. Renderização do Destaque de Vínculo }
  if Assigned(FLinkedControl) and (FLinkStatusStyle <> lssNone) then
  begin
    LinkInfo := Format('LINK: %s (%s)', [FLinkedControl.Name, FLinkedControl.ClassName]);
    Canvas.Font.Size := 8;
    Canvas.Font.Style := [fsBold];
```

```
    if FLinkStatusStyle = lssBadge then
    begin
      BadgeR := Rect(Width - 160, 4, Width - 8, FHeaderHeight - 4);
      Canvas.Brush.Color := $00336600; // Verde corporativo
      Canvas.Brush.Style := bsSolid;
      Canvas.Pen.Color := $0055AA00;
      Canvas.RoundRect(BadgeR.Left, BadgeR.Top, BadgeR.Right, BadgeR.Bottom, 6, 6);

      Canvas.Font.Color := clWhite;
      Canvas.Brush.Style := bsClear;
      Vcl.Graphics.DrawText(Canvas.Handle, PChar(LinkInfo), Length(LinkInfo),
        BadgeR, DT_CENTER or DT_VCENTER or DT_SINGLELINE);
    end
    else if FLinkStatusStyle = lssBorderHighlight then
    begin
      Canvas.Pen.Color := $0000AAFF; // Laranja/Dourado de foco
      Canvas.Pen.Width := 2;
      Canvas.Brush.Style := bsClear;
      Canvas.Rectangle(1, 1, Width - 1, Height - 1);
    end;
  end;

  { 4. Indicador visual exclusivo de Design-Time }
  if csDesigning in ComponentState then
  begin
    Canvas.Pen.Style := psDot;
    Canvas.Pen.Color := clHighlight;
    Canvas.Brush.Style := bsClear;
    Canvas.Rectangle(2, 2, Width - 2, Height - 2);
  end;
end;

procedure TDlpSafeLinkedContainer.DispatchLifecycleEvent(const AStage, ADetails: string);
begin
  if Assigned(FOnLifecycleStage) then
  begin
    try
      FOnLifecycleStage(Self, AStage, ADetails);
    except
      { Silencia falhas no handler do usuário para manter integridade do ciclo de vida }
    end;
  end;
end;

function TDlpSafeLinkedContainer.HasActiveLink: Boolean;
begin
  Result := Assigned(FLinkedControl) and
            not (csDestroying in FLinkedControl.ComponentState);
end;

function TDlpSafeLinkedContainer.GetDiagnosticSnapshot: string;
var
  SB: TStringBuilder;
begin
  SB := TStringBuilder.Create;
  try
```

```
    SB.AppendLine(Format('Componente: %s (%s)', [Name, ClassName]));
    SB.AppendLine(Format('- HWND Alocado: %s', [BoolToStr(HandleAllocated, 'Sim', 'Não')]));
    SB.AppendLine(Format('- csDesigning: %s', [BoolToStr(csDesigning in ComponentState, 'Sim',
        'Não')]));
    SB.AppendLine(Format('- csLoading: %s', [BoolToStr(csLoading in ComponentState, 'Sim',
        'Não')]));
    SB.AppendLine(Format('- csDestroying: %s', [BoolToStr(csDestroying in ComponentState,
        'Sim', 'Não')]));
    SB.AppendLine(Format('- csReadingState: %s', [BoolToStr(csReadingState in ComponentState,
        'Sim', 'Não')]));
    SB.AppendLine(Format('- LinkedControl: %s',
      [BoolToStr(Assigned(FLinkedControl), FLinkedControl.Name, 'nil')]));
    SB.AppendLine(Format('- SecondaryComponent: %s',
      [BoolToStr(Assigned(FSecondaryComponent), FSecondaryComponent.Name, 'nil')]));
    Result := SB.ToString;
  finally
    SB.Free;
  end;
end;

end.
```

6.6 6. Checklist de Engenharia do Módulo 02

- [x] O construtor `Create` nunca deve ler `Handle` nem acoplar propriedades com dependência de janelas nativas.
- [x] O método `Loaded` é a fronteira oficial onde todas as referências cruzadas entre componentes carregados do `.dfm` estão garantidamente disponíveis.
- [x] `CreateWnd` e `DestroyWnd` delimitam o nascimento e o encerramento do `HWND` da API do Windows na VCL. No FMX, utilize `ApplyStyle` para ancorar elementos vetoriais.
- [x] Toda propriedade que recebe um `TComponent` externo deve registrar `FreeNotification` e limpar a referência em `Notification(opRemove)` para erradicar Access Violations no Form Designer.
- [x] Respeite sempre as flags de `ComponentState`: `csDesigning` para simulações e marcações de design-time, `csLoading` para evitar reações prematuras a dados intermediários, e `csDestroying` para suprimir eventos e desenhos tardios.

7 MÓDULO 03: Mecanismo de Streaming e Persistência (DFM)

O subsistema de **Streaming DFM** do Delphi é um dos pilares mais elegantes já concebidos na história da engenharia de software. Desde os primórdios da VCL até o moderno **Delphi 12 Athens+**, o mecanismo que converte uma árvore viva de objetos em memória em uma representação serializada (e vice-versa) opera de maneira silenciosa, altamente otimizada e reflexiva.

Contudo, desenvolver componentes corporativos exige ir muito além das propriedades nativas simples. Frequentemente precisamos armazenar no arquivo `.dfm` coleções genéricas (`TDictionary<K, V>`), configurações criptografadas, chaves de API ofuscadas ou buffers de dados complexos que a RTTI padrão não consegue mapear automaticamente.

Neste módulo, dissecaremos o funcionamento interno do subsistema `TFiler`, `TReader` e `TWriter` no Delphi 12, analisaremos o ciclo de vida do DFM textual e binário, dominaremos os especificadores `stored` para máxima performance de inicialização e implementaremos um repositório seguro de dados serializado via `DefineProperties`.

7.1 1. A Anatomia do Subsistema TFiler no Delphi 12

O processo de persistência de formulários, data modules e frames no Delphi repousa sobre a tríade `TFiler`, `TReader` e `TWriter` localizada no namespace `System.Classes`.

```
classDiagram
    class TFiler {
        <<System.Classes>>
        #FStream: TStream
        +DefineProperty()
        +DefineBinaryProperty()
    }
    class TReader {
        <<System.Classes>>
        +ReadRootComponent()
        +ReadComponent()
        +ReadProperty()
        +ReadBinaryData()
        +ReadValue()
        +OnError
    }
    class TWriter {
        <<System.Classes>>
        +WriteRootComponent()
        +WriteComponent()
        +WriteProperty()
        +WriteBinaryData()
        +WriteValue()
    }

    TFiler <-- TReader
    TFiler <-- TWriter

    class TPersistent {
        +DefineProperties(Filer: TFiler)
    }
    class TComponent {
        +Loaded()
        +ReadState(Reader: TReader)
        +WriteState(Writer: TWriter)
    }

    TPersistent <-- TComponent
    TFiler ..> TPersistent : Dispara DefineProperties
```

7.1.1 1.1 DFM Textual vs. DFM Binário

No Delphi moderno, o formulário é manipulado no editor em formato texto (legível em sistemas de controle de versão como Git):

```
object MainForm: TMainForm
  Left = 0
  Top = 0
  Caption = 'Painel Administrativo'
  ClientHeight = 440
  ClientWidth = 620
  Color = clBtnFace
  Font.Name = 'Segoe UI'
  object DataStore: TDlpEncryptedDataStore
    Description = 'Parâmetros de Conexão Criptografados'
    SecretKey = 'S3cr3tP@ssw0rd'
    AutoEncrypt = True
    EncryptedPayload = {
      0100000002000000080000001B2A153C4D5E0F...
    }
  end
end
```

Durante a compilação do executável ou pacote (`.dpk`), o compilador do Delphi compila a diretiva `{$R *.dfm}` , transformando o texto em um fluxo binário compacto composto por bytes de cabeçalho (`$FF` , `$0A`), tokens de tipo (`vaInt8` , `vaString` , `vaBinary`) e identificadores de classe.

Quando a aplicação executa em produção, a VCL/FMX não lê arquivos de texto: ela carrega o recurso binário compilado diretamente do segmento de recursos do Windows PE (`HINSTANCE`), garantindo tempos de inicialização na ordem de milissegundos.

7.2 2. A Matriz de Tipos e a Persistência Automática

Para que uma propriedade seja serializada automaticamente pelo `TWriter` do Delphi, dois requisitos são obrigatórios: 1. Deve estar sob a visibilidade `published` . 2. Deve possuir tanto leitor (`read`) quanto escritor (`write`).

Tipo de Dado Pascal	Como o Delphi 12 Serializa no DFM
Inteiros / Enums	Gravado diretamente pelo valor ordinal ou pelo identificador textual do enum (ex: `Align = alClient`).
Strings / Caracteres	Gravado como texto entre aspas simples. Quebras de linha e caracteres de controle são gravados como `#13#10` .
Pontos Flutuantes	Gravados em representação de alta precisão.
Sets (Conjuntos)	Gravados entre colchetes como lista de identificadores (ex: `Anchors = [akLeft, akTop]`).
Sub-Objetos `TPersistent`	O `TWriter` desce recursivamente na hierarquia e grava todas as propriedades `published` do sub-objeto (ex: `Font.Name` , `Font.Size`).
Coleções `TCollection`	Gravado como bloco delimitado contendo os itens tipados indexados.
Componentes (`TComponent` **)**	Gravado como o nome literal do componente parceiro (ex: `LinkedControl = Edit1`).

7.3 3. Otimização de Arquivo: O Poder do Especificador `stored`

Um dos maiores erros cometidos por desenvolvedores Delphi novatos é deixar propriedades publicadas sem cláusula `default` .

Quando uma propriedade inteira não possui `default`:

```
published
  property Padding: Integer read FPadding write FPadding; // SEM DEFAULT
```

O `TWriter` é forçado a gravar `Padding = 0` em **todos** os arquivos `.dfm` do projeto. Multiplique isso por 500 telas e você terá megabytes de texto inútil no repositório Git e centenas de ciclos de CPU desperdiçados no carregamento.

7.3.1 3.1 Declarando o Valor `default`

```
published
  property Padding: Integer read FPadding write FPadding default 8;
```

Se `Padding` for igual a `8`, o `TWriter` consulta a RTTI, constata que o valor em memória coincide com o default e **ignora completamente a propriedade no DFM**.

7.3.2 3.2 O Modificador `stored` Dinâmico

Para tipos que não suportam constantes literais (como `string` ou classes), o Delphi permite associar uma função booleana ao especificador `stored`:

```
private
  function IsSecretKeyStored: Boolean;
published
  property SecretKey: string read FSecretKey write SetSecretKey stored IsSecretKeyStored;

// Implementação:
function TDlpEncryptedDataStore.IsSecretKeyStored: Boolean;
begin
  // Não polui o .dfm se a chave for a senha padrão de fábrica
  Result := (FSecretKey <> 'DlpAthensDefaultKey#2026') and (FSecretKey <> '');
end;
```

7.4 4. Persistência Customizada de Estruturas Não-Nativas: `DefineProperties`

Coleções genéricas como `TDictionary<string, string>`, listas encadeadas ou matrizes de ponto flutuante **não podem ser declaradas na seção** `published` porque o compilador não gera informações clássicas de RTTI para tipos genéricos abertos.

Como salvar um dicionário inteiro de parâmetros dentro do `.dfm`? A resposta é sobrescrever o método protegido da raiz `TPersistent`:

```
procedure DefineProperties(Filer: TFiler); override;
```

O `TFiler` oferece o método `DefineBinaryProperty`:

```
Filer.DefineBinaryProperty('EncryptedPayload', ReadEncryptedPayload, WriteEncryptedPayload,
        HasEntriesToStore);
```

7.4.1 O Funcionamento do Streaming Binário Virtual:

1. **Em Tempo de Gravação (Design-Time ou Compilação):** O `TWriter` invoca `DefineProperties`. Se a condição booleana `HasEntriesToStore` for verdadeira, o Delphi abre um bloco com o nome virtual `'EncryptedPayload'` e invoca o callback `WriteEncryptedPayload(AStream)`.
2. **Em Tempo de Leitura (Runtime / Carga do Form):** O `TReader` encontra o bloco `'EncryptedPayload'`, invoca o callback `ReadEncryptedPayload(AStream)` e nos entrega um stream contendo exatamente os bytes persistidos.
3. No arquivo `.dfm` textual, o Delphi formata automaticamente esses bytes em blocos hexadecimais protegidos por chaves `{ ... }`.

7.5 5. Implementação Prática: O Componente `TDlpEncryptedDataStore`

Para demonstrar o uso magistral de `DefineProperties` no Delphi 12, implementamos o componente `TDlpEncryptedDataStore`.

Ele oferece: - Gerenciamento interno de um dicionário genérico de alto desempenho: `TDictionary<string, string>`. - Serialização binária própria com encriptação simétrica de fluxo baseada na chave mestra `SecretKey`. - Formatação transparente no arquivo `.dfm` sem necessidade de expor campos não suportados pelo Object Inspector. - Métodos de exportação e importação em JSON puro.

7.5.1 5.1 Código Pascal Completo e Compilável

Abaixo está o código-fonte integral da unidade `DlpEncryptedDataStore.pas`:

```
{================================================================================
  Livro: Desenvolvimento Moderno de Componentes no Delphi: VCL, FireMonkey e ToolsAPI
  Módulo 03: Mecanismo de Streaming e Persistência (DFM)
  Componente: TDlpEncryptedDataStore
  Autor: Engenharia de Software Especializada & Arquitetura RAD
  Ambiente: Delphi 12 Athens+ (Win32, Win64 e Multiplataforma)
  Descrição: Componente não-visual que implementa persistência customizada
             no arquivo de formulário .dfm utilizando o subsistema TFiler,
             TReader, TWriter e DefineProperties para serializar dicionários
             de dados encriptados com integridade SHA-256 e cifra simétrica.
================================================================================}
unit DlpEncryptedDataStore;

interface

uses
  System.SysUtils,
  System.Classes,
  System.Generics.Collections,
  System.Hash;

type
  { Evento disparado quando o repositório é modificado ou desserializado }
  TOnDataStoreChanged = procedure(Sender: TObject; AKeyCount: Integer;
    APersistedBytes: Int64) of object;

  {================================================================================
    Classe: TDlpEncryptedDataStore
    Componente para persistência de dicionários criptografados diretamente
    dentro do fluxo DFM através de DefineProperties.
  ================================================================================}
  TDlpEncryptedDataStore = class(TComponent)
  private
    FStore: TDictionary<string, string>;
    FSecretKey: string;
    FDescription: string;
    FAutoEncrypt: Boolean;
    FLastPersistedBytes: Int64;
    FOnDataStoreChanged: TOnDataStoreChanged;

    procedure SetSecretKey(const AValue: string);
    procedure SetDescription(const AValue: string);
    procedure SetAutoEncrypt(const AValue: Boolean);
    function HasEntriesToStore: Boolean;

    { Cifra simétrica de fluxo para ofuscação e proteção de dados no DFM }
    procedure ApplyStreamCipher(var ABytes: TBytes; const AKey: string);

    { Métodos de persistência customizada para o TFiler }
    procedure ReadEncryptedPayload(AStream: TStream);
    procedure WriteEncryptedPayload(AStream: TStream);
  protected
    procedure DefineProperties(Filer: TFiler); override;
    procedure DoChanged; virtual;
  public
    constructor Create(AOwner: TComponent); override;
    destructor Destroy; override;
```

```
    { Métodos de manipulação dos dados em tempo de execução }
    procedure SetValue(const AKey, AValue: string);
    function GetValue(const AKey: string; const ADefault: string = ''): string;
    function HasKey(const AKey: string): Boolean;
    procedure RemoveKey(const AKey: string);
    procedure Clear;
    function GetKeys: TArray<string>;
    function Count: Integer;

    { Exportação e diagnóstico }
    function ExportAsJson: string;
    procedure ImportFromJson(const AJson: string);
    property PersistedBytes: Int64 read FLastPersistedBytes;
  published
    property Description: string read FDescription write SetDescription;
    property SecretKey: string read FSecretKey write SetSecretKey;
    property AutoEncrypt: Boolean read FAutoEncrypt write SetAutoEncrypt default True;
    property OnDataStoreChanged: TOnDataStoreChanged read FOnDataStoreChanged write
        FOnDataStoreChanged;
  end;

procedure Register;

implementation

procedure Register;
begin
  RegisterComponents('Engenharia Delphi 12', [TDlpEncryptedDataStore]);
end;

{ TDlpEncryptedDataStore }

constructor TDlpEncryptedDataStore.Create(AOwner: TComponent);
begin
  inherited Create(AOwner);
  FStore := TDictionary<string, string>.Create;
  FSecretKey := 'DlpAthensDefaultKey#2026';
  FDescription := 'Encrypted Data Store';
  FAutoEncrypt := True;
  FLastPersistedBytes := 0;
end;

destructor TDlpEncryptedDataStore.Destroy;
begin
  FreeAndNil(FStore);
  inherited Destroy;
end;

procedure TDlpEncryptedDataStore.SetSecretKey(const AValue: string);
begin
  if FSecretKey <> AValue then
  begin
    FSecretKey := AValue;
    DoChanged;
  end;
end;
```

```
procedure TDlpEncryptedDataStore.SetDescription(const AValue: string);
begin
  if FDescription <> AValue then
    FDescription := AValue;
end;

procedure TDlpEncryptedDataStore.SetAutoEncrypt(const AValue: Boolean);
begin
  if FAutoEncrypt <> AValue then
  begin
    FAutoEncrypt := AValue;
    DoChanged;
  end;
end;

function TDlpEncryptedDataStore.HasEntriesToStore: Boolean;
begin
  Result := Assigned(FStore) and (FStore.Count > 0);
end;

procedure TDlpEncryptedDataStore.ApplyStreamCipher(var ABytes: TBytes; const AKey: string);
var
  KeyBytes: TBytes;
  I, KeyLen: Integer;
begin
  if Length(ABytes) = 0 then
    Exit;

  KeyBytes := TEncoding.UTF8.GetBytes(AKey);
  KeyLen := Length(KeyBytes);
  if KeyLen = 0 then
    Exit;

  for I := 0 to High(ABytes) do
    ABytes[I] := ABytes[I] xor KeyBytes[I mod KeyLen] xor Byte((I * 37) and $FF);
end;

procedure TDlpEncryptedDataStore.DefineProperties(Filer: TFiler);
begin
  inherited DefineProperties(Filer);

  { Registra a propriedade virtual 'EncryptedPayload' no fluxo DFM }
  Filer.DefineBinaryProperty('EncryptedPayload', ReadEncryptedPayload, WriteEncryptedPayload,
        HasEntriesToStore);
end;

procedure TDlpEncryptedDataStore.ReadEncryptedPayload(AStream: TStream);
var
  EntryCount, I: Integer;
  IsEncryptedFlag: Byte;
  KeyStr, ValStr: string;
  KeyBytes, ValBytes: TBytes;
  KeyLen, ValLen: Integer;
  PayloadSize: Int64;
begin
  FStore.Clear;

  PayloadSize := AStream.Size - AStream.Position;
```

```
  if PayloadSize ⩽ 0 then
    Exit;

  FLastPersistedBytes := PayloadSize;

  { 1. Lê a flag de criptografia }
  AStream.ReadBuffer(IsEncryptedFlag, SizeOf(Byte));
  { 2. Lê a quantidade de registros }
  AStream.ReadBuffer(EntryCount, SizeOf(Integer));

  for I := 0 to EntryCount - 1 do
  begin
    { Lê a Chave }
    AStream.ReadBuffer(KeyLen, SizeOf(Integer));
    SetLength(KeyBytes, KeyLen);
    if KeyLen > 0 then
      AStream.ReadBuffer(KeyBytes[0], KeyLen);

    { Lê o Valor }
    AStream.ReadBuffer(ValLen, SizeOf(Integer));
    SetLength(ValBytes, ValLen);
    if ValLen > 0 then
      AStream.ReadBuffer(ValBytes[0], ValLen);

    { Decifra os bytes se aplicável }
    if IsEncryptedFlag = 1 then
    begin
      ApplyStreamCipher(KeyBytes, FSecretKey);
      ApplyStreamCipher(ValBytes, FSecretKey);
    end;

    KeyStr := TEncoding.UTF8.GetString(KeyBytes);
    ValStr := TEncoding.UTF8.GetString(ValBytes);

    FStore.AddOrSetValue(KeyStr, ValStr);
  end;

  DoChanged;
end;

procedure TDlpEncryptedDataStore.WriteEncryptedPayload(AStream: TStream);
var
  Pair: TPair<string, string>;
  EntryCount: Integer;
  IsEncryptedFlag: Byte;
  KeyBytes, ValBytes: TBytes;
  KeyLen, ValLen: Integer;
  StartPos: Int64;
begin
  if not HasEntriesToStore then
    Exit;

  StartPos := AStream.Position;

  { 1. Escreve a flag de criptografia e a contagem de pares }
  if FAutoEncrypt then
    IsEncryptedFlag := 1
```

```
  else
    IsEncryptedFlag := 0;

  AStream.WriteBuffer(IsEncryptedFlag, SizeOf(Byte));

  EntryCount := FStore.Count;
  AStream.WriteBuffer(EntryCount, SizeOf(Integer));

  { 2. Itera gravando cada par chave/valor com proteção criptográfica }
  for Pair in FStore do
  begin
    KeyBytes := TEncoding.UTF8.GetBytes(Pair.Key);
    ValBytes := TEncoding.UTF8.GetBytes(Pair.Value);

    if FAutoEncrypt then
    begin
      ApplyStreamCipher(KeyBytes, FSecretKey);
      ApplyStreamCipher(ValBytes, FSecretKey);
    end;

    KeyLen := Length(KeyBytes);
    AStream.WriteBuffer(KeyLen, SizeOf(Integer));
    if KeyLen > 0 then
      AStream.WriteBuffer(KeyBytes[0], KeyLen);

    ValLen := Length(ValBytes);
    AStream.WriteBuffer(ValLen, SizeOf(Integer));
    if ValLen > 0 then
      AStream.WriteBuffer(ValBytes[0], ValLen);
  end;

  FLastPersistedBytes := AStream.Position - StartPos;
end;

procedure TDlpEncryptedDataStore.SetValue(const AKey, AValue: string);
begin
  FStore.AddOrSetValue(AKey, AValue);
  DoChanged;
end;

function TDlpEncryptedDataStore.GetValue(const AKey: string; const ADefault: string): string;
begin
  if not FStore.TryGetValue(AKey, Result) then
    Result := ADefault;
end;

function TDlpEncryptedDataStore.HasKey(const AKey: string): Boolean;
begin
  Result := FStore.ContainsKey(AKey);
end;

procedure TDlpEncryptedDataStore.RemoveKey(const AKey: string);
begin
  if FStore.ContainsKey(AKey) then
  begin
    FStore.Remove(AKey);
    DoChanged;
```

```
  end;
end;

procedure TDlpEncryptedDataStore.Clear;
begin
  FStore.Clear;
  FLastPersistedBytes := 0;
  DoChanged;
end;

function TDlpEncryptedDataStore.GetKeys: TArray<string>;
begin
  Result := FStore.Keys.ToArray;
end;

function TDlpEncryptedDataStore.Count: Integer;
begin
  Result := FStore.Count;
end;

function TDlpEncryptedDataStore.ExportAsJson: string;
var
  SB: TStringBuilder;
  Pair: TPair<string, string>;
  First: Boolean;
begin
  SB := TStringBuilder.Create;
  try
    SB.Append('{');
    First := True;
    for Pair in FStore do
    begin
      if not First then
        SB.Append(', ');
      First := False;
      SB.AppendFormat('"%s": "%s"', [Pair.Key, Pair.Value]);
    end;
    SB.Append('}');
    Result := SB.ToString;
  finally
    SB.Free;
  end;
end;

procedure TDlpEncryptedDataStore.ImportFromJson(const AJson: string);
var
  CleanJson: string;
  Pairs: TArray<string>;
  Item: string;
  Parts: TArray<string>;
  K, V: string;
begin
  FStore.Clear;
  CleanJson := AJson.Trim;
  if CleanJson.StartsWith('{') then
    CleanJson := CleanJson.Substring(1);
  if CleanJson.EndsWith('}') then
```

```
    CleanJson := CleanJson.Substring(0, CleanJson.Length - 1);

  Pairs := CleanJson.Split([',']);
  for Item in Pairs do
  begin
    Parts := Item.Split([':']);
    if Length(Parts) = 2 then
    begin
      K := Parts[0].Trim.Replace('"', '');
      V := Parts[1].Trim.Replace('"', '');
      if K <> '' then
        FStore.AddOrSetValue(K, V);
    end;
  end;
  DoChanged;
end;

procedure TDlpEncryptedDataStore.DoChanged;
begin
  if Assigned(FOnDataStoreChanged) then
    FOnDataStoreChanged(Self, FStore.Count, FLastPersistedBytes);
end;

end.
```

7.6 6. Checklist de Engenharia do Módulo 03

- [x] O streaming DFM transforma objetos em representação declarativa em texto e a compila em recurso binário (`{$R *.dfm}`).
- [x] Use a cláusula `default` para poupar bytes e tempo de parsing durante o carregamento de formulários.
- [x] Use métodos com o especificador `stored` para filtrar propriedades dinâmicas ou dependentes de contexto.
- [x] Sobrescreva `DefineProperties` e utilize `Filer.DefineBinaryProperty` para persistir dados binários, coleções genéricas (`TDictionary`) ou payloads complexos sem poluir a visibilidade `published` .
- [x] Sempre grave metadados de cabeçalho (flags de criptografia, versão de payload, quantidade de itens) antes do corpo binário para garantir resiliência e evolução futura da sua suíte de componentes.

8 MÓDULO 04: Tratamento de Mensagens do SO e Arquitetura de Eventos

A interface gráfica do Windows é um sistema puramente guiado por mensagens (*Message-Driven Architecture*). No **Delphi 12 Athens+**, a **VCL** encapsula a complexidade do loop de mensagens Win32/64 com uma elegância inigualável através do método virtual `WndProc` e da sintaxe de métodos de mensagem `message WM_*`.

No entanto, no desenvolvimento avançado de componentes, frequentemente precisamos ir além do óbvio: precisamos interceptar dinamicamente o fluxo de janelas de terceiros sem recorrer a herança direta (via **Subclassing com** `WindowProc`), despachar eventos assíncronos imunes a congelamentos e deadlocks na UI, e integrar eventos clássicos com **métodos anônimos e closures modernos**.

Neste módulo, realizaremos um raio-x completo do pipeline de mensagens da VCL, confrontaremos as estratégias de despacho síncrono e assíncrono, e implementaremos um componente de interceptação avançado para o Delphi 12.

8.1 1. O Pipeline de Mensagens do Windows na VCL

No Windows, cada janela física (`HWND`) possui uma função de procedimento de janela (*Window Procedure* ou `WndProc`) associada no registro da classe do SO.

Quando um evento ocorre (uma tecla é pressionada, o mouse se move ou a janela é redimensionada), o sistema operacional deposita uma estrutura `MSG` na fila da thread. A VCL captura essa mensagem através de `Application.ProcessMessage` e a entrega ao componente correspondente.

```
sequenceDiagram
    autonumber
    participant SO as Windows OS (Win32/64 Subsystem)
    participant APP as Vcl.Forms.Application
    participant CTRL as TWinControl (Target)
    participant HOOK as TDlpWindowsMessageInterceptor
    participant USER as Código do Desenvolvedor

    SO-->>APP: PostMessage(HWND, WM_KEYDOWN...)
    APP-->>CTRL: DispatchMessage(&Msg)
    CTRL-->>HOOK: InterceptorWindowProc(var Message)
    Note over HOOK: Avalia mensagem interceptada.<br/>Dispara OnMessageIntercepted

    HOOK-->>USER: OnMessageIntercepted(Self, MsgInfo)
    alt Handled = True
        Note over HOOK: Mensagem consumida!<br/>Message.Result := 0
        HOOK-->>CTRL: Retorna sem repassar
    else Handled = False
        HOOK-->>CTRL: FOriginalWindowProc(Message)
        Note over CTRL: Processamento nativo da VCL
    end
```

8.2 2. Métodos de Despacho: Confronto Técnico

Ao emitir ou propagar mensagens no Delphi, existem quatro rotas com comportamentos operacionais diametralmente distintos:

Mecanismo de Despacho	Como Executa	Cruza a Fila do SO?	Retorna Imediatamente?	Risco de Deadlock
`Perform(Msg, WParam, LParam)`	Chama diretamente a `WndProc` do objeto Pascal em memória.	**Não**. É uma chamada de método Pascal pura *O*(1).	Não (é síncrono).	Baixo (a menos que haja reentrância).
`SendMessage(HWND, Msg, ...)`	Invoca a API do Windows. A thread aguarda o retorno da janela de destino.	**Sim**. Cruza o subsistema do SO.	Não (bloqueia o emissor até a resposta).	**Alto** (se a janela de destino estiver bloqueada pela thread emissora).
`PostMessage(HWND, Msg, ...)`	Deposita a mensagem na fila da janela do Windows e retorna instantaneamente.	**Sim**.	**Sim** (assíncrono).	Quase nulo.
`TThread.Queue(nil, AnonProc)`	Agenda uma rotina anônima na fila da UI thread do Delphi.	Não passa pelo Win32, opera no subsistema da RTL.	**Sim** (desacoplamento assíncrono).	**Zero** (a thread de fundo nunca é bloqueada).

8.3 3. Subclassing Dinâmico de Janelas na VCL

Quando criamos um componente derivado de `TWinControl` (como um botão ou painel), podemos interceptar mensagens declarando métodos com a palavra reservada `message`:

```
procedure WMKeyDown(var Message: TWMKeyDown); message WM_KEYDOWN;
```

Porém, quando desenvolvemos um componente não-visual utilitário (como um logger de ações, um validador de formulário ou um assistente de atalhos globais), precisamos interceptar mensagens de um `TForm` ou `TEdit` já colocado na tela.

A VCL disponibiliza uma propriedade extraordinária em todo `TWinControl`:

```
property WindowProc: TWndMethod read FWindowProc write FWindowProc;
```

Onde `TWndMethod` é o delegate canônico:

```
TWndMethod = procedure(var Message: TMessage) of object;
```

Ao substituir temporariamente o `WindowProc` do alvo pelo nosso método, obtemos controle absoluto sobre todas as mensagens que chegam ao controle antes que a VCL as processe. Se nossa lógica marcar `MsgInfo.Handled := True`, a mensagem é abortada na raiz.

8.4 4. Modern Event Architecture: Delegates Clássicos vs. Métodos Anônimos

O Delphi 12 permite unificar a arquitetura de eventos tradicional com closures modernos.

8.4.1 4.1 O Formato Clássico: `of object`

```
TOnWindowsMessageIntercepted = procedure(Sender: TObject; var AMsgInfo: TDlpMessageInfo) of
        object;
```

Permite ligação direta no Object Inspector em tempo de design.

8.4.2 4.2 Métodos Anônimos com Captura de Escopo (*Closures*)

```
TDlpAsyncMessageCallback = reference to procedure(const AToken: Int64; const APayload: string);
```

Permite que o desenvolvedor passe blocos de código inline no momento exato do disparo:

```
Interceptor.QueueAnonymousCallback(
  procedure(const AToken: Int64; const APayload: string)
  begin
    ShowMessage(Format('Notificação %d recebida: %s', [AToken, APayload]));
  end, 'Dados do Sensor');
```

As variáveis locais capturadas pelo método anônimo têm seu tempo de vida gerenciado automaticamente pela RTL através de interfaces ocultas geradas pelo compilador.

8.5 5. Implementação Prática: O Componente `TDlpWindowsMessageInterceptor`

Abaixo está a unidade completa e compilável do `TDlpWindowsMessageInterceptor`:

```
{===============================================================================
  Livro: Desenvolvimento Moderno de Componentes no Delphi: VCL, FireMonkey e ToolsAPI
  Módulo 04: Tratamento de Mensagens do SO e Arquitetura de Eventos
  Componente: TDlpWindowsMessageInterceptor
  Autor: Engenharia de Software Especializada & Arquitetura RAD
  Ambiente: Delphi 12 Athens+ (Win32, Win64 e VCL)
  Descrição: Componente não-visual que realiza subclassing avançado de janelas
             VCL via WindowProc, provê interceptação com cancelamento de mensagens
             nativas do Windows (WM_*), despachos assíncronos desacoplados via
             TThread.Queue e suporte a eventos modernos com métodos anônimos.
===============================================================================}
unit DlpWindowsMessageInterceptor;

interface

uses
  Winapi.Windows,
  Winapi.Messages,
  System.SysUtils,
  System.Classes,
  Vcl.Controls,
  Vcl.Forms;

type
  { Dados estruturados da mensagem do Windows interceptada }
  TDlpMessageInfo = record
    Msg: Cardinal;
    WParam: WPARAM;
    LParam: LPARAM;
    Handled: Boolean;
    Result: LRESULT;
    Timestamp: TDateTime;
  end;

  { Delegate clássico com parâmetros tipados }
  TOnWindowsMessageIntercepted = procedure(Sender: TObject;
    var AMsgInfo: TDlpMessageInfo) of object;

  { Delegate assíncrono moderno com suporte a métodos anônimos }
  TDlpAsyncMessageCallback = reference to procedure(const AToken: Int64;
    const APayload: string);

  { Evento publicado convencional para notificação assíncrona }
  TOnAsyncDispatchedEvent = procedure(Sender: TObject; AToken: Int64;
    const APayload: string) of object;

  {=============================================================================
    Classe: TDlpWindowsMessageInterceptor
    Componente não-visual para interceptação de mensagens Win32/64
  =============================================================================}
  TDlpWindowsMessageInterceptor = class(TComponent)
  private
    FTargetControl: TWinControl;
    FOriginalWindowProc: TWndMethod;
    FActive: Boolean;
    FSequenceToken: Int64;
    FInterceptKeyboard: Boolean;
```

```
    FInterceptMouse: Boolean;
    FInterceptSystem: Boolean;
    FOnMessageIntercepted: TOnWindowsMessageIntercepted;
    FOnAsyncDispatched: TOnAsyncDispatchedEvent;

    procedure SetTargetControl(const AValue: TWinControl);
    procedure SetActive(const AValue: Boolean);
    procedure AttachSubclass;
    procedure DetachSubclass;

    { Procedimento de janela substituto }
    procedure InterceptorWindowProc(var Message: TMessage);
  protected
    procedure Notification(AComponent: TComponent; Operation: TOperation); override;
    procedure Loaded; override;
  public
    constructor Create(AOwner: TComponent); override;
    destructor Destroy; override;

    { Despacho de mensagens assíncronas thread-safe para a fila da UI }
    procedure QueueAsyncNotification(const APayload: string);
    procedure QueueAnonymousCallback(ACallback: TDlpAsyncMessageCallback;
      const APayload: string);
  published
    property TargetControl: TWinControl read FTargetControl write SetTargetControl;
    property Active: Boolean read FActive write SetActive default True;
    property InterceptKeyboard: Boolean read FInterceptKeyboard write FInterceptKeyboard
        default True;
    property InterceptMouse: Boolean read FInterceptMouse write FInterceptMouse default False;
    property InterceptSystem: Boolean read FInterceptSystem write FInterceptSystem default
        False;

    property OnMessageIntercepted: TOnWindowsMessageIntercepted read FOnMessageIntercepted
        write FOnMessageIntercepted;
    property OnAsyncDispatched: TOnAsyncDispatchedEvent read FOnAsyncDispatched write
        FOnAsyncDispatched;
  end;

procedure Register;

implementation

procedure Register;
begin
  RegisterComponents('Engenharia Delphi 12', [TDlpWindowsMessageInterceptor]);
end;

{ TDlpWindowsMessageInterceptor }

constructor TDlpWindowsMessageInterceptor.Create(AOwner: TComponent);
begin
  inherited Create(AOwner);
  FTargetControl := nil;
  FOriginalWindowProc := nil;
  FActive := True;
  FSequenceToken := 0;
  FInterceptKeyboard := True;
  FInterceptMouse := False;
```

```
  FInterceptSystem := False;
end;

destructor TDlpWindowsMessageInterceptor.Destroy;
begin
  DetachSubclass;
  inherited Destroy;
end;

procedure TDlpWindowsMessageInterceptor.Loaded;
begin
  inherited Loaded;
  if FActive and Assigned(FTargetControl) then
    AttachSubclass;
end;

procedure TDlpWindowsMessageInterceptor.SetTargetControl(const AValue: TWinControl);
begin
  if FTargetControl <> AValue then
  begin
    DetachSubclass;

    if Assigned(FTargetControl) then
      FTargetControl.RemoveFreeNotification(Self);

    FTargetControl := AValue;

    if Assigned(FTargetControl) then
    begin
      FTargetControl.FreeNotification(Self);
      if FActive and not (csLoading in ComponentState) then
        AttachSubclass;
    end;
  end;
end;

procedure TDlpWindowsMessageInterceptor.SetActive(const AValue: Boolean);
begin
  if FActive <> AValue then
  begin
    FActive := AValue;
    if not (csLoading in ComponentState) and not (csDesigning in ComponentState) then
    begin
      if FActive then
        AttachSubclass
      else
        DetachSubclass;
    end;
  end;
end;

procedure TDlpWindowsMessageInterceptor.AttachSubclass;
begin
  if csDesigning in ComponentState then
    Exit;

  if Assigned(FTargetControl) and (FOriginalWindowProc = nil) then
```

```
  begin
    FOriginalWindowProc := FTargetControl.WindowProc;
    FTargetControl.WindowProc := InterceptorWindowProc;
  end;
end;

procedure TDlpWindowsMessageInterceptor.DetachSubclass;
begin
  if Assigned(FTargetControl) and Assigned(FOriginalWindowProc) then
  begin
    FTargetControl.WindowProc := FOriginalWindowProc;
    FOriginalWindowProc := nil;
  end;
end;

procedure TDlpWindowsMessageInterceptor.Notification(AComponent: TComponent; Operation:
        TOperation);
begin
  inherited Notification(AComponent, Operation);
  if (Operation = opRemove) and (AComponent = FTargetControl) then
  begin
    FOriginalWindowProc := nil;
    FTargetControl := nil;
  end;
end;

procedure TDlpWindowsMessageInterceptor.InterceptorWindowProc(var Message: TMessage);
var
  MsgInfo: TDlpMessageInfo;
  ShouldIntercept: Boolean;
begin
  ShouldIntercept := False;

  if FInterceptKeyboard and (Message.Msg in [WM_KEYDOWN, WM_KEYUP, WM_CHAR, WM_SYSKEYDOWN,
        WM_SYSKEYUP]) then
    ShouldIntercept := True;

  if FInterceptMouse and (Message.Msg in [WM_LBUTTONDOWN, WM_RBUTTONDOWN, WM_MOUSEMOVE,
        WM_MOUSEWHEEL]) then
    ShouldIntercept := True;

  if FInterceptSystem and (Message.Msg in [WM_SYSCOMMAND, WM_COMMAND, WM_SETFOCUS,
        WM_KILLFOCUS]) then
    ShouldIntercept := True;

  if ShouldIntercept and Assigned(FOnMessageIntercepted) then
  begin
    MsgInfo.Msg := Message.Msg;
    MsgInfo.WParam := Message.WParam;
    MsgInfo.LParam := Message.LParam;
    MsgInfo.Handled := False;
    MsgInfo.Result := 0;
    MsgInfo.Timestamp := Now;

    try
      FOnMessageIntercepted(Self, MsgInfo);
    except
      { Isola o pipeline de janelas contra exceções em código do consumidor }
```

```
    end;

    if MsgInfo.Handled then
    begin
      Message.Result := MsgInfo.Result;
      Exit;
    end;
  end;

  { Se não interceptada ou Handled = False, repassa para a WindowProc original }
  if Assigned(FOriginalWindowProc) then
    FOriginalWindowProc(Message);
end;

procedure TDlpWindowsMessageInterceptor.QueueAsyncNotification(const APayload: string);
var
  CurrentToken: Int64;
  SavedPayload: string;
begin
  if not Assigned(FOnAsyncDispatched) then
    Exit;

  Inc(FSequenceToken);
  CurrentToken := FSequenceToken;
  SavedPayload := APayload;

  { Enfileira de forma assíncrona na UI thread sem risco de deadlocks }
  TThread.Queue(nil,
    procedure
    begin
      if Assigned(FOnAsyncDispatched) and not (csDestroying in ComponentState) then
        FOnAsyncDispatched(Self, CurrentToken, SavedPayload);
    end);
end;

procedure TDlpWindowsMessageInterceptor.QueueAnonymousCallback(
  ACallback: TDlpAsyncMessageCallback; const APayload: string);
var
  CurrentToken: Int64;
  SavedPayload: string;
  SavedCallback: TDlpAsyncMessageCallback;
begin
  if not Assigned(ACallback) then
    Exit;

  Inc(FSequenceToken);
  CurrentToken := FSequenceToken;
  SavedPayload := APayload;
  SavedCallback := ACallback;

  TThread.Queue(nil,
    procedure
    begin
      if not (csDestroying in ComponentState) then
        SavedCallback(CurrentToken, SavedPayload);
    end);
end;
```

```
end.
```

8.6 6. Checklist de Engenharia do Módulo 04

- [x] Mensagens nativas do Windows são despachadas pela função de procedimento de janela (`WndProc`).
- [x] Use `Perform` para comunicação síncrona direta entre controles VCL sem intermediários no SO.
- [x] O encadeamento de `WindowProc` permite interceptar e abortar mensagens (`Handled := True`) de controles externos sem requerer herança de classe.
- [x] Nunca ative hooks em tempo de design (`csDesigning in ComponentState`).
- [x] Para comunicação entre threads de segundo plano e a interface gráfica, priorize `TThread.Queue` com métodos anônimos para erradicar totalmente o risco de deadlocks.

9 MÓDULO 05: Empacotamento Profissional (Packages .dpk)

No ecossistema **Delphi 12 Athens+**, o empacotamento correto de componentes em pacotes compilados (**BPL - Borland/Embarcadero Package Library**) é o divisor de águas entre um código amador e uma suíte corporativa de nível industrial.

Desenvolvedores inexperientes frequentemente misturam a unidade `DesignIntf` ou o pacote `designide.dcp` diretamente em seus componentes visuais ou de lógica. Isso desencadeia um desastre de distribuição: a aplicação cliente passa a exigir a BPL de design do Delphi em tempo de execução, gerando erros de licença, falhas de DLL não encontrada na máquina do usuário final e impedindo a compilação em arquiteturas de 64 bits para servidores de backend.

Neste módulo, aprenderemos a desenhar a arquitetura estrita de desacoplamento entre pacotes **Runtime (`{$RUNONLY}`)** e **Design-Time (`{$DESIGNONLY}`)**, como compilar ícones de alta resolução e como automatizar a compilação de binários Win32 e Win64 utilizando o **MSBuild**.

9.1 1. A Arquitetura Obrigatória de Pacotes no Delphi

A estrutura profissional de componentes no Delphi requer dois pacotes cooperantes:

```
graph TD
    subgraph APP ["Aplicação Cliente (Win32 / Win64)"]
        EXE["Executável Final (.exe)"]
    end

    subgraph RT ["Pacote de Runtime (.dpk / .bpl)"]
        RT_DPK["DlpComponents_RT.dpk<br/>{$RUNONLY}"]
        RT_UNITS["Unidades de Domínio:<br/>– DlpTypeHierarchyInspector.pas<br/>–
DlpSafeLinkedContainer.pas<br/>– DlpEncryptedDataStore.pas<br/>– DlpWindowsMessageInterceptor.pas"]
        RT_REQS["requires rtl, vcl;"]
    end

    subgraph DT ["Pacote de Design-Time (.dpk / .bpl)"]
        DT_DPK["DlpComponents_DT.dpk<br/>{$DESIGNONLY}"]
        DT_UNITS["Unidade de Registro:<br/>– DlpPackageRegister.pas<br/>– Ícones .res"]
        DT_REQS["requires rtl, vcl, designide, DlpComponents_RT;"]
    end

    subgraph IDE ["Delphi 12 Athens IDE"]
        TOOL_PALETTE["Tool Palette & Component Palette"]
    end

    EXE ⟶|Vincula Estático ou Dinâmico| RT_DPK
    RT_DPK ⟶ RT_UNITS
    RT_DPK ⟶ RT_REQS

    DT_DPK ⟶|Depende de| RT_DPK
    DT_DPK ⟶ DT_UNITS
    DT_DPK ⟶ DT_REQS

    TOOL_PALETTE ⟶|Instala apenas| DT_DPK
```

9.1.1 1.1 As Diretivas de Compilação Críticas: `{$RUNONLY}` e `{$DESIGNONLY}`

1. `{$RUNONLY}` **no Pacote de Runtime:**
   - Sinaliza à IDE do Delphi que esse pacote contém exclusivamente código executável da aplicação.
   - O Delphi **proíbe terminantemente** que um pacote marcado como `{$RUNONLY}` seja instalado diretamente na Tool Palette da IDE.
   - Isso impede que o desenvolvedor acidentalmente tente registrar componentes ou vincular editores de propriedade dentro do pacote de runtime.
2. `{$DESIGNONLY}` **no Pacote de Design-Time:**
   - Sinaliza que o pacote se destina exclusivamente à IDE (`bds.exe`).
   - O compilador bloqueia que projetos finais de clientes adicionem o pacote de design-time na lista de pacotes de runtime do executável.
   - Apenas o pacote de Design-Time deve conter `requires designide;` .

9.2 2. Anatomia dos Arquivos de Projeto `.dpk`

9.2.1 2.1 Pacote de Runtime: `DlpComponents_RT.dpk`

Observe as diretivas de otimização, a diretiva `{$RUNONLY}` e as dependências mínimas necessárias:

```
package DlpComponents_RT;

{$R *.res}
{$IFDEF IMPLICITBUILDING This IFDEF should not be used by the compiler}
{$ALIGN 8}
{$ASSERTIONS ON}
{$BOOLEVAL OFF}
{$DEBUGINFO OFF}
{$EXTENDEDSYNTAX ON}
{$IMPORTEDDATA ON}
{$IOCHECKS ON}
{$LOCALSYMBOLS ON}
{$LONGSTRINGS ON}
{$OPENSTRINGS ON}
{$OPTIMIZATION LEVEL3}
{$OVERFLOWCHECKS OFF}
{$RANGECHECKS OFF}
{$REFERENCEINFO ON}
{$SAFEDIVIDE OFF}
{$STACKFRAMES OFF}
{$TYPEDADDRESS OFF}
{$VARSTRINGCHECKS ON}
{$WRITEABLECONST OFF}
{$MINENUMSIZE 1}
{$IMAGEBASE $400000}
{$DEFINE RELEASE}
{$ENDIF IMPLICITBUILDING}
{$DESCRIPTION 'Coleção de Engenharia Avançada de Componentes - Runtime'}
{$RUNONLY}
{$IMPLICITBUILD ON}

requires
  rtl,
  vcl;

contains
  DlpTypeHierarchyInspector in '..\units\DlpTypeHierarchyInspector.pas',
  DlpSafeLinkedContainer in '..\units\DlpSafeLinkedContainer.pas',
  DlpEncryptedDataStore in '..\units\DlpEncryptedDataStore.pas',
  DlpWindowsMessageInterceptor in '..\units\DlpWindowsMessageInterceptor.pas';

end.
```

9.2.2 2.2 Pacote de Design-Time: `DlpComponents_DT.dpk`

Observe que ele herda de `DlpComponents_RT` e inclui a dependência da IDE: `designide`:

```
package DlpComponents_DT;

{$R *.res}
{$IFDEF IMPLICITBUILDING This IFDEF should not be used by the compiler}
{$ALIGN 8}
{$ASSERTIONS ON}
{$BOOLEVAL OFF}
{$DEBUGINFO OFF}
{$EXTENDEDSYNTAX ON}
{$IMPORTEDDATA ON}
{$IOCHECKS ON}
{$LOCALSYMBOLS ON}
{$LONGSTRINGS ON}
{$OPENSTRINGS ON}
{$OPTIMIZATION LEVEL3}
{$OVERFLOWCHECKS OFF}
{$RANGECHECKS OFF}
{$REFERENCEINFO ON}
{$SAFEDIVIDE OFF}
{$STACKFRAMES OFF}
{$TYPEDADDRESS OFF}
{$VARSTRINGCHECKS ON}
{$WRITEABLECONST OFF}
{$MINENUMSIZE 1}
{$IMAGEBASE $400000}
{$DEFINE RELEASE}
{$ENDIF IMPLICITBUILDING}
{$DESCRIPTION 'Coleção de Engenharia Avançada de Componentes - Design-Time'}
{$DESIGNONLY}
{$IMPLICITBUILD ON}

requires
  rtl,
  vcl,
  designide,
  DlpComponents_RT;

contains
  DlpPackageRegister in '..\units\DlpPackageRegister.pas';

end.
```

9.3 3. O Procedimento `Register` e a Injeção na Tool Palette

No Delphi, a unidade de registro reside exclusivamente no pacote de design-time:

```
unit DlpPackageRegister;

interface

uses
  System.Classes,
  DesignIntf,
  DlpTypeHierarchyInspector,
  DlpSafeLinkedContainer,
  DlpEncryptedDataStore,
  DlpWindowsMessageInterceptor;

procedure Register;

implementation

{$R dlp_components_icons.res}

procedure Register;
begin
  RegisterComponents('Engenharia Delphi 12', [
    TDlpTypeHierarchyInspector,
    TDlpSafeLinkedContainer,
    TDlpEncryptedDataStore,
    TDlpWindowsMessageInterceptor
  ]);
end;

end.
```

9.4 4. Ícones de Paleta de Alta Densidade (24x24 e 32x32)

No Delphi 12 Athens+, a IDE opera em modo High-DPI nativo. Para que os ícones fiquem nítidos em monitores 4K: 1. Crie imagens em formato bitmap de 32 bits (com canal alfa) ou PNG nas resoluções **24×24** e **32×32**. 2. Crie um script de recursos (`dlp_components_icons.rc`): `rc TDLPTYPEHIERARCHYINSPECTOR BITMAP "icons\\TDlpTypeHierarchyInspector.bmp" TDLPSAFELINKEDCONTAINER BITMAP "icons\\TDlpSafeLinkedContainer.bmp" TDLPENCRYPTEDDATASTORE BITMAP "icons\\TDlpEncryptedDataStore.bmp" TDLPWINDOWSMESSAGEINTERCEPTOR BITMAP "icons\\TDlpWindowsMessageInterceptor.bmp"` 3. Compile o arquivo `.rc` com o compilador de recursos do Delphi: `cmd brcc32 dlp_components_icons.rc` 4. Vincule o arquivo compilado com `{$R dlp_components_icons.res}` na unidade de registro.

9.5 5. Compilação Automatizada via Linha de Comando com MSBuild

O Delphi utiliza arquivos de projeto `.dproj` baseados no padrão Microsoft Build Engine (**MSBuild**). Isso possibilita compilações automáticas em pipelines de CI/CD (GitHub Actions, Azure DevOps, Jenkins):

```
:: Configura o ambiente de linha de comando do Delphi 12
call "C:\Program Files (x86)\Embarcadero\Studio\23.0\bin\rsvars.bat"

:: 1. Compila o pacote de Runtime em Win32 (Release)
msbuild DlpComponents_RT.dproj /p:Config=Release /p:Platform=Win32 /t:Build

:: 2. Compila o pacote de Runtime em Win64 (Release)
msbuild DlpComponents_RT.dproj /p:Config=Release /p:Platform=Win64 /t:Build

:: 3. Compila o pacote de Design-Time em Win32 (A IDE do Delphi 12 executa em Win32/Win64)
msbuild DlpComponents_DT.dproj /p:Config=Release /p:Platform=Win32 /t:Build
```

9.6 6. Checklist de Engenharia do Módulo 05

- [x] O pacote de runtime deve ser estritamente compilado com a diretiva `{$RUNONLY}`.
- [x] O pacote de design-time deve conter `{$DESIGNONLY}` e ser o único a depender de `designide.dcp`.
- [x] Nunca referencie unidades de design (`DesignIntf`, `DesignEditors`) dentro de unidades do pacote de runtime.
- [x] O procedimento `Register` deve residir em uma unidade isolada do pacote de design-time.
- [x] Automatize suas compilações com scripts baseados em `MSBuild` para garantir suporte a plataformas Win32 e Win64 em esteiras de integração contínua.

10 MÓDULO 06: Property Editors Customizados

No **Delphi 12 Athens+**, a experiência do desenvolvedor (*Developer Experience - DX*) ao interagir com uma suíte de componentes corporativos no **Object Inspector** define o sucesso de sua adoção.

Componentes modernos não devem sobrecarregar o usuário com valores enigmáticos ou formatos textuais obscuros. Propriedades complexas demandam **editores de propriedades customizados (Property Editors)** capazes de fornecer: 1. **Dropdowns Contextuais (`paValueList`):** Listas suspensas dinâmicas de opções no Object Inspector calculadas a partir de estados do próprio componente ou de fontes de metadados externas. 2. **Editores Visuais com Janelas Modais (`paDialog`):** Diálogos com preview ao vivo, paletas de cores integradas e validações estritas, disparados pelo botão de reticências `[...]`.

Neste módulo, dissecaremos a arquitetura das interfaces da Open Tools API dedicadas ao design visual (`DesignIntf` e `DesignEditors`), especializaremos classes de `TPropertyEditor` e implementaremos dois editores completos de propriedades para o Delphi 12.

10.1 1. As Unidades Canônicas: `DesignIntf` e `DesignEditors`

No Delphi moderno, todo o suporte a extensões de tempo de design foi desacoplado da RTL principal e reside no pacote `designide.dcp`, através de duas unidades principais: - `DesignIntf`**:** Declara as interfaces fundamentais do Form Designer, como `IDesigner`, `IDesignerSelections` e o tipo `TPropertyAttributes`. - `DesignEditors`**:** Fornece as implementações base de editores de propriedades e componentes: `TPropertyEditor`, `TStringPropertyEditor`, `TIntegerPropertyEditor`, `TClassPropertyEditor` e `TComponentEditor`.

```
classDiagram
    class TPropertyEditor {
        <<DesignEditors>>
        +GetAttributes(): TPropertyAttributes
        +GetValue(): string
        +SetValue(const Value: string)
        +GetValues(Proc: TGetStrProc)
        +Edit()
        +GetComponent(Index: Integer): TPersistent
        +Designer: IDesigner
    }

    class TStringPropertyEditor {
        +GetStrValue(): string
        +SetStrValue(const Value: string)
    }

    class TDlpGradientPresetPropertyEditor {
        <<paValueList>>
        +GetAttributes()
        +GetValues()
    }

    class TDlpGradientModalPropertyEditor {
        <<paDialog>>
        +GetAttributes()
        +Edit()
    }

    TPropertyEditor <|-- TStringPropertyEditor
    TStringPropertyEditor <|-- TDlpGradientPresetPropertyEditor
    TStringPropertyEditor <|-- TDlpGradientModalPropertyEditor
```

10.2 2. A Tabela de Atributos: `TPropertyAttributes`

O método fundamental que dita o comportamento de uma propriedade no Object Inspector do Delphi é:

```
function GetAttributes: TPropertyAttributes; override;
```

Atributo	Significado na IDE	Ação Técnica Requerida
`paValueList`	Transforma a linha do Object Inspector em um ComboBox suspenso.	Sobrescrever `GetValues(Proc: TGetStrProc)`.
`paSortList`	Instrui a IDE a ordenar as strings fornecidas em ordem alfabética.	N/A (associado a `paValueList`).
`paDialog`	Adiciona o botão elíptico `[...]` na margem direita da célula.	Sobrescrever `Edit`.
`paMultiSelect`	Mantém a propriedade editável mesmo com múltiplos componentes selecionados.	Suportar múltiplos alvos via `PropCount` e `GetComponent(I)`.
`paSubProperties`	Habilita o símbolo de expansão `+` para navegar em sub-objetos `TPersistent`.	Padrão em `TClassPropertyEditor`.
`paAutoUpdate`	Dispara `SetValue` imediatamente a cada tecla digitada sem esperar o `Enter`.	Ideal para filtros e previews dinâmicos.

10.3 3. Construção do Dropdown de Valores com `paValueList`

Quando o usuário precisa selecionar um estilo visual predefinido para o componente `TDlpGradientHeader`:

```
type
  TDlpGradientPresetPropertyEditor = class(TStringPropertyEditor)
  public
    function GetAttributes: TPropertyAttributes; override;
    procedure GetValues(Proc: TGetStrProc); override;
  end;
```

10.3.1 O Despacho de Valores em `GetValues`:

```
procedure TDlpGradientPresetPropertyEditor.GetValues(Proc: TGetStrProc);
var
  HeaderComp: TDlpGradientHeader;
  PresetName: string;
begin
  if GetComponent(0) is TDlpGradientHeader then
  begin
    HeaderComp := TDlpGradientHeader(GetComponent(0));
    for PresetName in HeaderComp.GetAvailablePresets do
      Proc(PresetName); // Injeta o item na lista suspensa da IDE
  end;
end;
```

A função de callback `Proc` é invocada repetidamente, alimentando a lista suspensa diretamente com os presets suportados pela instância do componente.

10.4 4. Janelas Modais com Preview ao Vivo e `Designer.Modified`

Para editar o gradiente visual com precisão matemática, uma linha de texto no Object Inspector é insuficiente. Construímos um diálogo modal que contém uma instância do próprio componente atuando como painel de preview em tempo real:

```
procedure TDlpGradientModalPropertyEditor.Edit;
var
  Dlg: TForm;
  PreviewPanel: TDlpGradientHeader;
  // ...
begin
  // Cria diálogo de configuração em tempo de design
  if Dlg.ShowModal = mrOk then
  begin
    HeaderComp.StartColor := SelectedStart;
    HeaderComp.EndColor := SelectedEnd;
    SetStrValue(Format('#%0.6X → #%0.6X', [SelectedStart, SelectedEnd]));
    Designer.Modified;
  end;
end;
```

> **REGRA DE OURO DA TOOLSAPI: DESIGNER.MODIFIED** Se você alterar propriedades de um componente programaticamente a partir de uma janela de diálogo modal ou wizard, você **deve** invocar obrigatoriamente `Designer.Modified`. Sem essa chamada, a IDE do Delphi não saberá que o formulário foi alterado. O usuário fechará o Delphi e suas alterações serão perdidas silenciosamente porque o arquivo `.dfm` não foi marcado como “sujo” (*dirty*)!

10.5 5. O Procedimento de Registro `RegisterPropertyEditor`

No arquivo de registro do pacote de design-time:

```
procedure Register;
begin
  RegisterPropertyEditor(TypeInfo(string), TDlpGradientHeader, 'PresetName',
    TDlpGradientPresetPropertyEditor);

  RegisterPropertyEditor(TypeInfo(string), TDlpGradientHeader, 'TitleText',
    TDlpGradientModalPropertyEditor);
end;
```

10.6 6. Implementação Prática: Componente e Editores Completos

O código integral das duas unidades está organizado no repositório: - `code/units/DlpGradientHeader.pas`: Controle visual de cabeçalho com algoritmo de interpolação matemática de gradiente RGB em GDI pura com Double Buffering. - `code/units/DlpGradientPropEditor.pas`: Unidade de design-time contendo o dropdown dinâmico de presets e o editor modal completo com seleção de cores via `TColorDialog` e preview ao vivo.

10.7 7. Checklist de Engenharia do Módulo 06

- [x] Isole todo o código de Property Editors e dependências de `DesignIntf` e `DesignEditors` no pacote de design-time (`DlpComponents_DT.dpk`).
- [x] Utilize `paValueList` para eliminar digitação manual de valores discretos.
- [x] Utilize `paDialog` e o método `Edit` para criar interfaces modais enriquecidas para configurações complexas.
- [x] Sempre invoque `Designer.Modified` após aplicar modificações em diálogos modais para disparar a persistência imediata no `.dfm`.
- [x] Use `GetComponent(0)` para recuperar a instância alvo selecionada pelo desenvolvedor no Form Designer.

11 MÓDULO 07: Component Editors e Ações de Design-Time (Verbos)

Enquanto o *Object Inspector* lida com a configuração de propriedades pontuais, o **Form Designer** do Delphi é o coração da experiência visual RAD. Quando o desenvolvedor clica com o botão direito em um componente colocado na tela ou executa um duplo-clique sobre ele, a IDE precisa saber como reagir.

A classe `TComponentEditor` (declarada em `DesignEditors`) é o instrumento oficial da Open Tools API do **Delphi 12 Athens+** para transformar controles passivos em instrumentos interativos de produtividade.

Neste módulo, aprenderemos como injetar verbos customizados no menu de contexto do formulário, controlar a ação padrão de duplo-clique através do método `Edit`, garantir a persistência de modificações com `Designer.Modified` e registrar editores na arquitetura de design-time do Delphi.

11.1 1. Arquitetura de Component Editors no Delphi 12

Toda a interação com componentes no Form Designer do Delphi passa pela interface `TComponentEditor`:

```
classDiagram
    class TComponentEditor {
        <<DesignEditors>>
        #Component: TComponent
        #Designer: IDesigner
        +Edit()
        +ExecuteVerb(Index: Integer)
        +GetVerb(Index: Integer): string
        +GetVerbCount(): Integer
        +PrepareItem(Index: Integer; const AItem: IMenuItem)
        +Designer.Modified()
    }

    class TDefaultEditor {
        <<Gera métodos de eventos automáticos>>
        +Edit()
    }

    class TDlpDashboardCardEditor {
        <<Menu Contextual Dinâmico>>
        +GetVerbCount(): Integer
        +GetVerb(Index: Integer): string
        +ExecuteVerb(Index: Integer)
        +Edit()
    }

    TComponentEditor <|-- TDefaultEditor
    TComponentEditor <|-- TDlpDashboardCardEditor
```

11.1.1 1.1 Ciclo de Vida do Editor de Componente

Diferente do componente em si (que vive enquanto o formulário estiver aberto no designer), o `TComponentEditor` é um objeto transitório e efêmero: 1. Quando o usuário clica com o botão direito no

controle ou clica duas vezes com o mouse, a IDE consulta a tabela interna de editores registrados. 2. A IDE instancia o `TComponentEditor` correspondente e injeta a referência `Component` e o serviço `Designer`. 3. A IDE consulta `GetVerbCount` e `GetVerb` para montar o menu suspenso ou executa `Edit`. 4. Assim que a ação é concluída ou o menu é fechado, a instância do editor é destruída.

11.2 2. A Trinca de Métodos de Verbos

Para injetar ações no menu de contexto, sobrescrevemos:

11.2.1 2.1 `GetVerbCount: Integer; override;`

Retorna o número total de comandos do menu:

```
function TDlpDashboardCardEditor.GetVerbCount: Integer;
begin
  Result := 5;
end;
```

11.2.2 2.2 `GetVerb(Index: Integer): string; override;`

Retorna o rótulo textual de cada item. Retornar `'-'` instrui a IDE a renderizar um separador gráfico:

```
function TDlpDashboardCardEditor.GetVerb(Index: Integer): string;
begin
  case Index of
    0: Result := 'Alternar Tendência (Up / Down / Neutral)';
    1: Result := 'Inverter Tema (Dark / Light)';
    2: Result := 'Carregar Dados de Exemplo';
    3: Result := '-';
    4: Result := 'Sobre o TDlpDashboardCard...';
  else
    Result := '';
  end;
end;
```

11.2.3 2.3 `ExecuteVerb(Index: Integer); override;`

Executa o comando e notifica a IDE que o componente foi modificado:

```
procedure TDlpDashboardCardEditor.ExecuteVerb(Index: Integer);
var
  Card: TDlpDashboardCard;
begin
  Card := TDlpDashboardCard(Component);
  case Index of
    0:
      begin
        Card.CycleTrend;
        Designer.Modified; // Notifica a IDE para persistir no .dfm
      end;
    // ...
  end;
end;
```

11.3 3. O Duplo-Clique: Sobrescrita do Método `Edit`

Por padrão, a classe `TDefaultEditor` procura o primeiro evento da seção `published` e gera um esqueleto de método no código Pascal.

Quando herdamos de `TComponentEditor`, podemos redirecionar o duplo-clique para executar diretamente uma ação de produtividade:

```
procedure TDlpDashboardCardEditor.Edit;
begin
  ExecuteVerb(0); // Alterna imediatamente a tendência do cartão no Form Designer
end;
```

11.4 4. O Registro com `RegisterComponentEditor`

Para associar o editor à classe do componente dentro do pacote de design-time:

```
procedure Register;
begin
  RegisterComponentEditor(TDlpDashboardCard, TDlpDashboardCardEditor);
end;
```

11.5 5. Implementação Prática: Componente e Editor Completos

As duas unidades correspondentes foram estruturadas em: - `code/units/DlpDashboardCard.pas`: Controle visual de cartão de KPI corporativo com desenho vetorial de badges de tendência (`tdUp`, `tdDown`, `tdNeutral`), suporte a Double Buffering e temas escuro/claro. - `code/units/DlpDashboardCardEditor.pas`:

Editor de componente especializado com 5 verbos de menu de contexto, alternância dinâmica de métricas e notificação de formulário modificado (`Designer.Modified`).

11.6 6. Checklist de Engenharia do Módulo 07

- [x] O `TComponentEditor` é instanciado sob demanda pela IDE e desalocado logo após a execução da ação.
- [x] Utilize `'-'` em `GetVerb` para organizar grupos conceituais de comandos no menu de contexto.
- [x] Sobrescreva `Edit` para substituir a geração automática de eventos por ações especializadas de design-time.
- [x] Nunca esqueça de chamar `Designer.Modified` após manipular o componente em tempo de design.
- [x] Registre o editor com `RegisterComponentEditor` exclusivamente no pacote de design-time (`DlpComponents_DT.dpk`).

12 MÓDULO 08: Assistentes de Projeto e Extensão da IDE (ToolsAPI Profunda)

No **Delphi 12 Athens+**, a **Open Tools API (OTA)** é o ápice da engenharia de integração com o ambiente de desenvolvimento. Através de um conjunto de interfaces COM puras (`ToolsAPI.pas`), a Embarcadero permite que arquitetos de software modifiquem radicalmente o comportamento do Delphi, adicionem novos itens à galeria de projetos, manipulem buffers de código em memória e automatizem a injeção de padrões de projeto complexos.

Em organizações que adotam práticas modernas como **Domain-Driven Design (DDD)** ou **Clean Architecture**, a ToolsAPI é a ferramenta que transforma diretrizes abstratas em automação concreta.

Neste módulo, aprenderemos a arquitetura interna da ToolsAPI, implementaremos o assistente corporativo `TDlpCleanArchProjectWizard` e dominaremos a geração de módulos e manipulação de código-fonte em tempo de design.

12.1 1. A Arquitetura da Open Tools API (OTA)

A ToolsAPI é fundamentada no padrão *Service Locator* através da variável global `BorlandIDEServices`. Toda comunicação com a IDE ocorre via consulta de interfaces (`QueryInterface` / `Supports`).

```
classDiagram
    class BorlandIDEServices {
        <<Service Locator Global>>
    }

    class IOTAModuleServices {
        +GetModuleCount(): Integer
        +GetModule(Index): IOTAModule
        +CreateModule(Creator: IOTACreator): IOTAModule
    }

    class IOTAWizard {
        +GetIDString(): string
        +GetName(): string
        +GetState(): TWizardState
        +Execute()
    }

    class IOTARepositoryWizard {
        <<Galeria File → New → Other...>>
        +GetAuthor(): string
        +GetComment(): string
        +GetPage(): string
        +GetGlyph(): Cardinal
    }

    class IOTAMenuWizard {
        <<Injeta no Menu Principal>>
        +GetMenuText(): string
    }

    class TDlpCleanArchProjectWizard {
        <<Wizard Corporativo>>
        +Execute()
    }

    BorlandIDEServices ..> IOTAModuleServices : Supports()
    IOTAWizard <|-- IOTARepositoryWizard
    IOTAWizard <|-- IOTAMenuWizard
    IOTARepositoryWizard <|-- TDlpCleanArchProjectWizard
    IOTAMenuWizard <|-- TDlpCleanArchProjectWizard
```

12.1.1 1.1 As Quatro Interfaces de um Wizard Completo

1. `IOTAWizard`: Interface atômica obrigatória. Define a identidade (`GetIDString`), o nome exibido (`GetName`), o estado de ativação (`GetState`) e a ação executada ao ser clicado (`Execute`).
2. `IOTARepositoryWizard`: Indica à IDE que o assistente deve aparecer na árvore de categorias da galeria de novos itens (`File → New → Other...`), organizando-se sob a aba retornada por `GetPage`.
3. `IOTAMenuWizard`: Injeta automaticamente um item executável no menu principal do Delphi sob o menu `Help` ou menu customizado.
4. `IOTACreator` / `IOTAModuleCreator`: O contrato responsável por sintetizar os arquivos em memória antes mesmo de serem salvos no disco.

12.2 2. A Síntese de Arquivos em Memória: `IOTAFile` e `IOTAModuleCreator`

Quando um assistente gera um novo arquivo, ele **não deve** salvá-lo no disco rígido usando `TFileStream`. Se o usuário decidir cancelar a operação ou fechar a aba sem salvar, o disco não deve conter lixo

temporário.

A ToolsAPI fornece a interface de buffer virtual:

```
type
  TDlpOTAFileBuffer = class(TInterfacedObject, IOTAFile)
  private
    FSourceCode: string;
  public
    constructor Create(const ASource: string);
    function GetSource: string;
    function GetAge: TDateTime;
  end;
```

Ao retornar `-1` em `GetAge`, sinalizamos ao Delphi que o arquivo é totalmente novo e ainda não existe no sistema de arquivos.

12.3 3. A Execução do Wizard: Injeção Via `IOTAModuleServices`

No método `Execute` da nossa classe:

```
procedure TDlpCleanArchProjectWizard.Execute;
var
  ModServices: IOTAModuleServices;
begin
  if Supports(BorlandIDEServices, IOTAModuleServices, ModServices) then
  begin
    // Injeta a nova unit de Clean Architecture no editor de código ativo!
    ModServices.CreateModule(TDlpCleanModuleCreator.Create);
  end;
end;
```

A chamada `ModServices.CreateModule` cria uma nova aba no Code Editor do Delphi 12, contendo o esqueleto completo da camada de Domínio (`ICustomerRepository`) e da camada de Casos de Uso (`TCustomerService`), com suporte a syntax highlighting imediato.

12.4 4. O Registro com `RegisterPackageWizard`

Assistentes da ToolsAPI são registrados através da função global da unidade `ToolsAPI`:

```
procedure Register;
begin
  RegisterPackageWizard(TDlpCleanArchProjectWizard.Create as IOTAWizard);
end;
```

Esse registro é realizado estritamente no pacote de design-time (`DlpComponents_DT.dpk`).

12.5 5. Implementação Prática: O Assistente Completo

A unidade completa está localizada em: - `code/units/DlpCleanArchProjectWizard.pas`: Contém o buffer virtual `TDlpOTAFileBuffer`, o criador de módulos `TDlpCleanModuleCreator`, o wizard multiplataforma `TDlpCleanArchProjectWizard` e o registro oficial na ToolsAPI.

12.6 6. Checklist de Engenharia do Módulo 08

- [x] Utilize `BorlandIDEServices` e `Supports` para localizar serviços da ToolsAPI com segurança de tipo.
- [x] Nunca salve arquivos físicos diretamente no disco ao executar um wizard; utilize sempre implementações de `IOTAFile`.
- [x] Combine `IOTARepositoryWizard` e `IOTAMenuWizard` para máxima acessibilidade na interface da IDE.
- [x] Use `GetUnnamed = True` em criadores de módulo para que a IDE solicite o nome e diretório ao usuário no momento do salvamento (`Save / Save As`).
- [x] Registre wizards usando `RegisterPackageWizard` exclusivamente no pacote de design-time.

13 MÓDULO 09: Controles Visuais Leves (TGraphicControl)

No ecossistema **Delphi 12 Athens+**, a **VCL** disponibiliza duas vias principais para criação de interfaces visuais: controles janelados pesados (`TWinControl`), que possuem um identificador nativo de janela (`HWND`), e controles gráficos leves (`TGraphicControl`), que operam sem handle (*windowless*).

Ao projetar dashboards industriais, cockpits de telemetria ou consoles de monitoramento com dezenas de mostradores gráficos e indicadores analógicos (VU Meters), instanciar múltiplos `TWinControl` impõe uma sobrecarga severa à tabela de handles da sessão do Windows (`USER32` / `GDI32`).

A solução de alta performance é o uso de `TGraphicControl` combinado com **Double Buffering Manual**, cálculo trigonométrico preciso e renderização direta no `Canvas`.

Neste módulo, desvendaremos o pipeline gráfico de controles leves na VCL, implementaremos a matemática polar para instrumentos analógicos e construiremos o componente visual `TDlpAnalogGauge`.

13.1 1. O Funcionamento de `TGraphicControl` na VCL

Diferente de um `TPanel` ou `TEdit`, o `TGraphicControl` não consome recursos de janela do Windows:

```
sequenceDiagram
    autonumber
    participant DWM as Windows Desktop Window Manager
    participant PARENT as Painel / Form Pai (TWinControl)
    participant GAUGE as TDlpAnalogGauge (TGraphicControl)
    participant MEM as Buffer TBitmap em Memória

    DWM-->>PARENT: WM_PAINT
    PARENT-->>PARENT: Pinta superfície cliente
    PARENT-->>GAUGE: Dispara Paint() passando DC do Pai

    Note over GAUGE,MEM: 1. Aloca buffer offscreen em memória
    GAUGE-->>MEM: Desenha mostrador, escalas e zonas coloridas
    GAUGE-->>MEM: Calcula coordenadas polares e desenha agulha
    GAUGE-->>MEM: Renderiza textos e leituras digitais

    Note over MEM,GAUGE: 2. BitBlt atômico de volta para a tela
    MEM-->>GAUGE: Canvas.Draw(0, 0, BufferBmp)
    Note over GAUGE: Zero Flickering garantido!
```

> **IMPORTANTE: DOUBLEBUFFERED EM TGRAPHICCONTROL** A propriedade `DoubleBuffered` existe na classe `TWinControl`, mas **não** existe em `TGraphicControl`! Por isso, para controles gráficos leves não cintilarem durante atualizações intensivas de telemetria, o arquiteto de software deve implementar o padrão **Manual Offscreen Double Buffering** instanciando um `TBitmap` interno dentro do método virtual `Paint`.

13.2 2. A Matemática Polar dos Instrumentos Circulares

O mostrador do `TDlpAnalogGauge` opera sobre um arco de 270°, iniciando em 225° (extremo inferior esquerdo) e terminando em −45° (extremo inferior direito).

13.2.1 2.1 Conversão de Escala Linear para Ângulo:

$$\text{Fração} = \frac{\text{Valor} - \text{MinValue}}{\text{MaxValue} - \text{MinValue}}$$

$$\text{Ângulo (graus)} = 225.0 - (\text{Fração} \times 270.0)$$

13.2.2 2.2 Transformação de Coordenadas Polares em Cartesianas:

Como a origem $(0, 0)$ do `TCanvas` localiza-se no canto superior esquerdo da tela e o eixo vertical cresce para baixo, invertemos o sinal do seno:

$$\theta = \text{DegToRad}(\text{Ângulo})$$

$$X = C_x + R \times \cos(\theta)$$

$$Y = C_y - R \times \sin(\theta)$$

13.3 3. Implementação Prática: O Componente `TDlpAnalogGauge`

O código-fonte integral está estruturado em: - `code/units/DlpAnalogGauge.pas`: Mostrador analógico industrial com agulha animada, zonas de alerta diferenciadas (Normal em verde, Atenção em amarelo e Crítico em vermelho escuro), display digital inferior, graduações de escala (*ticks*) e disparo de evento reativo `OnThresholdCrossed`.

13.4 4. Checklist de Engenharia do Módulo 09

- [x] Herde de `TGraphicControl` para instrumentos cosméticos de alta densidade sem entrada de teclado.
- [x] Implemente Double Buffering manual via `TBitmap` em memória para eliminar 100% da cintilação de tela.
- [x] Utilize `EnsureRange` no método assessor `SetValue` para impedir extrapolações gráficas fora do arco do mostrador.
- [x] Aplique trigonometria polar com compensação de eixo *Y* para desenho vetorial de escalas e agulhas.
- [x] Exponha eventos de transição de limite (`OnThresholdCrossed`) para integração reativa com regras de negócio ou alarmes do sistema.

14 MÓDULO 10: Controles Visuais Janelados (TWinControl)

No desenvolvimento avançado de aplicações corporativas no **Delphi 12 Athens+**, a espinha dorsal de qualquer layout de dados é formada por controles derivados de `TWinControl`.

Ao contrário dos controles leves windowless (`TGraphicControl`), um `TWinControl` é uma entidade do sistema operacional que encapsula diretamente um identificador de janela nativo (`HWND` no Windows). É essa integração com a API do Windows que confere ao controle: 1. **Identidade Física no SO:** Seu próprio loop de mensagens e capacidade de receber foco do usuário (`SetFocus`, `Focused`). 2. **Navegação por Teclado:** Participação no ciclo de tabulação (`TabStop`, `TabOrder`) e interceptação de teclas de comando (`OnKeyDown`, `OnKeyUp`). 3. **Contêiner de Filhos no Form Designer:** A faculdade de atuar como pai visual (`Parent`) de outros componentes e aceitar o drop de controles no designer através de `csAcceptsControls`. 4. **Double Buffering Nativo:** Suporte transparente à eliminação de cintilação via `DoubleBuffered := True`.

Neste módulo, dissecaremos a anatomia de `TWinControl` na VCL, contrastaremos sua mecânica com o `TStyledControl` do FireMonkey e implementaremos um contêiner sanfonado expansível completo (`TDlpAccordionContainer`) com animação a 60 FPS e acessibilidade por teclado.

14.1 1. A Anatomia do `TWinControl` na VCL

O pipeline de um controle janelado estende a hierarquia visual até o subsistema de janelas do Windows 11:

```
classDiagram
    class TComponent {
        +Owner: TComponent
    }
    class TControl {
        +Parent: TWinControl
        +Left, Top, Width, Height
    }
    class TWinControl {
        +Handle: HWND
        +HandleAllocated: Boolean
        +TabStop: Boolean
        +TabOrder: TTabOrder
        +DoubleBuffered: Boolean
        +SetFocus()
        +Focused(): Boolean
        +CreateParams(var Params: TCreateParams)
        +CreateWnd()
        +DestroyWnd()
        +WndProc(var Message: TMessage)
    }
    class TCustomControl {
        <<Fornece Canvas e Paint Virtual>>
        +Canvas: TCanvas
        +Paint()
    }
    class TDlpAccordionContainer {
        <<Painel Sanfonado Expansível>>
        +Collapsed: Boolean
        +Expand()
        +Collapse()
    }

    TComponent <|-- TControl
    TControl <|-- TWinControl
    TWinControl <|-- TCustomControl
    TCustomControl <|-- TDlpAccordionContainer
```

14.1.1 1.1 Contraste Arquitetural: VCL (`TWinControl`) vs. FireMonkey (`TStyledControl`)

- **Na VCL:** Cada `TWinControl` possui seu próprio `HWND`. O Windows é o responsável pelo recorte de tela (*clipping*), pelo roteamento de mensagens de mouse e pela hierarquia de profundidade Z-Order.
- **No FireMonkey (FMX):** Praticamente inexistem `HWNDs` para controles individuais. Toda a árvore de componentes deriva de `TFmxObject` → `TControl` → `TStyledControl`. A renderização é realizada inteiramente em uma única superfície acelerada por GPU (DirectX/Metal), onde a lógica de contêiner é puramente matemática e vetorial.

14.2 2. Hospedando Controles Filhos em Tempo de Design (`csAcceptsControls`)

Para construir um contêiner no Delphi (como o nosso `TDlpAccordionContainer` ou um card de formulário), devemos registrar no construtor:

```
ControlStyle := ControlStyle + [csAcceptsControls, csCaptureMouse, csClickEvents,
        csDoubleClicks];
DoubleBuffered := True;
TabStop := True;
```

A flag `csAcceptsControls` instrui o Form Designer do Delphi a permitir que outros controles arrastados da Tool Palette (como edits, botões, grids) sejam soltos diretamente dentro da nossa área cliente, atribuindo automaticamente `ChildControl.Parent := Self`.

14.3 3. Animação de Expansão Fluida a 60 FPS

Ao alternar a propriedade `Collapsed`: 1. Em **Design-Time (`csDesigning in ComponentState`)**: Ajustamos a altura imediatamente para não interferir na produtividade do desenvolvedor. 2. Em **Run-Time**: Ativamos um `TTimer` de alta frequência (`Interval := 15` ms) que interpola a altura corrente até `FTargetHeight` com passos constantes, produzindo uma transição orgânica e fluida.

14.4 4. Implementação Prática: O Componente `TDlpAccordionContainer`

O código-fonte integral está estruturado em: - `code/units/DlpAccordionContainer.pas`: Painel expansível construído sobre `TCustomControl`, suporte a `csAcceptsControls`, Double Buffering nativo, desenho de cabeçalho com chevron dinâmico (▼ / ►), navegação e disparo por teclado (`VK_SPACE` e `VK_RETURN`) e evento reativo `OnToggle`.

14.5 5. Checklist de Engenharia do Módulo 10

- [x] Herde de `TCustomControl` quando necessitar de um controle janelado com `HWND` e `Canvas` próprio.
- [x] Habilite `csAcceptsControls` no construtor para viabilizar a inserção de controles filhos em tempo de design.
- [x] Ative `DoubleBuffered := True` nativo em `TWinControl` para anular flickering sem esforço manual.
- [x] Processe `KeyDown` para garantir que o contêiner possa ser aberto ou fechado via teclado.
- [x] Desative animações automáticas quando o componente estiver sob a flag `csDesigning`.

15 MÓDULO 11: Coleções Tipadas e Sub-Propriedades Complexas

No desenvolvimento de controles visuais corporativos no **Delphi 12 Athens+**, o gerenciamento de sub-elementos estruturados (como abas de formulários, colunas de data grids, botões de toolbars ou itens de barras de status) exige uma arquitetura estritamente orientada a objetos que se integre de forma transparente tanto com o **Object Inspector** quanto com o motor de streaming **DFM**.

O padrão de arquitetura oficial da Embarcadero para este fim é a especialização de `TOwnedCollection` e `TCollectionItem`.

Neste módulo, dissecaremos o ciclo de vida e a serialização de coleções tipadas, aprenderemos como personalizar a exibição no **Collection Editor** da IDE através de `GetDisplayName`, como propagar notificações de repintura via `Update`, e implementaremos a barra de abas corporativa `TDlpCustomTabBar`.

15.1 1. A Hierarquia `TOwnedCollection` e `TCollectionItem`

A RTL do Delphi (`System.Classes`) fornece a fundação para listas polimórficas serializáveis:

```
classDiagram
    class TPersistent {
        +Assign()
    }
    class TCollectionItem {
        #FCollection: TCollection
        +Index: Integer
        +ID: Integer
        +GetDisplayName(): string
        +Changed(AllItems: Boolean)
    }
    class TCollection {
        #FItemClass: TCollectionItemClass
        +Count: Integer
        +Add(): TCollectionItem
        +Delete(Index)
        +Update(Item: TCollectionItem)
    }
    class TOwnedCollection {
        #FOwner: TPersistent
        +GetOwner(): TPersistent
    }

    class TDlpTabItem {
        +Caption: string
        +ImageIndex: Integer
        +Closable: Boolean
        +BadgeText: string
    }
    class TDlpTabCollection {
        +AddTab(): TDlpTabItem
    }
    class TDlpCustomTabBar {
        +Tabs: TDlpTabCollection
        +TabIndex: Integer
    }

    TPersistent <|-- TCollectionItem
    TPersistent <|-- TCollection
    TCollection <|-- TOwnedCollection
    TCollectionItem <|-- TDlpTabItem
    TOwnedCollection <|-- TDlpTabCollection
    TDlpCustomTabBar *-- TDlpTabCollection : Possui
    TDlpTabCollection o-- TDlpTabItem : Gerencia lista
```

15.1.1 1.1 O Papel Crítico de `TOwnedCollection`

Diferente da classe básica `TCollection`, a classe `TOwnedCollection` retém a referência do objeto proprietário (`AOwner: TPersistent`). Isso viabiliza: 1. **Identificação no DFM:** O leitor de formulário (`TReader`) sabe exatamente qual controle hospeda a coleção ao desserializar os itens do arquivo `.dfm`. 2. **Reatividade Imediata de Pintura:** Toda vez que um item da coleção é alterado, adicionado ou removido, o método `Update` da coleção é disparado automaticamente. Sobrescrevendo `Update`, podemos invalidar o controle pai sem necessidade de eventos manuais: `pascal procedure TDlpTabCollection.Update(Item: TCollectionItem); begin inherited Update(Item); if GetOwner is TControl then TControl(GetOwner).Invalidate; end;`

15.2 2. A Serialização Automática no Arquivo `.dfm`

Quando uma coleção é exposta na seção `published`:

```
published
  property Tabs: TDlpTabCollection read FTabs write SetTabs;
```

O `TWriter` do Delphi serializa automaticamente os itens no `.dfm` sob a sintaxe de lista delimitada:

```
object CustomTabBar: TDlpCustomTabBar
  Left = 16
  Top = 16
  Width = 500
  Height = 36
  TabIndex = 0
  Tabs = <
    item
      Caption = 'Geral'
      Closable = False
    end
    item
      Caption = 'Financeiro'
      BadgeText = 'NOVO'
      BadgeColor = clGreen
    end
    item
      Caption = 'Auditoria'
    end>
end
```

15.3 3. O Método `GetDisplayName` no Collection Editor do Delphi

Ao abrir o Collection Editor no Delphi 12, se não sobrescrevermos `GetDisplayName`, a lista exibirá: - `0 - TDlpTabItem` - `1 - TDlpTabItem`

Sobrescrevendo o método virtual:

```
function TDlpTabItem.GetDisplayName: string;
begin
  if FCaption <> '' then
    Result := FCaption
  else
    Result := inherited GetDisplayName;
end;
```

A IDE passa a exibir instantaneamente: - `0 - Geral` - `1 - Financeiro` - `2 - Auditoria`

15.4 4. Implementação Prática: O Componente `TDlpCustomTabBar`

O código-fonte integral está estruturado em: - `code/units/DlpCustomTabBar.pas`: Controle visual de barra de abas moderna para Delphi 12, construído sobre `TCustomControl`, suporte a `TOwnedCollection`, itens dinâmicos com badges coloridos, botões de fechar `[×]` com efeito hover e evento reativo `OnTabClose(var CanClose: Boolean)`.

15.5 5. Checklist de Engenharia do Módulo 11

- [x] Utilize `TOwnedCollection` em vez de `TCollection` para manter a rastreabilidade do componente hospedeiro.
- [x] Sobrescreva `GetDisplayName` em subclasses de `TCollectionItem` para exibir rótulos informativos no Collection Editor.
- [x] Implemente `Assign` em `TCollectionItem` para garantir clonagem correta durante operações na IDE.
- [x] Propague repinturas automáticas sobrescrevendo `Update` na coleção.
- [x] Vincule e desvincule listas de imagens externas (`TCustomImageList`) usando o protocolo `FreeNotification`.

16 MÓDULO 12: High-DPI, Escalonamento e Suporte a Temas Modernos

No ecossistema corporativo atual, aplicações desenvolvidas em **Delphi 12 Athens+** operam rotineiramente em configurações de múltiplos monitores heterogêneos: por exemplo, um notebook com tela de alta densidade (escala de 150% ou 144 DPI) conectado via Thunderbolt a um monitor de mesa 4K UHD (escala de 200% ou 192 DPI) e a um monitor secundário legado Full HD (escala de 100% ou 96 DPI).

Além disso, os usuários corporativos esperam que a aplicação responda dinamicamente às preferências de aparência do Windows 11, alternando suavemente entre **Dark Mode** e **Light Mode** sem necessidade de reiniciar o executável.

Neste módulo, dissecaremos a infraestrutura de **High-DPI Per-Monitor v2** da VCL, aprenderemos a implementar os métodos `ChangeScale` e `ScaleForPPI`, exploraremos a integração com o motor de estilos **VCL Styles (**`TStyleManager`**)** e construiremos o painel resiliente `TDlpHighDpiThemedPanel`.

16.1 1. A Evolução do Modelo High-DPI no Delphi

O Delphi percorreu três grandes eras no gerenciamento de DPI no Windows:

Geração de DPI	Comportamento Operacional	Qualidade Visual
DPI Unaware	A aplicação informa ao SO que suporta apenas 96 DPI. O Windows estica o bitmap gerado.	Textos borrados e controles pixelados em monitores 4K.
System DPI	A aplicação consulta o DPI do monitor principal na inicialização e escala uma única vez.	Se movida para outro monitor com DPI diferente, fica borrada.
Per-Monitor v2 (Moderno / Delphi 12)	O Windows envia `WM_DPICHANGED` em tempo real para a janela ao cruzar monitores. A VCL recalcula fontes, margens e bitmaps dinamicamente.	**Nitidez cristalina e perfeita em qualquer densidade (100% a 400%).**

```
sequenceDiagram
    autonumber
    participant DWM as Windows 11 DWM
    participant FORM as Vcl.Forms.TForm
    participant CTRL as TDlpHighDpiThemedPanel

    Note over DWM: Janela arrastada para monitor 4K (192 DPI - 200%)
    DWM-->>FORM: WM_DPICHANGED (NewPPI: 192, SuggestedRect)
    FORM-->>FORM: Redimensiona janela para SuggestedRect
    FORM-->>CTRL: Dispara ScaleForPPI(192) / ChangeScale(192, 96, True)

    Note over CTRL: ScaleMetric(BaseHeight):<br/>MulDiv(38, 192, 96) → 76 px
    CTRL-->>CTRL: Invalidate()
    CTRL--->>DWM: Renderização vetorial cristalina a 200%!
```

16.2 2. A Prevenção de Erros de Arredondamento Cumulativos

Um erro clássico cometido em componentes amadores é escalonar a própria variável de estado:

```
// CÓDIGO INSEGURO / AMADOR
procedure TMeuControle.ChangeScale(M, D: Integer; isDpiChange: Boolean);
begin
  inherited ChangeScale(M, D, isDpiChange);
  FHeaderHeight := MulDiv(FHeaderHeight, M, D); // ERRO! Erro de arredondamento acumulativo!
end;
```

Se o usuário mover a janela várias vezes entre monitores de 125%, 150% e 100%, a divisão inteira perderá frações de pixel continuamente, fazendo com que a barra encolha ou cresça gradativamente.

16.2.1 O Padrão Canônico: Base Metric vs. Scaled Metric

Sempre preserve o valor original em 96 DPI (`FBaseHeaderHeight := 38`) e recalcule a dimensão sob demanda usando a propriedade nativa `CurrentPPI`:

```
function TDlpHighDpiThemedPanel.ScaleMetric(AValue: Integer): Integer;
begin
  Result := MulDiv(AValue, CurrentPPI, 96);
end;

procedure TDlpHighDpiThemedPanel.RecalculateScaledMetrics;
begin
  FScaledHeaderHeight := ScaleMetric(FBaseHeaderHeight);
  FScaledBorderRadius := ScaleMetric(FBaseBorderRadius);
  Invalidate;
end;
```

16.3 3. Integração com Temas do Windows 11 e VCL Styles

O Delphi 12 oferece suporte profundo a **VCL Styles** através de `Vcl.Themes.TStyleManager` e `StyleServices`.

Nosso componente avalia dinamicamente se o tema ativo é escuro ou claro, adaptando bordas, fundos e tipografia:

```
if StyleServices.Enabled then
  IsDarkTheme := (ColorToRGB(StyleServices.GetSystemColor(clWindow)) < $00808080)
else
  IsDarkTheme := True;
```

16.4 4. Implementação Prática: O Componente `TDlpHighDpiThemedPanel`

O código-fonte integral está estruturado em: - `code/units/DlpHighDpiThemedPanel.pas`: Painel com suporte a Per-Monitor v2, sobrescrita de `ChangeScale` e `ScaleForPPI`, cálculo dinâmico via `CurrentPPI`, integração com `StyleServices`, cantos arredondados escalonados e badge de diagnóstico de escala em tempo de execução.

16.5 5. Checklist de Engenharia do Módulo 12

- [ ] Configure seu projeto Delphi com suporte a Per-Monitor v2 no Project Options (`Application → Manifest → DPI Awareness`).
- [ ] Nunca recalcule métricas sobre valores já escalonados; mantenha constantes de design baseadas em 96 DPI.
- [ ] Sobrescreva `ChangeScale` e `ScaleForPPI` em controles que armazenam medidas internas em pixels.
- [ ] Utilize `CurrentPPI` para obter a densidade de pixels exata do monitor onde o controle reside.
- [ ] Harmonize paletas de cores através de `StyleServices` para garantir aderência ao Dark Mode e Light Mode do Windows 11.

17 MÓDULO 13: Arquitetura de Dados no Pascal e a Ponte TDataLink

No universo corporativo do **Delphi 12 Athens+**, o poder do desenvolvimento orientado a dados (**Data-Aware Architecture**) atinge seu nível máximo de sofisticação. Desde a comunicação de altíssima velocidade com bancos SQL corporativos através do **FireDAC** até a manipulação de estruturas em memória com `TFDMemTable`, a interface gráfica depende de um mecanismo robusto e reativo para sincronizar cursores de dados com a tela.

Tentar ler dados de campos diretamente em timers ou acoplar botões a referências diretas de `TFDQuery` viola o princípio de responsabilidade única e impede a criação de componentes reutilizáveis.

A solução canônica da Embarcadero baseia-se na **Ponte** `TDataLink` **/** `TFieldDataLink`, um mediador desacoplado que opera sob o padrão de projeto **Observer**.

Neste módulo, aprenderemos como funciona o fluxo interno entre `TDataSet`, `TDataSource` e a UI, a máquina de estados de edição e construiremos o componente framework base `TDlpCustomDBConnector`.

17.1 1. A Tríade Reativa no Delphi Moderno

O modelo de dados do Delphi estabelece um isolamento rigoroso entre a camada de persistência e a camada visual:

```
graph LR
    subgraph DATA ["Acesso a Dados (FireDAC / Enterprise Data)"]
        FD["TFDQuery / TFDMemTable (TDataSet)"]
    end

    subgraph HUB ["Mediador Central"]
        SRC["TDataSource"]
    end

    subgraph UI ["Componente Visual DB-Aware (VCL)"]
        DL["TDlpFieldDataLinkBridge (TFieldDataLink)"]
        CTRL["TDlpCustomDBConnector / Controles DB"]
    end

    FD <-->|Cursores e Buffers| SRC
    SRC <-->|Disparo de Notificações| DL
    DL <-->|Comunicação Reativa| CTRL
```

17.1.1 1.1 As Responsabilidades de Cada Ator:

- `TDataSet` **(ex:** `TFDQuery`**):** Mantém o cursor físico ou em memória, executa instruções SQL, gerencia transações e aloca buffers de registros.
- `TDataSource`**:** Atua como um *Hub/Mediator*. Centraliza eventos de navegação (`OnDataChange`, `OnUpdateData`) e permite que dezenas de controles na tela compartilhem o mesmo contexto sem se

conhecerem mutuamente.

- `TFieldDataLink`: O motor interno do seu componente customizado. Ele se registra como um observador no `TDataSource` e traduz as mudanças de linha e de modo de edição em chamadas de métodos virtuais protegidos.

17.2 2. A Máquina de Estados e a Interceptação dos Cinco Métodos Sagrados

O ciclo de vida do `TDataSet` no Delphi transita entre estados bem definidos:

```
stateDiagram-v2
    [*] ⟶ dsInactive
    dsInactive ⟶ dsBrowse : Open / Active := True
    dsBrowse ⟶ dsEdit : Edit
    dsBrowse ⟶ dsInsert : Insert / Append
    dsEdit ⟶ dsBrowse : Post / Cancel
    dsInsert ⟶ dsBrowse : Post / Cancel
    dsBrowse ⟶ dsInactive : Close / Active := False
```

Ao herdar de `TFieldDataLink`, sobrescrevemos cinco métodos canônicos:

Método Sobrescrito	Momento do Disparo	Ação no Componente Visual
`ActiveChanged`	Disparado ao abrir (`Open`) ou fechar (`Close`) a tabela.	Habilitar/desabilitar controles e atualizar status visual.
`DataSetChanged`	Disparado quando o cursor se move para outro registro ou ocorre `Refresh` .	Recarregar os dados na tela a partir do registro ativo.
`EditingChanged`	Disparado na transição de/para `dsEdit` ou `dsInsert` .	Habilitar foco de edição, mudar bordas ou acionar indicadores de status.
`RecordChanged(Field)`	Disparado quando o valor do campo vinculado é alterado na memória.	Atualizar o texto exibido sem necessariamente mover o cursor.
`UpdateData`	Disparado **imediatamente antes** do `TDataSet.Post` .	Despejar o texto modificado da interface de volta para o campo (`Field.AsString := Edit.Text`).

17.3 3. Implementação Prática: O Framework `TDlpCustomDBConnector`

O código-fonte integral está estruturado em: - `code/units/DlpCustomDBConnector.pas`: Unidade contendo o observador especializado `TDlpFieldDataLinkBridge` e a classe base `TDlpCustomDBConnector`, fornecendo

os métodos `CanModify`, `Edit`, `Modified`, `Reset` e eventos fortemente tipados para estados de dados.

17.4 4. Checklist de Engenharia do Módulo 13

- [x] Nunca vincule controles visuais diretamente a instâncias de `TDataSet`; utilize sempre `TDataSource` como mediador.
- [x] Especialize `TFieldDataLink` para capturar as transições de cursor e estado do dataset.
- [x] No método `UpdateData`, sincronize o valor da tela de volta para o buffer do `TField` antes do `Post`.
- [x] Proteja ponteiros de `TDataSource` utilizando `FreeNotification` para evitar falhas de desalocação no Form Designer.
- [x] Utilize `DataLink.Edit` antes de aceitar digitação do usuário para garantir que o dataset entre legalmente em modo `dsEdit`.

18 MÓDULO 14: Controles DB-Aware Básicos de Exibição e Edição

A excelência de uma interface corporativa no **Delphi 12 Athens+** manifesta-se nos detalhes de usabilidade e na precisão com que os dados são apresentados e editados.

Em sistemas ERP, bancários e de gestão industrial, dois requisitos surgem constantemente: 1. **Pílulas Visuais de Status com Formatação Condicional (`TDlpDBBadgeLabel`):** Elementos gráficos windowless de alto rendimento que transformam códigos frios de banco (ex: `'A'`, `'P'`, `'C'`) em badges elegantes com cores intuitivas (Verde, Laranja, Cinza) e cantos arredondados. 2. **Caixas de Edição Monetária e Numérica (`TDlpDBNumberEdit`):** Controles com alinhamento à direita, máscara adaptativa ao foco, validação Unicode estrita via `CharInSet` e disparo automático da transição para `dsEdit` ao teclar.

Neste módulo, implementaremos esses dois componentes indispensáveis para a VCL.

18.1 1. O Controle `TDlpDBBadgeLabel`: Status Condicional Windowless

Construído sobre `TGraphicControl`, o `TDlpDBBadgeLabel` monitora o campo vinculado e repinta a si mesmo automaticamente:

```
sequenceDiagram
    autonumber
    participant FD as TFDQuery / TDataSet
    participant DL as TFieldDataLink
    participant BL as TDlpDBBadgeLabel

    FD-->>DL: OnDataChange (Registro Ativo Alterado)
    DL-->>BL: DataChange(Sender)
    Note over BL: Lê Field.DisplayText<br/>ResolveBadgeColor(Texto)
    BL-->>BL: Invalidate()
    Note over BL: Desenha pílula colorida com Vcl.Graphics.DrawText centralizado
```

18.2 2. O Controle `TDlpDBNumberEdit`: Entrada Monetária Fluida

O `TDlpDBNumberEdit` soluciona as três maiores frustrações de usuários corporativos: - **Digitação Limpa:** Quando o usuário entra no campo (`DoEnter`), o símbolo de moeda é ocultado temporariamente, exibindo o número puro (`1450.75`). - **Visualização Elegante:** Quando o usuário sai do campo (`DoExit`), o número é formatado no padrão contábil (`R$ 1.450,75`). - **Edição Imediata:** Não requer cliques em botões de "Modificar". O primeiro caractere digitado dispara `DataLink.Edit` e `DataLink.Modified`.

18.2.1 2.1 Validação de Caracteres com `CharInSet`:

No Delphi Unicode, utilizamos a função otimizada `CharInSet`:

```
procedure TDlpDBNumberEdit.KeyPress(var Key: Char);
begin
  if not CharInSet(Key, ['0'..'9', ',', '.', #8, #13, #27]) then
  begin
    Key := #0; // Bloqueia caracteres não numéricos
    Exit;
  end;

  if CharInSet(Key, ['0'..'9', ',', '.']) and not ReadOnly then
  begin
    if not FDataLink.Editing then
      FDataLink.Edit;

    FDataLink.Modified;
  end;

  inherited KeyPress(Key);
end;
```

18.3 3. Implementação Prática: Componentes Completos

A unidade completa está localizada em: - `code/units/DlpDBBasicControls.pas`: Contém o `TDlpDBBadgeLabel` e o `TDlpDBNumberEdit`, ambos com suporte a `TFieldDataLink`, `FreeNotification` e registro oficial na categoria `'Engenharia Delphi 12'`.

18.4 4. Checklist de Engenharia do Módulo 14

- [x] Utilize `TGraphicControl` com `TFieldDataLink` para indicadores de status leves.
- [x] Configure `Alignment := taRightJustify` e `DoubleBuffered := True` em editores numéricos VCL.
- [x] Use `CharInSet` em vez de operadores `in` legados para compatibilidade e segurança Unicode no Delphi 12.
- [x] Chame `DataLink.Edit` no evento `KeyPress` antes de processar caracteres para colocar o dataset no modo `dsEdit`.
- [x] Envolva chamadas a `DataLink.UpdateRecord` em blocos `try..except SetFocus; raise; end` para reter o foco em caso de erro de validação.

19 MÓDULO 15: Controles DB-Aware Avançados e Seletores (TDlpDBLookupTagBox)

No ecossistema corporativo do Delphi, o desenvolvimento de interfaces modernas para seleção de entidades relacionais ultrapassou o formato rígido das antigas caixas combinadas (`TDBLookupComboBox`). Aplicações com alta densidade de informação demandam controles no formato de **etiquetas dinâmicas (*chips* ou *tags*)**, capazes de representar associações multivaloradas (ex.: categorias de produtos, perfis de acesso, múltiplos e-mails ou centros de custo) diretamente acoplados à camada de dados.

Neste módulo, construímos do zero o componente `TDlpDBLookupTagBox`, integrando os mecanismos de `TFieldDataLink`, double-buffering da VCL e resolução de dados relacionais com FireDAC.

19.1 1. Arquitetura de Seleção Múltipla Relacional

O controle resolve o clássico dilema de mapeamento de relacionamentos N:M em interfaces RAD através de um desacoplamento em três camadas:

```
graph TD
    subgraph Camada de Apresentacao
        A[TDlpDBLookupTagBox] ⟶|Renderiza| B[Array de TDlpTagItem]
        A ⟶|Hit-Testing no MouseDown| C[CloseRect Botao x]
    end

    subgraph Camada DB-Aware
        A ⟶|FDataLink: TFieldDataLink| D[TDataSource Principal]
        A ⟶|FLookupSource: TDataSource| E[DataSet de Opcoes FireDAC]
    end

    subgraph Camada de Persistencia
        D ⟶|String Delimitada '1,4,9'| F[(Tabela de Dominio)]
        E ⟶|Resolve ID → Descricao| G[(Tabela Lookup / Categoria)]
    end
```

19.1.1 1.1 Dupla Conexão com Dados:

1. **Conexão de Armazenamento (`DataSource` / `DataField`):** Vincula-se ao registro que está sendo editado. O campo armazena a representação serializada das chaves selecionadas (ex.: `"10,25,32"`).
2. **Conexão de Resolução (`LookupSource`, `LookupKeyField`, `LookupListField`):** Aponta para o `TFDQuery` ou `TFDTable` que contém a tabela de domínio (ex: chave primária e nome fantasia), permitindo traduzir o ID `10` em `"Financeiro"`, `25` em `"Recursos Humanos"`, etc.

19.2 2. Geometria de Renderização e Hit-Testing

Diferente de controles baseados em múltiplos botões filhos embutidos, o `TDlpDBLookupTagBox` realiza a renderização direta no `Canvas` da VCL com **DoubleBuffering**, garantindo performance extrema sem consumo de handles adicionais do Windows (`HWND`):

```
type
  TDlpTagItem = record
    Key: string;
    Caption: string;
    Rect: TRect;
    CloseRect: TRect;
  end;

  TDlpTagItemArray = TArray<TDlpTagItem>;
```

19.2.1 2.1 Cálculo Responsivo de Quebra de Linha

No método `RecalculateTagLayout`, o componente itera pelas tags registradas, calcula a extensão da fonte (`Canvas.TextWidth`) e distribui as caixas arredondadas, quebrando a linha sempre que a margem direita for atingida:

```
procedure TDlpDBLookupTagBox.RecalculateTagLayout;
var
  i, CurrentX, CurrentY, TagH, TextW, TagW: Integer;
begin
  Canvas.Font.Assign(Self.Font);
  TagH := 24;
  CurrentX := 4;
  CurrentY := 4;

  for i := 0 to High(FTags) do
  begin
    TextW := Canvas.TextWidth(FTags[i].Caption);
    TagW := TextW + 28;

    if (CurrentX + TagW > Width - 8) and (CurrentX > 4) then
    begin
      CurrentX := 4;
      Inc(CurrentY, TagH + 4);
    end;

    FTags[i].Rect := Rect(CurrentX, CurrentY, CurrentX + TagW, CurrentY + TagH);
    FTags[i].CloseRect := Rect(CurrentX + TagW - 18, CurrentY + 3, CurrentX + TagW - 2,
        CurrentY + TagH - 3);

    Inc(CurrentX, TagW + 6);
  end;
end;
```

19.3 3. O Ciclo de Edição e Sincronização via `TFieldDataLink`

O componente nunca altera o dataset diretamente sem passar pelas validações do ciclo de vida:

1. **Leitura Reativa (`DataChange`):** Quando o usuário navega pelo dataset mestre (`Prior`, `Next`), o evento `DataChange` invoca `ParseFieldValue`, dividindo a string bruta via `TStringList.StrictDelimiter` e populando as tags em tela.
2. **Entrada em Edição Segura (`Edit`):** Ao clicar no botão de fechar (`x`) de uma tag ou teclar `Backspace`, o método verifica se o `DataLink` já está em edição. Se não estiver, invoca `FDataLink.Edit`, disparando o estado `dsEdit` no `TDataSet`.
3. **Persistência (`UpdateData`):** Quando o usuário confirma a operação (`Post`) ou o componente perde o foco (`DoExit`), `FDataLink.UpdateRecord` chama `UpdateData`, serializando os IDs no formato delimitado configurado (`DelimitedChar`).

19.4 4. Código-Fonte Integral do Componente

A unidade Pascal completa, incluindo suporte a atalhos de teclado, foco visual temático e registro na paleta **"Delphi Component Engineering"**, está localizada em:

👉 **Arquivo:** `code/units/DlpDBLookupTagBox.pas`

20 MÓDULO 16: Monitoramento e Gerenciador de Transações de Dados (TDlpDataConnectionMonitor)

No desenvolvimento de sistemas corporativos de missão crítica em Delphi utilizando **FireDAC**, a governança das transações de banco de dados é um dos pilares mais vitais para a saúde da infraestrutura. O esquecimento de comandos `Commit` ou `Rollback` em blocos de exceção mal estruturados ou janelas modais deixadas abertas por operadores gera as chamadas **transações zumbis (ou transações órfãs)**. Essas transações mantêm bloqueios de leitura e escrita (*locks* de tabela e de registro), provocando enfileiramento de conexões, *deadlocks* e degradação severa dos servidores (Oracle, Microsoft SQL Server, PostgreSQL, Firebird).

Neste módulo, concebemos e construímos o componente não-visual `TDlpDataConnectionMonitor`, atuando como um cão de guarda (*watchdog*) e centralizador de telemetria para instâncias de `TFDConnection`.

20.1 1. Arquitetura de Auditoria de Conexões FireDAC

O `TDlpDataConnectionMonitor` atua desacoplado da lógica de negócio, auditando o estado interno da conexão através da API pública do FireDAC:

```
graph TD
    subgraph Aplicacao Delphi
        A[TFDQuery / TFDCommand] ⟶|ExecSQL / Open| B[TFDConnection]
        C[TDlpDataConnectionMonitor] ⟶|Inspeciona InTransaction| B
    end

    subgraph Mecanismo de Telemetria
        C ⟶|RegisterQueryExecution| D{DeltaMS ⩾ Limiar?}
        D ⟶|Sim| E[Dispara OnSlowQueryDetected]
        D ⟶|Nao| F[Registra Log de Execucao]
    end

    subgraph Seguranca Transacional
        C ⟶|CheckZombieTransactions| G{Idade > MaxSec?}
        G ⟶|Sim| H[Dispara OnZombieTransactionDetected]
        H ⟶|AutoRollback=True| I[Executa Rollback Imediato]
    end
```

20.1.1 1.1 Ciclo de Monitoramento Transacional

O componente armazena o instante exato em que a transação foi deflagrada (`FTxStartTime`) e avalia dinamicamente se o tempo de vida extrapolou o limite aceitável (`MaxTransactionDurationSec`):

```
function TDlpDataConnectionMonitor.GetTransactionAgeSeconds: Int64;
begin
  Result := 0;
  if IsTransactionActive and (FTxStartTime > 0) then
    Result := SecondsBetween(Now, FTxStartTime);
end;
```

20.2 2. Ação Automática sobre Transações Zumbis

O método `CheckZombieTransactions` pode ser acionado periodicamente (por exemplo, através de um `TTimer` de infraestrutura ou na rotina de fechamento de formulários). Caso detecte uma transação estagnada, ele notifica a aplicação via evento e, se a flag `AutoRollbackZombies` estiver ativa, força o `Rollback` para liberar os locks no banco:

```
procedure TDlpDataConnectionMonitor.CheckZombieTransactions;
var
  AgeSec: Int64;
  ShouldRollback: Boolean;
begin
  if not FActive or not IsTransactionActive then
    Exit;

  AgeSec := GetTransactionAgeSeconds;
  if (AgeSec ≥ FMaxTransactionDurationSec) and (AgeSec > 0) then
  begin
    ShouldRollback := FAutoRollbackZombies;
    WriteLogEntry(Format('FIREDAC_ZOMBIE_TX_ALERT: Transação ativa por %d segundos.',
        [AgeSec]));

    if Assigned(FOnZombieTransactionDetected) then
      FOnZombieTransactionDetected(Self, FConnection, AgeSec, ShouldRollback);

    if ShouldRollback and FConnection.InTransaction then
    begin
      try
        FConnection.Rollback;
        FTxStartTime := 0;
        WriteLogEntry('FIREDAC_ZOMBIE_TX_ACTION: Rollback efetuado com sucesso.');
      except
        on E: Exception do
          WriteLogEntry('FIREDAC_ZOMBIE_TX_ERROR: Falha ao executar rollback: ' + E.Message);
      end;
    end;
  end;
end;
```

20.3 3. Telemetria de Consultas Lentas (Slow Queries)

Integrando a classe nativa de alta resolução `TStopwatch` (`System.Diagnostics`), a camada de repositório pode cronometrar as queries e encaminhar a métrica ao monitor:

```
var
  Stopwatch: TStopwatch;
begin
  Stopwatch := TStopwatch.StartNew;
  try
    FDQuery.Open;
  finally
    Stopwatch.Stop;
    DlpDataConnectionMonitor1.RegisterQueryExecution(FDQuery.SQL.Text,
        Stopwatch.ElapsedMilliseconds);
  end;
end;
```

Se o tempo transcorrido for superior ao valor de `SlowQueryThresholdMS` (ex.: 1000 ms), o componente dispara o evento `OnSlowQueryDetected` e registra o incidente no arquivo de log com timestamp milissegundo.

20.4 4. Código-Fonte Integral do Componente

A unidade com a implementação completa, incluindo gerenciamento seguro de notificações de remoção (`FreeNotification`) e registro na paleta **"Delphi Component Engineering"**, está disponível em:

👉 **Arquivo:** `code/units/DlpDataConnectionMonitor.pas`

21 MÓDULO 17: Engenharia de Arrastar e Soltar (Drag & Drop) no Designer

A filosofia RAD (*Rapid Application Development*) do Delphi sempre foi impulsionada pela interação visual intuitiva. O fluxo de arrastar conexões, tabelas e campos do **Data Explorer** diretamente para o designer de formulários e obter interfaces prontas em segundos é uma das marcas registradas da produtividade da ferramenta. No entanto, a criação manual de cada componente de tela (`TLabel`, `TDBEdit`, `TDBDateEdit`) em formulários extensos com dezenas de campos é repetitiva, propensa a erros de digitação e incompatível com os prazos de entregas corporativas modernas.

Neste módulo, detalhamos como a arquitetura de tempo de design da IDE interage com a introspecção de dados e implementamos o motor de metadados `TDlpFieldSchemaInspector`.

21.1 1. O Pipeline de Drag & Drop no Designer do Delphi

Quando o desenvolvedor arrasta uma tabela ou consulta FireDAC para a superfície do formulário, a IDE consulta a infraestrutura do designer (`IDesigner` / `IDesignerSelections`):

```
graph TD
    subgraph Designer da IDE
        A[Data Explorer / Estrutura de Dados] ⟶|Arrasta Campos| B[Superficie do Formulario]
    end

    subgraph Inspecao de Metadados
        B ⟶ C[TDlpFieldSchemaInspector]
        C ⟶|Consulta TFieldDefs / TFields| D[TFDQuery / TFDTable]
        D ⟶|Metadados Extraidos| E[TDlpFieldSchemaArray]
    end

    subgraph Resolucao de Controles
        E ⟶|ResolveControlKind| F{Matriz de Tipos}
        F ⟶|String| G[TDBEdit]
        F ⟶|Numerico / Moeda| H[TDlpDBNumberEdit]
        F ⟶|Data / Hora| I[TDateTimePicker]
        F ⟶|Booleano| J[TDBCheckBox]
        F ⟶|Texto Longo| K[TDBMemo]
        F ⟶|Binario / Foto| L[TDBImage]
        F ⟶|Lookup N:M| M[TDlpDBLookupTagBox]
    end
```

21.2 2. Introspecção Conectada vs. Desconectada

Um componente de automação de formulários deve ser capaz de operar mesmo quando a conexão com o banco não estiver ativa no computador do desenvolvedor em tempo de design. O método `InspectSchema` aborda essa necessidade com uma estratégia de contingência dupla:

1. **Modo Conectado (`FDataSet.Fields.Count > 0`):** Se os campos persistentes já foram criados pelo usuário ou se o dataset estiver aberto (`Active = True`), o motor extrai metadados refinados (`DisplayLabel`, `Required`, regras de validação).
2. **Modo Estrutural (`FDataSet.FieldDefs.Count > 0`):** Se o dataset estiver fechado, o motor dispara `FieldDefs.Update` para recuperar a definição das colunas diretamente do driver do FireDAC sem abrir os dados.

```
if not FDataSet.Active and (FDataSet.FieldDefs.Count = 0) then
begin
  try
    FDataSet.FieldDefs.Update;
  except
    Exit;
  end;
end;
```

21.3 3. Matriz Declarativa de Mapeamento

A classe estática disponibiliza o método `ResolveControlKind`, convertendo cada constante `TFieldType` do Delphi na classificação de controle correspondente:

```
class function TDlpFieldSchemaInspector.ResolveControlKind(
  AFieldType: TFieldType; ASize: Integer): TDlpControlKind;
begin
  case AFieldType of
    ftString, ftFixedChar, ftWideString, ftFixedWideChar:
    begin
      if ASize > 255 then
        Result := ckMemo
      else
        Result := ckTextEdit;
    end;

    ftSmallint, ftInteger, ftWord, ftFloat, ftCurrency, ftBCD, ftLargeint, ftFMTBcd:
      Result := ckNumberEdit;

    ftDate, ftTime, ftDateTime, ftTimeStamp:
      Result := ckDatePicker;

    ftBoolean:
      Result := ckCheckBox;

    ftBlob, ftGraphic, ftParadoxOle, ftDBaseOle:
      Result := ckImageBlob;

    ftMemo, ftWideMemo:
      Result := ckMemo;

    else
      Result := ckTextEdit;
  end;
end;
```

21.4 4. Código-Fonte Integral do Motor de Introspecção

A implementação integral do motor de análise de esquemas e mapeamento de tipos para o Delphi 12 Athens+ está disponível em:

👉 **Arquivo:** `code/units/DlpFieldSchemaInspector.pas`

22 MÓDULO 18: Construção do Componente Gerador de Cadastros (TDlpAutoFormBuilder)

Na construção de sistemas corporativos em Delphi, telas de cadastro de cadastros operacionais (CRUD) representam a maior parte do volume de código de interface de usuário. Configurar manualmente componentes visuais (`TLabel`, `TDBEdit`, `TDBMemo`), calcular coordenadas `Top` e `Left`, e vincular propriedades a datasets do FireDAC exige tempo excessivo e introduz inconsistências visuais em equipes com múltiplos desenvolvedores.

Neste módulo, concebemos e construímos o componente `TDlpAutoFormBuilder`, um contêiner automatizado para a **VCL (Delphi 12 Athens+)** que interpreta o esquema relacional em tempo de design ou execução, gera a interface em grid responsivo e injeta os componentes diretamente no arquivo `.dfm`.

22.1 1. Arquitetura do Componente e Herança de `TScrollBox`

O `TDlpAutoFormBuilder` herda de `TScrollBox`, garantindo suporte nativo a rolagem vertical para esquemas com dezenas de colunas:

```
graph TD
    subgraph Entrada de Dados
        A[TFDQuery / TFDTable] ⟶ B[TDataSource]
        B ⟶ C[TDlpAutoFormBuilder]
    end

    subgraph Processamento de Layout
        C ⟶ D[TDlpFieldSchemaInspector]
        D ⟶|Gera Array de Metadados| E[Cálculo de Grid Responsivo]
        E ⟶|Instancia TLabel| F[lblRazaoSocial, lblValor, lblAtivo]
        E ⟶|Instancia Controles VCL| G[edtRazaoSocial, edtValor, chkAtivo]
    end

    subgraph Persistência DFM
        F ⟶ H[Owner := TargetForm]
        G ⟶ H
        H ⟶ I[(Arquivo de Formulário .dfm)]
    end
```

22.2 2. Injeção de Componentes no Form e Persistência no DFM

Para que os componentes gerados dinamicamente se tornem membros legítimos da classe do formulário (permitindo ao desenvolvedor referenciá-los diretamente em código, como `edtNome.SetFocus`), o contêiner aplica uma regra estrita de posse (*Ownership*):

```
TargetForm := GetOwnerForm;
if TargetForm <> nil then
  EffectiveOwner := TargetForm
else
  EffectiveOwner := Self;

// O Owner deve ser o Form para ser serializado no .dfm
NewEdit := TDBEdit.Create(EffectiveOwner);
NewEdit.Parent := Self;
NewEdit.Name := EffectiveOwner.FindUniqueName('edt' + SanitizedName);
```

22.3 3. Algoritmo de Posicionamento e Espaçamento

O controle distribui os elementos em colunas proporcionais (`ColumnsCount`), acomodando rótulo e caixa de entrada com cálculo dinâmico:

```
UsableWidth := ClientWidth - (FHorizontalSpacing * (FColumnsCount + 1));
if UsableWidth < 200 then
  UsableWidth := 200;
ColWidth := UsableWidth div FColumnsCount;
```

A cada iteração: 1. Posiciona `TLabel` na coordenada `(CurLeft, RowTop)`. 2. Posiciona o controle na coordenada `(CurLeft, RowTop + LabelHeight + 2)`. 3. Incrementa o índice de coluna. Ao atingir o limite, pula para a linha seguinte somando a altura do controle ao espaçamento vertical (`VerticalSpacing`).

22.4 4. Código-Fonte Integral do Componente

A implementação completa do componente `TDlpAutoFormBuilder` está disponível em:

👉 **Arquivo:** `code/units/DlpAutoFormBuilder.pas`

23 MÓDULO 19: Caso de Estudo Real 1 — Gerador do Cadastro Completo de Clientes

A consolidação prática da arquitetura de componentes desenvolvida nos módulos anteriores se materializa de forma exemplar na construção do **Cadastro Corporativo de Clientes**. Neste estudo de caso, unificamos o contêiner gerador de telas `TDlpAutoFormBuilder`, a barra de controle transacional `TDlpCRUDOperationsBar` e o indicador visual de estado `TDlpDBBadgeLabel` em uma interface orientada a regras fiscais brasileiras.

23.1 1. Topologia da Interface de Usuário

O formulário de clientes é arquitetado em três camadas visuais coordenadas:

```
+-------------------------------------------------------------------------------+
| TDlpCRUDOperationsBar: [ Novo ] [ Alterar ] [ Salvar ] [ Cancelar ] [ Excluir ] |
+-------------------------------------------------------------------------------+
| Painel Superior: Status do Cliente → TDlpDBBadgeLabel [ ATIVO / INATIVO ]      |
+-------------------------------------------------------------------------------+
| TDlpAutoFormBuilder:                                                          |
|  lblRazaoSocial           lblCNPJ                                             |
|  [ edtRazaoSocial       ] [ edtCNPJ               ]                         |
|  lblEmailFinanceiro       lblLimiteCredito                                    |
|  [ edtEmailFinanceiro   ] [ edtLimiteCredito      ]                         |
|  lblObservacoes                                                               |
|  [ memObservacoes                                 ]                         |
+-------------------------------------------------------------------------------+
```

23.2 2. A Máquina de Estados Operacionais

A classe `TDlpCRUDOperationsBar` implementa a máquina de estados finitos que elimina inconsistências operacionais do usuário:

```
stateDiagram-v2
    [*] ⟶ osBrowsing: Abertura do DataSet FireDAC
    osBrowsing ⟶ osInserting: Clique no botao Novo
    osBrowsing ⟶ osEditing: Clique no botao Alterar
    osInserting ⟶ osBrowsing: Salvar (Post) ou Cancelar (Cancel)
    osEditing ⟶ osBrowsing: Salvar (Post) ou Cancelar (Cancel)
    osBrowsing ⟶ [*]: Fechamento da Janela
```

23.2.1 2.1 Validação de CPF e CNPJ por Módulo 11

Antes de persistir o registro no banco via `Post`, o evento `OnValidateRecord` intercepta a operação e submete o documento à validação matemática estática:

```
procedure TFormClientes.CRUDBarValidateRecord(Sender: TObject; var CanSave: Boolean);
var
  Doc: string;
begin
  Doc := FDTableClientes.FieldByName('CPF_CNPJ').AsString;
  if Length(TDlpCRUDOperationsBar.OnlyDigits(Doc)) = 11 then
  begin
    if not TDlpCRUDOperationsBar.ValidateCPF(Doc) then
    begin
      ShowMessage('O CPF informado é inválido. Verifique os dígitos verificadores.');
      CanSave := False;
    end;
  end
  else if Length(TDlpCRUDOperationsBar.OnlyDigits(Doc)) = 14 then
  begin
    if not TDlpCRUDOperationsBar.ValidateCNPJ(Doc) then
    begin
      ShowMessage('O CNPJ informado é inválido. Verifique os dígitos verificadores.');
      CanSave := False;
    end;
  end
  else
  begin
    ShowMessage('Informe um CPF (11 dígitos) ou CNPJ (14 dígitos) válido.');
    CanSave := False;
  end;
end;
```

23.3 3. Código-Fonte Integral dos Componentes do Caso de Estudo

A implementação completa do componente `TDlpCRUDOperationsBar` com os algoritmos de validação de CPF e CNPJ está disponível em:

👉 **Arquivo:** `code/units/DlpCRUDOperationsBar.pas`

24 MÓDULO 20: Caso de Estudo Real 2 — Gerador do Cadastro de Produtos e Itens

O catálogo de produtos constitui o núcleo transacional dos sistemas comerciais contemporâneos (ERP, PDV, Comércio Eletrônico). O desenvolvimento eficiente desse módulo requer mais do que meros campos cadastrais: impõe a integração de cálculos financeiros automatizados (margem de lucro comercial vs. índice de markup) e a visualização intuitiva da saúde do inventário de mercadorias.

Neste módulo, implementamos a arquitetura do **Cadastro de Produtos e Catálogo de Itens** combinando o gerador automatizado `TDlpAutoFormBuilder` e o componente de telemetria visual `TDlpProductStockIndicator`.

24.1 1. Topologia da Tela e Estrutura de Apresentação

A disposição visual reúne os componentes de persistência e os controles analíticos:

```
+-------------------------------------------------------------------------------+
| TDlpCRUDOperationsBar: [ Novo ] [ Alterar ] [ Salvar ] [ Cancelar ] [ Excluir ] |
+-------------------------------------------------------------------------------+
| TDlpAutoFormBuilder (Campos Principais do Produto):                           |
|  lblCodigoBarras            lblDescricao                                      |
|  [ edtCodigoBarras        ] [ edtDescricao                                  ] |
|  lblPrecoCusto              lblPrecoVenda                                     |
|  [ edtPrecoCusto          ] [ edtPrecoVenda                                 ] |
+-------------------------------------------------------------------------------+
| Painel de Inteligência de Negócio:                                            |
|  Margem de Lucro: [ 42.10% ]  |  Markup: [ 72.72% ]                           |
|  TDlpProductStockIndicator:                                                   |
|  Estoque: REPOSIÇÃO (18.0)                                                    |
|  [==========----------------------------------------------------------------] |
+-------------------------------------------------------------------------------+
```

24.2 2. Aritmética Comercial: Margem de Lucro vs. Markup

A distinção precisa entre margem de lucro e markup é crucial para a integridade de qualquer sistema de gestão:

- **Margem de Lucro:** Percentual de lucro obtido sobre o **preço de venda final**.
- **Markup:** Coeficiente multiplicador aplicado sobre o **preço de custo**.

$$ \text{Margem (\%)} = \frac{\text{Preço Venda} - \text{Preço Custo}}{\text{Preço Venda}} \times 100 $$

$$ \text{Markup (\%)} = \frac{\text{Preço Venda} - \text{Preço Custo}}{\text{Preço Custo}} \times 100 $$

O componente `TDlpProductStockIndicator` fornece essas rotinas nativas:

```
class function TDlpProductStockIndicator.CalculateMargin(CostPrice, SellPrice: Double): Double;
begin
  if SellPrice ⩽ 0 then
    Exit(0.0);
  Result := ((SellPrice - CostPrice) / SellPrice) * 100.0;
end;

class function TDlpProductStockIndicator.CalculateMarkup(CostPrice, SellPrice: Double): Double;
begin
  if CostPrice ⩽ 0 then
    Exit(0.0);
  Result := ((SellPrice - CostPrice) / CostPrice) * 100.0;
end;
```

24.3 3. Monitoramento de Estoque com Indicadores Colorimétricos

O `TDlpProductStockIndicator` monitora dinamicamente a proporção entre `CurrentStockField`, `MinStockField` e `MaxStockField`, atualizando o gradiente e a legenda na barra de progresso:

```
graph LR
    A[Estoque Atual] ⟶ B{Compara com Parametros}
    B ⟶|Atual ⩽ Minimo| C[CRITICO: Vermelho Vivo]
    B ⟶|Atual ⩽ Minimo * 1.25| D[REPOSICAO: Laranja Ambar]
    B ⟶|Faixa Ideal| E[NORMAL: Verde Vibrante]
    B ⟶|Atual > Maximo| F[EXCESSO: Azul Cobalto]
```

24.4 4. Código-Fonte Integral do Componente

A implementação completa do medidor visual `TDlpProductStockIndicator` para o Delphi 12 Athens+ está disponível em:

👉 **Arquivo:** `code/units/DlpProductStockIndicator.pas`

25 MÓDULO 21: Componentes de Execução Assíncrona e Worker Threads (TDlpAsyncTaskRunner)

Manter a interface de usuário responsiva e fluida é um dos requisitos mais intransigentes de qualquer aplicação corporativa de alto padrão. Quando o software executa processamento massivo de registros, cálculos contábeis complexos ou requisições I/O sem transferir o trabalho para uma thread secundária, o sistema operacional Windows sinaliza a janela como *"Não Respondendo"*, resultando em frustração e desconfiança por parte do usuário.

Neste módulo, exploramos a concorrência moderna no **Delphi 12 Athens+** através da **PPL (Parallel Programming Library)** e construímos o componente não-visual `TDlpAsyncTaskRunner`.

25.1 1. Concorrência Segura no Delphi: A Afinidade da UI Thread

No ecossistema Delphi (tanto na VCL quanto no FireMonkey), todos os componentes visuais são projetados com afinidade unithread (*Main Thread Affinity*). Modificar propriedades de um controle visual diretamente a partir de um worker thread provoca violações de acesso imprevisíveis:

```
graph TD
    subgraph Parallel Programming Library
        A[TDlpAsyncTaskRunner.RunAsync] ⟶ B[ITask da PPL em ThreadPool]
        B ⟶|Executa Trabalho Pesado| C[OnExecuteTask]
    end

    subgraph Sincronizacao Assincrona
        C ⟶|Notificacao Parcial| D[TThread.Queue]
        D ⟶|Despacha para UI Thread| E[OnProgress: Atualiza Barra de Progresso]
    end

    subgraph Conclusao
        B ⟶|Termino ou Excecao| F[TThread.Queue]
        F ⟶|Despacha para UI Thread| G[OnComplete: Notifica Sucesso / Erro]
    end
```

25.1.1 1.1 `TThread.Synchronize` vs. `TThread.Queue`

- `Synchronize`: Interrompe (*bloqueia*) a thread de trabalho até que a thread principal desenhe a interface. Isso cria gargalos severos e pode provocar *Deadlock* caso a tela esteja aguardando o término da thread via `TTask.Wait`.
- `Queue`: Enfileira o método anônimo no loop de mensagens do Windows de forma totalmente assíncrona. A thread de trabalho continua seu processamento em máxima velocidade sem qualquer bloqueio.

25.2 2. Cancelamento Cooperativo com `ITask`

A interrupção de tarefas concorrentes deve sempre respeitar o princípio do **Cancelamento Cooperativo**, no qual o código em execução consulta regularmente a variável booleana `Cancelled` para encerrar seus loops de maneira limpa:

```
procedure TFormPrincipal.AsyncTaskRunnerExecuteTask(Sender: TObject; var Cancelled: Boolean);
var
  i: Integer;
begin
  for i := 1 to 500 do
  begin
    if Cancelled then
      Exit; // Interrupção segura sem vazamento de ponteiros

    ExecutarEtapaPesada(i);

    AsyncTaskRunner.NotifyProgress(i, 500, Format('Processando etapa %d de 500...', [i]));
  end;
end;
```

25.3 3. Código-Fonte Integral do Componente

A implementação completa do componente `TDlpAsyncTaskRunner`, compatível com VCL e FireMonkey no Delphi 12 Athens+, está disponível em:

👉 **Arquivo:** `code/units/DlpAsyncTaskRunner.pas`

26 MÓDULO 22: Componentes de Comunicação de Rede e Consumo de APIs (TDlpRestClientBridge)

A integração entre clientes de desktop corporativos e serviços em nuvem (APIs RESTful, microsserviços em Go/.NET/Node, gateways de autorização OAuth2/JWT e mensageria) é indispensável para sistemas modernos. Durante anos, desenvolvedores Delphi dependeram da suíte legada Indy (`TIdHTTP`), que sofria com complexidades de SSL/OpenSSL e exigia sincronização manual de threads.

Com a introdução do framework nativo `System.Net.HttpClient` aliada à **PPL (Parallel Programming Library)** no **Delphi 12 Athens+**, dispomos de uma arquitetura nativa com aceleração de hardware, validação de certificados via SO e suporte assíncrono de fábrica. Neste módulo, concebemos o componente `TDlpRestClientBridge`.

26.1 1. A Pilha Nativa de Rede: `System.Net.HttpClient`

Diferente de sockets legados em modo síncrono bloqueante, a biblioteca `System.Net.HttpClient` delega a comunicação para as APIs do sistema operacional (`WinHTTP` no Windows, `NSURLSession` no macOS/iOS e `HttpURLConnection` no Android):

```
graph TD
    subgraph Aplicacao VCL / FMX
        A[UI Thread / Form] ⟶|ExecuteAsync| B[TDlpRestClientBridge]
    end

    subgraph Parallel Programming Library
        B ⟶ C[TTask.Run em ThreadPool]
        C ⟶ D[THTTPClient]
        D ⟶|Injeta CustomHeaders e Bearer Token| E[API RESTful Remota]
        E ⟶|Resposta HTTP 200/201/400/500| D
    end

    subgraph Retorno Seguro
        D ⟶|TThread.Queue| F[UI Message Queue]
        F ⟶|Callback Anonimo| A
    end
```

26.2 2. Injeção de Segurança e Headers Customizados

A classe encapsula a configuração de cabeçalhos e controle de timeout milissegundo:

```
function TDlpRestClientBridge.CreateConfiguredClient: THTTPClient;
var
  i: Integer;
begin
  Result := THTTPClient.Create;
  Result.ConnectionTimeout := FTimeoutSeconds * 1000;
  Result.ResponseTimeout := FTimeoutSeconds * 1000;
  Result.Accept := 'application/json';

  if FBearerToken <> '' then
    Result.CustomHeaders['Authorization'] := 'Bearer ' + FBearerToken;

  for i := 0 to FCustomHeaders.Count - 1 do
    Result.CustomHeaders[FCustomHeaders.Names[i]] := FCustomHeaders.ValueFromIndex[i];
end;
```

26.3 3. Despacho Assíncrono com Métodos Anônimos

O método `ExecuteAsync` dispara a requisição em um `TTask` isolado e retorna o resultado no contexto da thread de interface via `TThread.Queue`:

```
procedure TFormPedidos.BtnSincronizarClick(Sender: TObject);
begin
  RestClient.ExecuteAsync('POST', '/api/v1/pedidos', JsonPayload,
    procedure(StatusCode: Integer; const ResponseBody, ErrorMsg: string)
    begin
      if StatusCode = 201 then
        ShowMessage('Pedido sincronizado com sucesso!')
      else
        ShowMessage(Format('Erro [%d]: %s', [StatusCode, ErrorMsg]));
    end);
end;
```

26.4 4. Código-Fonte Integral do Componente

A implementação completa do cliente RESTful `TDlpRestClientBridge` para Delphi 12 Athens+ está disponível em:

👉 **Arquivo:** `code/units/DlpRestClientBridge.pas`

27 MÓDULO 23: Componentes de Hardware, I/O e Comunicação Serial (TDlpSerialDeviceConnector)

No ecossistema de automação comercial e chão de fábrica (PDVs de supermercados, farmácias, balanças industriais, leitores de código de barras 2D/QRCode, impressoras térmicas ESC/POS e sensores industriais), a comunicação através de portas seriais físicas (RS-232, RS-485) e virtuais (USB CDC Serial Emulation) permanece como um dos pilares mais estáveis da tecnologia da informação.

Neste módulo, construímos o componente não-visual `TDlpSerialDeviceConnector`, implementado diretamente sobre a API nativa de comunicações do Windows (Win32 Communications API) no **Delphi 12 Athens+**, dotado de thread de leitura de baixa latência e decodificador integrado de protocolos de balanças de pesagem (Toledo, Filizola).

27.1 1. A API de Comunicações do Windows (Win32 Comm API)

O Windows trata portas de comunicação como arquivos orientados a fluxo de bytes, gerenciados por manipuladores de arquivo (`HANDLE`):

```
graph TD
    subgraph Aplicacao Delphi
        A[Frente de Caixa PDV] ⟶|RequestWeight| B[TDlpSerialDeviceConnector]
    end

    subgraph Win32 Communications API
        B ⟶|CreateFile \\.\COM1 a COM256| C[Handle da Porta Serial]
        B ⟶|SetCommState com TDCB| C
        B ⟶|SetCommTimeouts| C
    end

    subgraph Thread de Leitura
        C ⟶ D[TDlpSerialReaderThread]
        D ⟶|ReadFile em Loop Nao-Bloqueante| E[Buffer de Bytes]
        E ⟶|TThread.Queue| F[UI Message Loop]
    end

    subgraph Eventos de Negocio
        F ⟶ G[OnDataReceived: Dados Brutos]
        F ⟶ H[OnWeightReceived: Peso em KG Decodificado]
    end
```

27.1.1 1.1 O Prefixo de Proteção de Portas no Windows

Ao abrir portas COM com índice igual ou maior que 10 (ex.: `COM11`, `COM18`), a chamada clássica `CreateFile('COM11', ...)` falha com erro de dispositivo não encontrado. É obrigatório estruturar a rota com a convenção de namespace do kernel:

```
PortPath := '\\.\' + FPort; // Ex: '\\.\COM14'
FPortHandle := CreateFile(PChar(PortPath), GENERIC_READ or GENERIC_WRITE,
  0, nil, OPEN_EXISTING, FILE_ATTRIBUTE_NORMAL, 0);
```

27.2 2. Decodificação de Protocolos de Balanças de Checkout

As balanças de checkout mais consagradas no varejo nacional (Toledo Prix 3, Toledo 9094, Filizola Platina) seguem o protocolo padronizado de comando e resposta:

1. O sistema envia o caractere `ENQ` (`#5`) solicitando o peso estável.
2. A balança transmite uma mensagem contendo `STX` (`#2`), 5 caracteres ASCII numéricos representando o peso em gramas, e `ETX` (`#3`):

Quadro Retornado: [STX] **0 3 5 0 0** [ETX] ⟹ 3.500 kg

A função estática `ParseToledoWeight` filtra os dígitos e converte o valor diretamente em ponto flutuante:

```
class function TDlpSerialDeviceConnector.ParseToledoWeight(const ARaw: string): Double;
var
  CleanDigits: string;
  i, ValInt: Integer;
begin
  Result := 0.0;
  CleanDigits := '';
  for i := 1 to Length(ARaw) do
  begin
    if CharInSet(ARaw[i], ['0'..'9']) then
      CleanDigits := CleanDigits + ARaw[i];
  end;

  if Length(CleanDigits) ⩾ 5 then
  begin
    CleanDigits := Copy(CleanDigits, Length(CleanDigits) - 4, 5);
    ValInt := StrToIntDef(CleanDigits, 0);
    Result := ValInt / 1000.0;
  end;
end;
```

27.3 3. Código-Fonte Integral do Componente

A implementação completa do conector de periféricos seriais `TDlpSerialDeviceConnector` está disponível em:

👉 **Arquivo:** `code/units/DlpSerialDeviceConnector.pas`

28 MÓDULO 24: Testes Unitários de Componentes, Profiling e Prevenção de Leaks (DUnitX & FastMM5)

No desenvolvimento corporativo de componentes no Delphi, a robustez é medida pela capacidade da biblioteca de operar sob condições extremas sem provocar vazamentos de memória (*memory leaks*), degradações de desempenho por repinturas desnecessárias ou falhas de ponteiro órfão. A introdução de uma esteira automatizada de testes garante que cada alteração de código preserve a estabilidade de toda a suíte.

Neste módulo, implementamos uma bateria de testes unitários com o framework moderno **DUnitX** e estabelecemos as práticas de profiling de memória com **FastMM5** no **Delphi 12 Athens+**.

28.1 1. Automação de Testes com DUnitX

O DUnitX é o framework padrão de testes xUnit do Delphi moderno, baseado em atributos (`[TestFixture]`, `[Test]`):

```
graph TD
    subgraph Suite de Testes DUnitX
        A[TDlpComponentLifecycleTest] ⟶ B[TestInstantiateAndDestroy]
        A ⟶ C[TestNotificationCrossReference]
        A ⟶ D[TestFiscalValidationCPF]
        A ⟶ E[TestFiscalValidationCNPJ]
        A ⟶ F[TestFinancialCalculations]
    end

    subgraph DUnitX Console / GUI Runner
        B ⟶ G[Assert.IsNotNull / Assert.AreEqual]
        C ⟶ G
        D ⟶ G
        E ⟶ G
        F ⟶ G
        G ⟶ H{Sucesso nos Testes?}
        H ⟶|Sim| I[Build Homologado para Producao]
        H ⟶|Nao| J[Falha no Pipeline CI/CD]
    end
```

28.1.1 1.1 Validando a Imunidade contra Ponteiros Órfãos

O teste unitário reproduz a destruição antecipada de um componente de formulário para certificar que o método `Notification` da classe host remove o ponteiro da lista interna:

```
procedure TDlpComponentLifecycleTest.TestNotificationCrossReference;
var
  HostContainer: TDlpSafeLinkedContainer;
  TargetObj: TComponent;
begin
  HostContainer := TDlpSafeLinkedContainer.Create(nil);
  try
    TargetObj := TComponent.Create(nil);
    try
      HostContainer.LinkComponent(TargetObj);
      Assert.AreEqual(1, HostContainer.LinkedCount);
    finally
      TargetObj.Free; // Simula remoção no designer ou fechamento de tela
    end;

    // A contagem deve ter retornado a zero sem corrupção de memória
    Assert.AreEqual(0, HostContainer.LinkedCount);
  finally
    HostContainer.Free;
  end;
end;
```

28.2 2. Auditoria e Detecção de Vazamentos com FastMM5

O Delphi inclui de fábrica o FastMM integrado, permitindo ativar a detecção automática de leaks na inicialização do projeto de testes (`.dpr`):

```
program DlpTestRunner;

uses
  FastMM5, // Deve ser a primeira unit do uses para interceptar todas as alocações
  DUnitX.AutoDetect.Console;

begin
  ReportMemoryLeaksOnShutdown := True;
  TDUnitX.CheckCommandLineAndRun;
end.
```

O FastMM5 provê proteção adicional contra *Buffer Overruns* e corrupção de tabelas de métodos virtuais (VMT), interceptando acessos a instâncias já liberadas (*Use-After-Free*).

28.3 3. Otimização de Repintura e o Padrão `WM_SETREDRAW`

Para controles janelados pesados (`TWinControl`), o uso de `LockWindowUpdate` pode congelar inadvertidamente outras janelas do sistema operacional. O padrão mais performático e elegante consiste em enviar a mensagem `WM_SETREDRAW`:

```
procedure TDlpCustomTabBar.BeginUpdate;
begin
  if FUpdateCount = 0 then
    SendMessage(Handle, WM_SETREDRAW, 0, 0);
  Inc(FUpdateCount);
end;

procedure TDlpCustomTabBar.EndUpdate;
begin
  if FUpdateCount > 0 then
  begin
    Dec(FUpdateCount);
    if FUpdateCount = 0 then
    begin
      SendMessage(Handle, WM_SETREDRAW, 1, 0);
      RedrawWindow(Handle, nil, 0, RDW_INVALIDATE or RDW_ALLCHILDREN or RDW_UPDATENOW);
    end;
  end;
end;
```

28.4 4. Código-Fonte Integral da Suíte de Testes

A suíte completa de testes automatizados para Delphi 12 Athens+ está disponível em:

👉 **Arquivo:** `code/units/DlpComponentTests.pas`

29 MÓDULO 25: Publicação, Versionamento e Distribuição nos Ecossistemas Oficiais (GetIt & CI/CD com MSBuild)

A coroação da engenharia de componentes no Delphi é a sua distribuição para desenvolvedores e corporações ao redor do mundo. Um componente excepcional perde valor se sua instalação for manual, complexa ou dependente de passos obscuros de configuração de *Library Paths*. No Delphi moderno (**Delphi 12 Athens+**), o padrão ouro de distribuição é a inclusão no **GetIt Package Manager** e a validação contínua em esteiras de **CI/CD** utilizando o compilador de linha de comando acionado via **MSBuild**.

Neste módulo final, estabelecemos a estrutura profissional de distribuição, o manifesto oficial do GetIt e o pipeline automatizado de integração contínua.

29.1 1. Estrutura Corporativa de Repositório Delphi

Um repositório pronto para distribuição global e múltiplos targets de plataforma (Win32 e Win64) deve seguir a estrutura canônica:

```
repo-root/
├── .github/
│   └── workflows/
│       └── ci_msbuild.yml       # Pipeline automatizado de compilação e testes
├── docs/                        # Documentação técnica e guias de uso
├── src/                         # Fontes de execução das unidades (.pas)
├── design/                      # Editores de propriedades, verbos e wizards (.pas)
├── packages/
│   ├── DlpComponents_RT.dpk     # Pacote de Runtime (Win32 / Win64)
│   ├── DlpComponents_DT.dpk     # Pacote de Designtime (Win32 / IDE)
│   └── getit_manifest.json      # Manifesto de submissão ao GetIt
├── test/                        # Bateria de testes unitários com DUnitX
└── README.md                    # Apresentação e instruções de compilação
```

29.2 2. O Ecossistema GetIt Package Manager

O **GetIt** é o repositório central oficial mantido pela Embarcadero, acessível diretamente de dentro da IDE (*Tools → GetIt Package Manager*):

```
graph TD
    subgraph Repositorio do Fabricante
        A[Codigo-Fonte Versionado] ⟶ B[Tag Git v1.0.0]
        C[Manifesto getit_manifest.json] ⟶ D[Validacao de Metadados]
    end

    subgraph Embarcadero GetIt Cloud
        D ⟶|Submissao Oficial| E[Catalogo Central GetIt]
    end

    subgraph Delphi IDE do Desenvolvedor
        E ⟶|Busca por Nome ou Categoria| F[GetIt Package Manager GUI]
        F ⟶|Instalacao em 1 Clique| G[Compilacao, BPLs e Paleta de Componentes]
    end
```

29.2.1 2.1 Estrutura do Manifesto GetIt (`getit_manifest.json`)

O manifesto instrui o instalador do GetIt sobre quais projetos compilar e quais diretórios registrar no *Library Path*:

```
{
  "id": "dlp.component.engineering",
  "name": "Delphi Component Engineering Suite",
  "version": "1.0.0",
  "compiler": "12.0",
  "product_name": "Delphi 12 Athens+",
  "platforms": [ "Win32", "Win64" ],
  "packages": [
    { "project": "DlpComponents_RT.dpk", "type": "Runtime", "platforms": [ "Win32", "Win64" ]
      },
    { "project": "DlpComponents_DT.dpk", "type": "Designtime", "platforms": [ "Win32" ] }
  ],
  "lib_paths": {
    "Win32": [ "lib\\Win32\\Release" ],
    "Win64": [ "lib\\Win64\\Release" ]
  }
}
```

29.3 3. Automação de CI/CD via MSBuild

O compilador do Delphi (`dcc32` e `dcc64`) integra-se nativamente ao motor de build padrão do Windows (`MSBuild`). Isso viabiliza compilar toda a suíte de pacotes em servidores headless (GitHub Actions ou GitLab Runners) sem necessidade de abrir a IDE:

```
# Compilação do pacote de runtime para 32-bit e 64-bit
msbuild code/packages/DlpComponents_RT.dpk /p:Config=Release /p:Platform=Win32 /t:Build
msbuild code/packages/DlpComponents_RT.dpk /p:Config=Release /p:Platform=Win64 /t:Build

# Compilação do pacote de design-time (exclusivo 32-bit para a IDE)
msbuild code/packages/DlpComponents_DT.dpk /p:Config=Release /p:Platform=Win32 /t:Build
```

A esteira executa em seguida a suíte DUnitX, garantindo que nenhum merge seja concluído sem 100% de aprovação nos testes e zero vazamentos de memória.

29.4 4. Códigos e Arquivos de Infraestrutura

Os arquivos completos de manifesto e automação estão disponíveis em:

👉 **Manifesto GetIt:** `code/packages/getit_manifest.json`
👉 **Pipeline GitHub Actions:** `code/ci/github_actions_msbuild.yml`

30 MÓDULO 26: Gráficos de Alta Densidade em Tempo Real (TDlpRealtimeChartEngine)

O processamento e a renderização fluida de séries temporais de alta frequência representam um dos desafios gráficos mais exigentes no ecossistema Delphi. Em cenários com milhares de pontos por segundo (monitoramento de telemetria médica, osciloscópios digitais e dashboards industriais), bibliotecas tradicionais causam congelamentos na thread principal e perda inaceitável de quadros (*dropped frames*).

Neste módulo, implementamos o componente visual `TDlpRealtimeChartEngine`, explorando técnicas de alocação estática com **buffer circular para 100.000 pontos**, **decimação de alta performance** e desenho acelerado em memória com a **VCL (Delphi 12 Athens+)**.

30.1 1. Arquitetura do Motor de Telemetria

O componente desacopla o momento da amostragem (`AppendPoint`) da etapa de renderização gráfica (`Paint`):

```
graph TD
    subgraph Entrada em Alta Frequência
        A[Sensor / Worker Thread] ⟶|AppendPoint em O 1| B[TDlpRealtimeChartEngine]
        B ⟶ C[(Array Circular Estático de 100.000 Pontos)]
    end

    subgraph Pipeline de Decimação
        C ⟶|GetDecimatedPoints| D[Calcula Amostras por Pixel de Largura]
        D ⟶ E[Vetor Compactado Otimizado]
    end

    subgraph Renderização 60 FPS
        E ⟶ F[Offscreen TBitmap]
        F ⟶|Double Buffering| G[Superfície de Tela da VCL]
    end
```

30.2 2. Inserção sem Alocação Dinâmica (Zero Heap Allocations)

Para sustentar 100.000 registros sem acionar o gerenciador de memória repetidamente, o tamanho do array é fixado na inicialização:

```
procedure TDlpRealtimeChartEngine.AppendPoint(const AValue: Double; const ATimestamp:
        TDateTime);
begin
  FBuffer[FHead].Value := AValue;
  FBuffer[FHead].Timestamp := ATimestamp;
  FHead := (FHead + 1) mod FCapacity;

  if FCount < FCapacity then
    Inc(FCount);

  if FAutoRange then
  begin
    if (FCount = 1) then
    begin
      FMinValue := AValue;
      FMaxValue := AValue;
    end
    else
    begin
      if AValue < FMinValue then FMinValue := AValue;
      if AValue > FMaxValue then FMaxValue := AValue;
    end;
  end;

  Invalidate;
end;
```

30.3 3. Decimação Dinâmica por Resolução de Tela

Quando o volume de amostras ultrapassa o limiar `DecimationThreshold`, o componente comprime os dados proporcionalmente ao número de pixels úteis da largura (`Width`), garantindo renderização instantânea a 60 FPS:

```
if FCount > FDecimationThreshold then
  RenderPoints := GetDecimatedPoints(Width)
else
  RenderPoints := GetDecimatedPoints(FCount);
```

30.4 4. Código-Fonte Integral do Componente

A unidade completa do componente `TDlpRealtimeChartEngine` para Delphi 12 Athens+ está disponível em:

👉 **Arquivo:** `code/units/DlpRealtimeChartEngine.pas`

31 MÓDULO 27: Medidores de Nível e Tanques Industriais (TDlpIndustrialTankGauge)

Em sistemas supervisórios SCADA e interfaces de controle de processos desenvolvidas em Delphi, a representação de silos e tanques de líquidos (combustíveis, grãos, produtos químicos) exige uma modelagem gráfica que vá além de simples barras de progresso retangulares. A equipe de engenharia necessita identificar visualmente o formato cilíndrico do recipiente, marcas de calibração numérica e a iminência de transbordo por meio de diferenciação de cores.

Neste módulo, construímos o componente vetorial `TDlpIndustrialTankGauge`, operando nativamente no Canvas da **VCL (Delphi 12 Athens+)** com baixo consumo de memória e notificações orientadas a eventos de segurança industrial.

31.1 1. Arquitetura Gráfica do Reservatório

O componente projeta um tanque industrial com proporções configuráveis e renderização antialias das bordas:

```
graph TD
    subgraph Sinais de Telemetria
        A[Controlador PLC / Rede CAN] ⟶|Nivel em m³| B[TDlpIndustrialTankGauge]
    end

    subgraph Maquina de Estados
        B ⟶ C{EvaluateAlarmState}
        C ⟶|⩽ LowAlarmLevel| D[asLowAlarm]
        C ⟶|⩾ HighAlarmLevel| E[asHighAlarm]
        C ⟶▷ Capacity| F[asOverflow]
        C ⟶|Faixa Ideal| G[asNormal]
    end

    subgraph Disparo de Acoes
        D ⟶|OnAlarmTriggered| H[Aciona Solenoide / Log de Alerta]
        E ⟶|OnAlarmTriggered| H
        F ⟶|OnAlarmTriggered| H
    end
```

31.2 2. Renderização de Linhas de Calibração e Cúpula

O desenho utiliza funções vetoriais diretas do Canvas (`Rectangle`, `Arc`, `LineTo`), garantindo total nitidez em monitores com escalonamento High-DPI:

```
// Paredes e domo do tanque
Canvas.MoveTo(TankLeft, TankTop + 10);
Canvas.LineTo(TankLeft, TankBottom);
Canvas.LineTo(TankRight, TankBottom);
Canvas.LineTo(TankRight, TankTop + 10);
Canvas.Arc(TankLeft, TankTop, TankRight, TankTop + 20,
           TankRight, TankTop + 10, TankLeft, TankTop + 10);
```

31.3 3. Código-Fonte Integral do Componente

A unidade com a implementação completa do medidor `TDlpIndustrialTankGauge` para Delphi 12 Athens+ está disponível em:

👉 **Arquivo:** `code/units/DlpIndustrialTankGauge.pas`

32 MÓDULO 28: Gráficos de Radar e Donut Vetorial (TDlpVectorDonutChart)

Em aplicações executivas e suítes de BI construídas em Delphi, a clareza e a sobriedade na apresentação de dados categóricos determinam o sucesso da interface. Os gráficos tipo **Donut (Rosca)** constituem o padrão estético dos principais sistemas operacionais modernos, combinando simplicidade visual com alto aproveitamento do espaço de tela.

Neste módulo, concebemos o componente vetorial `TDlpVectorDonutChart`, operando nativamente sobre o `Canvas` da **VCL (Delphi 12 Athens+)**, provendo detecção de hover em tempo real via matemática polar e suporte completo a estilos visuais claros e escuros.

32.1 1. Arquitetura da Rosca e Projeção Trigonométrica

O controle decompõe o problema da geometria em setores circulares concêntricos:

```
graph TD
    subgraph Dados da Coleção
        A[TDlpDonutSliceCollection] ⟶ B[Soma dos Valores Absolutos]
    end

    subgraph Rasterização no Canvas
        B ⟶ C[Loop das Fatias com Canvas.Pie]
        C ⟶ D[Canvas.Ellipse: Máscara do Orifício]
        D ⟶ E[Renderização do Rótulo Central e Percentual]
    end

    subgraph Telemetria de Interação
        F[MouseMove: Coordenadas X, Y] ⟶ G[HitTestSlice com ArcTan2]
        G ⟶ H[HoveredIndex: Dispara OnSliceHover]
    end
```

32.2 2. Detecção Angular Polar sem Overhead de CPU

Para detectar qual fatia está sob o ponteiro do mouse sem sobrecarregar a thread de interface, o método `HitTestSlice` calcula a distância radial ao centro e o ângulo correspondente em graus:

```
AngleRad := ArcTan2(Y - Cy, X - Cx);
AngleDeg := RadToDeg(AngleRad);
if AngleDeg < 0 then
  AngleDeg := AngleDeg + 360.0;
```

Se a distância estiver entre `InnerR` e `OuterR`, o componente itera pelas fatias acumulando `SweepAngle` até identificar o segmento exato, atualizando imediatamente o rótulo central com a porcentagem

correspondente.

32.3 3. Código-Fonte Integral do Componente

A unidade com a implementação completa do componente `TDlpVectorDonutChart` para Delphi 12 Athens+ está disponível em:

👉 **Arquivo:** `code/units/DlpVectorDonutChart.pas`

33 MÓDULO 29: Visualizador de Mapas de Calor (TDlpMatrixHeatmap)

A visualização de densidade numérica em matrizes multidimensionais (*Heatmaps*) é um pilar da análise de inteligência de negócios (*BI*), bioinformática, matrizes de correlação em finanças quantitativas e monitoramento de servidores em escala.

Neste módulo, construímos o `TDlpMatrixHeatmap`, projetado especificamente para Delphi 12 Athens+ VCL, utilizando a biblioteca gráfica nativa do Windows (GDI/Canvas com aceleração por double-buffering) e um algoritmo de interpolação termográfica em espaço vetorial RGB.

33.1 1. Arquitetura do Componente e Interpolação Cromática

O componente gerencia uma matriz bidimensional dinâmica fortemente tipada (`TArray<TArray<Double>>`). Os valores são mapeados continuamente entre as propriedades `MinColor` e `MaxColor`.

```
graph TD
    subgraph Entrada de Dados Matriciais
        A[Matriz Dinâmica 2D: TArray of TArray of Double] ⟶ B[RecalculateLimits: Busca Min e Max]
    end

    subgraph Pipeline Gráfico VCL
        B ⟶ C[Loop de Renderização no Paint]
        C ⟶ D[Cálculo de Proporção Tau por Célula]
        D ⟶ E[InterpolateColor: Interpolação Linear RGB]
        E ⟶ F[Vcl.Graphics.Canvas.FillRect]
        F ⟶ G[Cálculo de Luminância BT.601 para Fonte]
        G ⟶ H[Vcl.Graphics.DrawText com Alinhamento Central]
    end

    subgraph Interação do Usuário
        I[MouseDown do Usuário] ⟶ J[Conversão Coordenadas X, Y em Linha e Coluna]
        J ⟶ K[Atualiza SelectedRow e SelectedCol]
        K ⟶ L[Dispara Evento OnCellClick]
    end
```

33.1.1 Equação de Normalização e Interpolação Vetorial

Para cada elemento da matriz $M_{r,c}$, o percentual de transição cromática τ é calculado por:

$$ \tau = \text{Clamp}\left(\frac{M_{r, c} - V_{\min}}{V_{\max} - V_{\min}}, 0.0, 1.0\right) $$

A cor resultante é expressa pela composição aditiva:

$$C(\tau) = \text{RGB}(R_1 + \tau(R_2 - R_1), G_1 + \tau(G_2 - G_1), B_1 + \tau(B_2 - B_1))$$

33.2 2. Implementação e Seleção de Foco

O método `MouseDown` traduz a posição do ponteiro em índices matriciais imediatos sem iterações de laço desnecessárias:

```
procedure TDlpMatrixHeatmap.MouseDown(Button: TMouseButton; Shift: TShiftState; X, Y: Integer);
var
  CellW, CellH: Single;
  R, C: Integer;
begin
  inherited MouseDown(Button, Shift, X, Y);
  if (Button = mbLeft) and (FRowCount > 0) and (FColCount > 0) then
  begin
    CellW := Width / FColCount;
    CellH := Height / FRowCount;
    C := Trunc(X / CellW);
    R := Trunc(Y / CellH);
    if (R ⩾ 0) and (R < FRowCount) and (C ⩾ 0) and (C < FColCount) then
    begin
      FSelectedRow := R;
      FSelectedCol := C;
      Invalidate;
      if Assigned(FOnCellClick) then
        FOnCellClick(Self, R, C, FData[R, C]);
    end;
  end;
end;
```

33.3 3. Código-Fonte e Registro na VCL

O código-fonte integral encontra-se em `DlpMatrixHeatmap.pas`.

O componente é registrado na paleta `'Delphi Engenharia Avancada'` através de `RegisterComponents`, com suporte total ao Object Inspector do Delphi 12 Athens+.

34 MÓDULO 30: Motor Gerador de PDFs e Documentos Financeiros (TDlpFastPdfReportBuilder)

A geração de relatórios impressos e faturamento digital em formato PDF é indispensável em ecossistemas de software empresarial desenvolvidos em Delphi. A utilização de suítes de terceiros volumosas frequentemente acarreta overhead de memória, taxas de licenciamento elevadas ou dependências de arquivos de runtime adicionais.

Neste módulo, implementamos o `TDlpFastPdfReportBuilder` para Delphi 12 Athens+ (VCL e FMX), capaz de sintetizar binários PDF em conformidade com as normas Adobe PDF 1.4 de forma 100% nativa em Object Pascal.

34.1 1. Gramática do PDF e Diagrama de Fluxo

O gerador opera através de uma abordagem orientada a streaming: conforme os comandos de desenho (`DrawLine`, `DrawRect`, `DrawText`) são invocados pela aplicação consumidora, os operadores literais de baixo nível da especificação PostScript/PDF são gravados no buffer da página ativa.

```
graph TD
    subgraph Inicialização do Documento
        A[BeginDoc] ⟶ B[Cria Buffer da Página 1]
    end

    subgraph Emissão Gráfica e Textual
        B ⟶ C[SetDrawColor / SetFillColor]
        C ⟶ D[DrawText: Escapa caracteres e calcula coordenadas Y invertidas]
        D ⟶ E[DrawFinancialRow: Formata linha de extrato zebrada]
        E ⟶ F{Necessita de Nova Página?}
        F ⟶|Sim| G[NewPage: Aloca novo buffer de stream]
        G ⟶ C
        F ⟶|Não| H[Continua renderização]
    end

    subgraph Serialização Final
        H ⟶ I[SaveToFile / SaveToStream]
        I ⟶ J[Grava Header %PDF-1.4]
        J ⟶ K[Grava Catálogo Obj 1 e Árvore de Páginas Obj 2]
        K ⟶ L[Grava Fontes Type1 Obj 3]
        L ⟶ M[Grava Objetos de Página e Content Streams]
        M ⟶ N[Compila Tabela xref com deslocamentos exatos]
        N ⟶ O[Grava Trailer e %%EOF]
    end
```

34.2 2. Inversão do Plano Cartesiano e Escapamento Tipográfico

No Delphi moderno com suporte a strings Unicode (UTF-16), a serialização para o stream binário do PDF requer conversão precisa para ANSI/WinAnsiEncoding:

```
function TDlpFastPdfReportBuilder.EscapePdfString(const S: string): string;
var
  i: Integer;
begin
  Result := '';
  for i := 1 to Length(S) do
  begin
    case S[i] of
      '(': Result := Result + '\(';
      ')': Result := Result + '\)';
      '\': Result := Result + '\\';
    else
      Result := Result + S[i];
    end;
  end;
end;
```

A escrita de streams numéricos de ponto flutuante adota obrigatoriamente `TFormatSettings.Invariant`, impedindo que vírgulas decimais da localidade brasileira (PT-BR) corrompam a sintaxe padrão do PDF:

```
Cmd := Format('%.2f %.2f %.2f %.2f re %s'#10, [Px, Py, Pw, Ph, Op], TFormatSettings.Invariant);
GetCurrentPage.ContentStream.WriteString(Cmd);
```

34.3 3. Código-Fonte e Registro na Paleta

O código integral está disponível em `DlpFastPdfReport.pas`.

O componente é registrado em `Delphi Engenharia Avancada` e pode ser utilizado tanto em aplicações cliente Desktop quanto em microserviços de backend (Console, Windows Services, Linux Daemons).

35 MÓDULO 31: Pipeline de Exportação em Segundo Plano (TDlpDataExportPipeline)

Em aplicações corporativas construídas sobre a suíte FireDAC ou instâncias heterogêneas de `TDataSet` no Delphi 12 Athens+, a conversão e o despejo de grandes massas de dados para formatos interoperáveis (CSV, JSONL, planilhas) pode consumir dezenas de segundos ou minutos. A retenção da thread de interface durante esse processamento acarreta degradação severa da experiência do operador.

Neste módulo, implementamos o `TDlpDataExportPipeline`, projetado para desacoplar integralmente a exportação de dados através de `TThread` autônoma, garantindo segurança transacional, desativação de controles visuais com `DisableControls` e feedback de progresso sem concorrência.

35.1 1. Topologia de Concorrência e Tratamento Transacional

```
graph TD
    subgraph UI Thread - VCL/FMX
        A[Invocação: TDlpDataExportPipeline.StartExport] ⟶ B[Validação de Estado e Propriedades]
        B ⟶ C[Disparo de TDlpDataExportWorkerThread]
        H[Disparo de OnProgress na GUI] ⟶ I[Atualização de TProgressBar]
        J[Disparo de OnComplete] ⟶ K[Habilitação de Botões e Alerta de Sucesso]
    end

    subgraph Worker Thread em Segundo Plano
        C ⟶ D[DataSet.DisableControls]
        D ⟶ E[Abre TFileStream em Disco]
        E ⟶ F[Laço de Leitura EOF com Serialização de Linhas]
        F ⟶|A cada 100 linhas| G[Synchronize: Notifica Main Thread]
        G ⟶ H
        F ⟶|EOF ou Terminated| L[Fecha TFileStream]
        L ⟶ M[DataSet.EnableControls]
        M ⟶ N[Synchronize: Finalização]
        N ⟶ J
    end
```

35.1.1 Isolamento de Falhas e Cancelamento Graceful

A thread respeita a propriedade `Terminated` em cada iteração do laço. Se o usuário clicar no botão Cancelar, o pipeline interrompe imediatamente a gravação e limpa os recursos sem corromper o cursor do banco de dados:

```
procedure TDlpDataExportPipeline.CancelExport;
begin
  if FIsBusy and Assigned(FWorkerThread) then
  begin
    FWorkerThread.Terminate;
    FWorkerThread.WaitFor;
    FreeAndNil(FWorkerThread);
    FIsBusy := False;
  end;
end;
```

35.2 2. Formatos Suportados: CSV, JSONL e Planilhas XML

1. **CSV (RFC 4180):** Separação por ponto-e-vírgula (`;`) ou vírgula, com sanitização de quebras de linha e escape de aspas duplas.
2. **JSONL (JSON Lines):** O padrão universal para pipelines de Machine Learning, pipelines de embeddings e exportações para Elasticsearch / MongoDB.
3. **XML Spreadsheet:** Documentos em conformidade com o esquema Microsoft Office 2003, permitindo abertura nativa no Excel com formatação sem necessitar de instalação de DLLs COM/ActiveX.

35.3 3. Código-Fonte e Registro na Paleta

O código integral encontra-se em `DlpDataExportPipeline.pas`.

O componente é registrado na paleta `'Delphi Engenharia Avancada'`, pronto para uso em projetos VCL e FireMonkey.

36 MÓDULO 32: Processador e Validador Fiscal XML (TDlpFiscalXmlValidator)

Em sistemas ERP de alta performance desenvolvidos em Delphi, a validação de XMLs de Notas Fiscais Eletrônicas (NF-e/NFC-e) antes da transmissão para a SEFAZ ou no momento da importação de arquivos de fornecedores (DANFE/XML de entrada) evita inconsistências tributárias, multas e paralisações fiscais.

Neste módulo, implementamos o `TDlpFiscalXmlValidator` para Delphi 12 Athens+ utilizando a infraestrutura nativa `System.Hash` e `System.NetEncoding`, garantindo velocidade de execução e independência de bibliotecas MSXML ou wrappers COM legados.

36.1 1. Pipeline de Auditoria em Três Camadas

```
graph TD
    subgraph 1. Análise Sintática
        A[Entrada: XML String / Arquivo] ⟶ B{Possui infNFe ou infCte?}
        B ⟶|Não| C[Falha: XML Fiscal Desconhecido]
        B ⟶|Sim| D[Captura Atributo Id com 44 Dígitos]
    end

    subgraph 2. Aritmética Módulo 11
        D ⟶ E[Isola 43 Dígitos e DV Informado]
        E ⟶ F[Algoritmo Módulo 11 com Pesos 2..9]
        F ⟶ G{DV Calculado = DV Informado?}
        G ⟶|Não| H[Falha: Chave Inválida]
        G ⟶|Sim| I[Chave Fiscal Aprovada]
    end

    subgraph 3. Auditoria Criptográfica
        I ⟶ J[Extrai Bloco XML infNFe]
        J ⟶ K[Computa THashSHA1.GetHashBytes]
        K ⟶ L[TNetEncoding.Base64.Encode]
        L ⟶ M[Compara com Tag DigestValue da Assinatura]
        M ⟶|Igual| N[Validação 100% Concluída com Sucesso]
        M ⟶|Diferente| O[Alerta de Integridade / Hash Divergente]
    end
```

36.2 2. Implementação das Rotinas Criptográficas Nativas

Graças aos utilitários modernos do Delphi 12 Athens+, o hash e a codificação Base64 são computados sem alocação desnecessária de streams temporários:

```
function TDlpFiscalXmlValidator.ComputeSha1Base64(const Data: string): string;
var
  HashBytes: TBytes;
begin
  HashBytes := THashSHA1.GetHashBytes(TEncoding.UTF8.GetBytes(Data));
  Result := TNetEncoding.Base64.EncodeBytesToString(HashBytes);
end;
```

O registro de resultado `TDlpFiscalValidationResult` consolida o diagnóstico, incluindo logs cronológicos estruturados em `TStrings` para exibição direta em componentes visuais de interface (como `TMemo` ou `TListBox`).

36.3 3. Código-Fonte e Registro na Paleta

O código completo do componente está disponível em `DlpFiscalXmlValidator.pas`.

O componente é registrado na paleta `'Delphi Engenharia Avancada'`, funcionando perfeitamente em aplicações Desktop VCL, FMX Multiplataforma e serviços Windows/Linux.

37 MÓDULO 33: Containers ZIP Criptografados (TDlpEncryptedZipArchive)

A automação de backups, pacotes de atualização remota e troca segura de arquivos fiscais ou contábeis entre filiais exige mecanismos robustos de compressão associados a salvaguardas criptográficas. A dependência de utilitários de linha de comando como 7-Zip ou WinRAR externos fragiliza ambientes de distribuição e gera riscos de segurança.

Neste módulo, implementamos o `TDlpEncryptedZipArchive` para Delphi 12 Athens+, estruturando pacotes binários ZIP nativos com auditoria de integridade por polinômio CRC-32 (`System.Hash.THashCRC32`), suporte a chaves simétricas de criptografia e eventos desacoplados para renderização de progresso na interface de usuário.

37.1 1. Arquitetura de Empacotamento Binário

```
graph TD
    subgraph Adição de Arquivos
        A[AddFile: Caminho do Arquivo em Disco] ⟶ B[Abre TFileStream]
        B ⟶ C[Calcula CRC-32 via THashCRC32.Update]
        C ⟶ D[Armazena Tamanhos e Flags no TDlpZipItemEntry]
    end

    subgraph Geração do Contêiner: BuildArchive
        D ⟶ E[Inicia Escrita do TFileStream de Saída]
        E ⟶ F[Laço: Grava Local File Headers 0x04034B50]
        F ⟶ G[Cifra de Fluxo do Payload com Chave de Usuário]
        G ⟶ H[Notifica OnProgress na GUI]
        H ⟶ I[Grava Central Directory Headers 0x02014B50]
        I ⟶ J[Grava End of Central Directory Record 0x06054B50]
        J ⟶ K[Arquivo ZIP Finalizado e Íntegro]
    end
```

37.2 2. Compatibilidade Binária com Especificação PKWARE

O componente escreve cada campo dos cabeçalhos com tamanhos inteiros estritamente definidos (`UInt16` de 2 bytes, `UInt32` de 4 bytes) em arquitetura *Little-Endian*, permitindo que os arquivos gerados sejam lidos por qualquer ferramenta do mercado:

```
{ Escreve Local File Header }
OutStream.WriteBuffer(LocalSig, 4);      { 0x04034B50 }
OutStream.WriteBuffer(VersionNeeded, 2); { Versão mínima }
OutStream.WriteBuffer(GeneralFlag, 2);   { Bit 0: Flag de Criptografia }
OutStream.WriteBuffer(CompMethod, 2);    { Método de Compressão }
OutStream.WriteBuffer(DosTime, 2);
OutStream.WriteBuffer(DosDate, 2);
OutStream.WriteBuffer(Entry.Crc32Value, 4);
OutStream.WriteBuffer(Entry.CompressedSize, 4);
OutStream.WriteBuffer(Entry.UncompressedSize, 4);
OutStream.WriteBuffer(NameLen, 2);
OutStream.WriteBuffer(ExtraLen, 2);
OutStream.WriteBuffer(Pointer(NameBytes)^, NameLen);
```

37.3 3. Código-Fonte e Registro na Paleta

O código completo está disponível em `DlpEncryptedZipArchive.pas`.

O componente é registrado na paleta `'Delphi Engenharia Avancada'`, oferecendo uma interface limpa e intuitiva para empacotamento criptografado de dados.

38 MÓDULO 34: Validador e Emissor de Tokens JWT (TDlpJwtSecurityManager)

Em arquiteturas empresariais modernas desenvolvidas em Delphi 12 Athens+ (incluindo backends Horse, MARS-Curiosity, RAD Server e clientes ricos VCL/FMX), a circulação de credenciais puras de usuário em cada requisição é inadmissível. O uso de tokens **JWT (RFC 7519)** com criptografia simétrica HMAC-SHA256 permite validação descentralizada e alta vazão sem sobrecarregar bancos de dados relacionais com consultas de sessão repetitivas.

Neste módulo, implementamos o `TDlpJwtSecurityManager`, integrando os recursos de hash otimizados de `System.Hash` e codificação Base64URL em conformidade com as diretivas de segurança corporativa.

38.1 1. Fluxo Criptográfico de Emissão e Validação

```
graph TD
    subgraph Emissão de Credencial
        A[Entrada: Subject, TokenId e Claims Customizadas] ⟶ B[Gera Epoch UTC para iat e exp]
        B ⟶ C[Serializa Header e Payload JSON]
        C ⟶ D[Codifica em Base64URL sem Padding]
        D ⟶ E[Calcula HMAC-SHA256 sobre SigningInput]
        E ⟶ F[Concatena Token: Header.Payload.Signature]
    end

    subgraph Validação e Revogação
        F ⟶ G[Recepção do Bearer Token]
        G ⟶ H[Verifica Integridade HMAC-SHA256 em Tempo Constante]
        H ⟶|Falha| I[Rejeição: jvsInvalidSignature]
        H ⟶|Sucesso| J{TokenId jti Consta na Denylist?}
        J ⟶|Sim| K[Rejeição: jvsRevoked]
        J ⟶|Não| L{Data UTC Corrente > exp?}
        L ⟶|Sim| M[Rejeição: jvsExpired]
        L ⟶|Não| N[Token Válido: Libera Operação]
    end
```

38.2 2. Padrões de Segurança: Timing Attacks e Base64URL

A codificação de URL padrão RFC 7519 difere do Base64 comum por substituir `+` por `-`, `/` por `_` e omitir o preenchimento com `=`, evitando que os tokens quebrem cabeçalhos HTTP `Authorization: Bearer <token>`:

```
function TDlpJwtSecurityManager.Base64UrlEncode(const InputBytes: TBytes): string;
begin
  Result := TNetEncoding.Base64URL.EncodeBytesToString(InputBytes);
  Result := StringReplace(Result, #13, '', [rfReplaceAll]);
  Result := StringReplace(Result, #10, '', [rfReplaceAll]);
  Result := StringReplace(Result, '=', '', [rfReplaceAll]);
end;
```

A validação de assinatura adota obrigatoriamente rotina em tempo constante para eliminar riscos de vazamento de chave por análise de latência.

38.3 3. Código-Fonte e Registro na Paleta

O código completo está disponível em `DlpJwtSecurityManager.pas`.

O componente é registrado na paleta `'Delphi Engenharia Avancada'`, permitindo fácil acoplamento a módulos de autenticação e comunicação HTTP.

39 MÓDULO 35: Cofre Seguro de Credenciais e Segredos (TDlpCryptoVaultStorage)

A segurança em camadas (*Defense in Depth*) prescreve que dados altamente sensíveis não devem residir desprotegidos em nenhum ponto do ciclo de vida da aplicação. Ferramentas forenses e malwares especializados (como *Process Dumper* e *Mimikatz*) realizam varreduras na memória do processo buscando padrões de tokens OAuth, chaves de API e strings de conexão de bancos de dados como Oracle, PostgreSQL e SQL Server.

Neste módulo, implementamos o `TDlpCryptoVaultStorage` para Delphi 12 Athens+, garantindo que segredos empresariais permaneçam cifrados mesmo durante sua residência na memória RAM e sejam eliminados deterministicamente após o uso.

39.1 1. Topologia de Proteção em Memória

```
graph TD
    subgraph Inicialização Dinâmica
        A[Injeção de Entropia: PID + TickCount + Hora + Rand] ⟶ B[Geração de FMasterRuntimeKey via
THashSHA2]
    end

    subgraph Operação StoreSecret
        C[Entrada: Chave e Segredo] ⟶ D[Converte em TBytes]
        D ⟶ E[Gera Byte Salt Exclusivo]
        E ⟶ F[Transformação XOR com Chave Mestre e Salt]
        F ⟶ G[Armazena em TDlpVaultEntry]
        G ⟶ H[SecureClearBuffer: Sobrescreve TBytes com Zeros]
    end

    subgraph Operação RetrieveSecret
        I[Solicitação da Chave] ⟶ J[Decifra em Buffer Efêmero]
        J ⟶ K[Gera String de Retorno]
        K ⟶ L[SecureClearBuffer: Limpeza Imediata do Buffer Efêmero]
    end
```

39.2 2. Destruição Determinística de Buffers (ZeroMemory)

Para prevenir que o compilador otimize ou descarte chamadas de limpeza consideradas “mortas” (*Dead Store Elimination*), o componente força a sobrescrita byte a byte:

```
procedure TDlpCryptoVaultStorage.SecureClearBuffer(var Buffer; Size: Integer);
var
  P: PByte;
  i: Integer;
begin
  P := PByte(@Buffer);
  for i := 0 to Size - 1 do
    P[i] := 0;
end;
```

A persistência em arquivo (`SaveVaultToFile` e `LoadVaultFromFile`) utiliza derivação de chave simétrica com salting estendido, assegurando que o arquivo do cofre armazenado em disco seja indecifrável sem a senha mestra configurada pelo administrador do sistema.

39.3 3. Código-Fonte e Registro na Paleta

O código completo do componente está disponível em `DlpCryptoVaultStorage.pas`.

O componente é registrado na paleta `'Delphi Engenharia Avancada'`, permitindo fácil integração com formulários de login, data modules e microserviços de backend.

40 MÓDULO 36: Motor de Permissões Baseadas em Papéis (TDlpSecurityRoleManager)

A segurança em nível de apresentação (*Presentation-Layer Security*) garante que interfaces com o usuário reflitam fielmente os privilégios concedidos pelo modelo de controle de acesso corporativo. Sem um gerenciador centralizado, sistemas corporativos no Delphi 12 Athens+ acabam repletos de código imperativo redundante nos formulários VCL, aumentando drasticamente o custo de manutenção e a probabilidade de falhas de segurança.

Neste módulo, implementamos o `TDlpSecurityRoleManager`, um componente VCL que adota o padrão declarativo de coleção de regras, inspeciona recursivamente a hierarquia de `TWinControl` e aplica restrições contextuais em tempo de execução.

40.1 1. Fluxo de Varredura e Hierarquia VCL

```
graph TD
    A[Sessão de Usuário Definida] ⟶ B[Chamada TDlpSecurityRoleManager.ApplySecurity Form]
    B ⟶ C[Itera sobre Controls de AContainer]
    C ⟶ D{Existe Regra Mapeada para Ctrl.Name?}
    D ⟶|Sim| E{HasPermission ou HasRole ADMIN?}
    E ⟶|Não| F[Aplica Ação: uaHide / uaDisable / uaReadOnly / uaMask]
    E ⟶|Sim| G[Mantém Estado Original]
    D ⟶|Não| G
    G ⟶ H{Ctrl is TWinControl?}
    F ⟶ H
    H ⟶|Sim| I[Chamada Recursiva ApplySecurity Ctrl]
    H ⟶|Não| J[Próximo Controle]
    I ⟶ J
```

40.2 2. Flexibilidade e Design-Time Support

Ao implementar a coleção tipada `TDlpRolePermissionRules` herdada de `TCollection`, o componente permite que o arquiteto de software ou desenvolvedor configure todas as regras de permissão da tela visualmente através do editor de coleções da IDE do Delphi.

Em tempo de execução, a chamada única no evento `OnShow` do formulário ativa a proteção completa:

```
procedure TfrmCadastroClientes.FormShow(Sender: TObject);
begin
  SecurityRoleManager.ApplySecurity(Self);
end;
```

40.3 3. Código-Fonte e Registro na Paleta

O código completo do componente está disponível em `DlpSecurityRoleManager.pas`.

O componente é registrado na paleta `'Delphi Engenharia Avancada'`, pronto para distribuição e integração corporativa.

41 MÓDULO 37: Auditor de Integridade de Executáveis (TDlpIntegrityWatchdog)

A segurança cibernética de aplicações comerciais e industriais compiladas no Delphi 12 Athens+ exige proteção ativa contra alterações não autorizadas no código de máquina. A modificação de binários (.exe / .dll) em clientes finais para contornar licenciamento, alterar rotinas fiscais ou extrair segredos industriais é uma ameaça tangível.

Neste módulo, implementamos o `TDlpIntegrityWatchdog`, integrando cálculo de integridade SHA-256 do arquivo executável (`System.Hash.THashSHA2`), interrogação direta à API do Windows (`IsDebuggerPresent`) e análise de prólogo de memória contra *hooks* maliciosos.

41.1 1. Fluxo de Inspeção em Camadas

```
graph TD
    subgraph Inicialização da Aplicação
        A[Application.Initialize] ⟶ B[TDlpIntegrityWatchdog.VerifyBinaryIntegrity]
        B ⟶ C[Lê ParamStr 0 com FileStream Compartilhado]
        C ⟶ D[THashSHA2.GetHashString]
        D ⟶ E{Hash = ExpectedHash?}
        E ⟶|Não| F[Dispara Alerta / Termina Processo]
        E ⟶|Sim| G[TDlpIntegrityWatchdog.CheckSystemSecurity]
    end

    subgraph Inspeção de Ambiente de Depuração
        G ⟶ H{IsDebuggerPresent WinAPI}
        H ⟶|Sim| I[Identificado Debugger: Aborta com Halt 1]
        H ⟶|Não| J{Varredura de Opcode 0xCC em Memória}
        J ⟶|Sim| I
        J ⟶|Não| K[Aplicação Autêntica e Liberada]
    end
```

41.2 2. Aborto Seguro e Confiabilidade

Quando uma violação de integridade é confirmada, o componente não deve exibir mensagens de depuração detalhadas que ajudem o invasor. Ele dispara o evento `OnTamperDetected` para telemetria remota e encerra imediatamente a execução através do procedimento nativo `Halt(1)`:

```
procedure TDlpIntegrityWatchdog.TriggerViolation(AType: TDlpTamperType; const ADesc: string);
var
  ShouldAbort: Boolean;
begin
  ShouldAbort := FTerminateOnViolation;
  if Assigned(FOnTamperDetected) then
    FOnTamperDetected(Self, AType, ADesc, ShouldAbort);
  if ShouldAbort then
    Halt(1);
end;
```

41.3 3. Código-Fonte e Registro na Paleta

O código integral encontra-se em `DlpIntegrityWatchdog.pas`.

O componente é registrado na paleta `'Delphi Engenharia Avancada'`, fornecendo uma linha de defesa essencial para softwares comerciais.

42 MÓDULO 38: Cliente de Mensageria IoT via MQTT (TDlpMqttClientBridge)

A integração de softwares desenvolvidos em Delphi 12 Athens+ com ecossistemas de Internet das Coisas (balanças eletrônicas de pesagem contínua, CLPs industriais, sensores de telemetria predial e frotas veiculares) requer suporte ao protocolo padrão **MQTT 3.1.1**.

Neste módulo, implementamos o `TDlpMqttClientBridge`, manipulando buffers binários (`TBytes`) e montando pacotes com codificação de comprimento remanescente, identificação de pacotes e tratamento assíncrono de eventos de recepção.

42.1 1. Fluxo de Troca de Mensagens MQTT

```
sequenceDiagram
    autonumber
    participant App as Delphi App (TDlpMqttClientBridge)
    participant Broker as Broker MQTT (Mosquitto/EMQX)
    participant Sensor as Dispositivo IoT / Sensor

    App->>Broker: CONNECT [ClientId, CleanSession=1, KeepAlive=60]
    Broker-->>App: CONNACK [ReturnCode=0 (Accepted)]
    App->>Broker: SUBSCRIBE [PacketId=1, Topic="telemetry/temp", QoS=1]
    Broker-->>App: SUBACK [PacketId=1, GrantedQoS=1]

    Sensor->>Broker: PUBLISH [Topic="telemetry/temp", Payload="24.8 C"]
    Broker->>App: PUBLISH [Topic="telemetry/temp", Payload="24.8 C"]
    App->>App: Dispara OnMessageReceived(Topic, Payload)
```

42.2 2. Parsing de Pacotes de Entrada

O componente intercepta o fluxo de dados brutos e decodifica o cabeçalho fixo (*Fixed Header*) para despachar o evento correto à interface da aplicação:

```
procedure TDlpMqttClientBridge.ProcessIncomingBytes(const AData: TBytes);
var
  PacketType: Byte;
  TopicLen: Word;
  TopicStr, PayloadStr: string;
begin
  if Length(AData) < 2 then Exit;
  PacketType := AData[0] shr 4;

  case PacketType of
    2: { CONNACK }
      begin
        if (Length(AData) ⩾ 4) and (AData[3] = 0) then
        begin
          FIsConnected := True;
          if Assigned(FOnConnectionChange) then
            FOnConnectionChange(Self, True, 'Conexao com broker MQTT confirmada.');
        end;
      end;

    3: { PUBLISH Recebido }
      begin
        if Length(AData) > 4 then
        begin
          TopicLen := (AData[2] shl 8) or AData[3];
          TopicStr := TEncoding.UTF8.GetString(AData, 4, TopicLen);
          PayloadStr := TEncoding.UTF8.GetString(AData, 4 + TopicLen, Length(AData) - (4 +
        TopicLen));
          if Assigned(FOnMessageReceived) then
            FOnMessageReceived(Self, TopicStr, PayloadStr, qosAtMostOnce, False);
        end;
      end;
  end;
end;
```

42.3 3. Código-Fonte e Registro na Paleta

O código integral encontra-se em `DlpMqttClientBridge.pas`.

O componente é registrado na paleta `'Delphi Engenharia Avancada'`, pronto para conectar aplicações Delphi ao mundo IoT e mensageria distribuída.

43 MÓDULO 39: Servidor de Webhooks Embutido (TDlpEmbeddedWebhookServer)

A modernização de aplicações ERP e PDV no Delphi 12 Athens+ passa pela capacidade de receber notificações ativas da nuvem sem a necessidade de manter laços repetitivos de consulta (*Polling* de banco ou requisições HTTP contínuas). A recepção direta de Webhooks de provedores de pagamento (Stripe, Mercado Pago, Asaas) acelera a confirmação de faturamento e reduz drasticamente o tráfego de rede.

Neste módulo, implementamos o `TDlpEmbeddedWebhookServer`, construindo um microservidor HTTP baseado na API de Sockets nativa do Windows (`Winapi.WinSock2`), garantindo velocidade de resposta na ordem de microssegundos e consumo mínimo de memória.

43.1 1. Topologia de Conexão e Ciclo de Vida HTTP

```
graph TD
    A[Gateway Remoto: POST /webhook/cobranca] ⟶ B[WinSock accept na Porta Configurada]
    B ⟶ C[TDlpWebhookListenerThread.HandleClient]
    C ⟶ D[Leitura de Bytes até Fim de Cabeçalhos HTTP]
    D ⟶ E[Parse de Método, Rota, Parâmetros e Headers]
    E ⟶ F[Disparo Sincronizado de OnWebhookReceived]
    F ⟶ G[Geração de Resposta HTTP 200 OK com Resumo JSON]
    G ⟶ H[WinSock send do Response Payload]
    H ⟶ I[closesocket e Liberação de Recursos]
```

43.2 2. Extração de Assinaturas e Proteção Anti-Spoofing

A segurança de webhooks exige que o servidor receptor valide a assinatura criptográfica contida nos cabeçalhos (como `X-Signature` ou `X-Hub-Signature-256`) comparando com o token secreto configurado em `SecretToken`.

A estrutura `TDlpWebhookRequest` isola o método HTTP, rota, cabeçalhos de assinatura e corpo bruto da mensagem:

```
TDlpWebhookRequest = record
  Method: string;
  Path: string;
  QueryString: string;
  ContentType: string;
  Signature: string;
  RawBody: string;
  ClientIP: string;
end;
```

43.3 3. Código-Fonte e Registro na Paleta

O código integral encontra-se em `DlpEmbeddedWebhookServer.pas`.

O componente é registrado na paleta `'Delphi Engenharia Avancada'`, operando perfeitamente em aplicações Desktop VCL, FMX e em Daemons/Windows Services.

44 MÓDULO 40: Cliente WebSocket com Reconexão Automática (TDlpWebSocketLiveClient)

A construção de interfaces reativas no Delphi 12 Athens+ (seja em painéis de controle industriais, telas de cotação ou aplicações corporativas distribuídas) depende de canais WebSocket (RFC 6455). Conexões de longa duração inevitavelmente sofrem quedas transitórias em redes móveis e ambientes corporativos com proxies restritivos.

Neste módulo, implementamos o `TDlpWebSocketLiveClient`, manipulando buffers binários `TBytes`, gerando o enquadramento mascarado do cliente e gerenciando uma fila de mensagens offline associada a um algoritmo de reconexão por backoff exponencial.

44.1 1. Máquina de Estados e Tolerância a Desconexões

```
stateDiagram-v2
    [*] ⟶ wssDisconnected
    wssDisconnected ⟶ wssConnecting : Connect()
    wssConnecting ⟶ wssConnected : Handshake Sucesso
    wssConnecting ⟶ wssReconnecting : Falha de Rede
    wssConnected ⟶ wssReconnecting : Network Drop
    wssConnected ⟶ wssDisconnected : Disconnect()

    state wssReconnecting {
        [*] ⟶ AguardaBackoff
        AguardaBackoff ⟶ TentaReconectar : Timer Delay Expira
        TentaReconectar ⟶ Sucesso
        TentaReconectar ⟶ Falha : Multiplica Delay x 2
        Falha ⟶ AguardaBackoff
    }

    wssReconnecting ⟶ wssConnected : Sucesso + FlushOfflineQueue()
```

44.2 2. Enfileiramento de Mensagens em Modo Desconectado

Quando a interface gráfica invoca `SendText` durante uma oscilação de rede, o componente retém o payload sem disparar exceções:

```
procedure TDlpWebSocketLiveClient.SendText(const AText: string);
begin
  if FState = wssConnected then
    BuildClientFrame(AText)
  else
    FOfflineQueue.Add(AText);
end;
```

Na recuperação do canal, todas as mensagens pendentes são enviadas ordenadamente mantendo a integridade da conversação.

44.3 3. Código-Fonte e Registro na Paleta

O código completo do componente está disponível em `DlpWebSocketLiveClient.pas`.

O componente é registrado na paleta `'Delphi Engenharia Avancada'`, operando tanto no ecossistema VCL quanto em aplicações multiplataforma FireMonkey (FMX).

45 MÓDULO 41: Hub de Notificações Push Desktop/Mobile (TDlpPushNotificationHub)

Em ecossistemas comerciais baseados em Delphi 12 Athens+ onde clientes utilizam tanto aplicações desktop VCL no escritório quanto aplicativos móveis FireMonkey (FMX) no Android ou iOS, a centralização do envio de mensagens push em um componente desacoplado acelera a implementação de fluxos de autorização em dois fatores (2FA), alertas de estoque baixo e chamadas operacionais urgentes.

Neste módulo, implementamos o `TDlpPushNotificationHub`, gerando payloads JSON canônicos e configurando prioridades de entrega para FCM v1, APNs e WebPush.

45.1 1. Arquitetura Multicanal do Hub

```
graph TD
    A[Disparo: TDlpPushNotificationHub.DispatchNotification] ⟶ B{Propriedade Provider}
    B ⟶|ppFirebaseFcmV1| C[BuildFcmV1Payload: Estrutura canônica Google v1]
    B ⟶|ppAppleApns| D[BuildApnsPayload: Dicionário aps + badges Apple]
    B ⟶|ppGenericWebPush| E[BuildWebPushPayload: W3C Push API]
    C ⟶ F[Despacho via HTTP POST com Bearer Token]
    D ⟶ F
    E ⟶ F
    F ⟶ G[Disparo de OnNotificationSent na Aplicação Delphi]
```

45.2 2. Tipagem Estruturada e Métodos Construtores

Em vez de concatenar strings JSON manuais propensas a falhas de pontuação e aspas, o desenvolvedor utiliza o método de fábrica `CreateMessage`:

```
var
  Msg: TDlpPushMessage;
begin
  Msg := PushHub.CreateMessage(
    'eX8jF...token_do_aparelho',
    'Pedido Aprovado!',
    'O faturamento do pedido #1893 foi concluído com sucesso.'
  );
  Msg.Priority := ppHigh;
  Msg.Badge := 1;
  PushHub.DispatchNotification(Msg);
end;
```

45.3 3. Código-Fonte e Registro na Paleta

O código completo está disponível em `DlpPushNotificationHub.pas`.

O componente é registrado na paleta `'Delphi Engenharia Avancada'`, pronto para integração em soluções corporativas modernas.

46 MÓDULO 42: Dispatcher de Prompts para LLMs Locais (TDlpAiPromptDispatcher)

A demanda por inteligência artificial em sistemas comerciais construídos em Delphi 12 Athens+ exige soluções seguras contra vazamento de segredos industriais e custos imprevistos de APIs em nuvem. A execução de servidores de inferência locais como Ollama, LM Studio e vLLM em estações com placas de vídeo dedicadas ou servidores locais corporativos permite integrar assistentes de IA de forma soberana e privada.

Neste módulo, implementamos o `TDlpAiPromptDispatcher`, capaz de despachar requisições para LLMs locais, gerenciar streaming de tokens em tempo real através de `TThread` isolada e mensurar latência via `System.Diagnostics.TStopwatch`.

46.1 1. Topologia de Concorrência e Métricas

```
graph TD
    subgraph UI Thread Delphi
        A[Invocação: TDlpAiPromptDispatcher.GenerateStream] ⟶ B[Alocação de TDlpAiStreamWorkerThread]
        G[Disparo de OnTokenReceived] ⟶ H[Efeito 'Máquina de Escrever' no TMemo]
        I[Disparo de OnComplete] ⟶ J[Exibe Duração Ms e Tokens por Segundo]
    end

    subgraph Streaming Worker Thread
        B ⟶ C[Disparo de TStopwatch.StartNew]
        C ⟶ D[Requisição HTTP com stream=true]
        D ⟶ E[Laço de Leitura de Fragmentos do Modelo]
        E ⟶ F[Synchronize: Entrega Chunks Parciais]
        F ⟶ G
        E ⟶|Fim da Geração / isDone| K[TStopwatch.Stop e Cálculo de Vazão]
        K ⟶ L[Synchronize: Finalização]
        L ⟶ I
    end
```

46.2 2. Padrão de Integração e Cancelamento

A arquitetura orientada a eventos permite que a interface visual permaneça fluida e reativa durante todo o processo de inferência:

```
procedure TfrmAtendimento.btnGerarRespostaClick(Sender: TObject);
begin
  MemoResposta.Lines.Clear;
  AiDispatcher.GenerateStream(MemoPergunta.Text);
end;

procedure TfrmAtendimento.AiDispatcherTokenReceived(Sender: TObject;
  const ATokenChunk, ACumulativeText: string);
begin
  MemoResposta.Lines.Text := ACumulativeText;
  MemoResposta.SelStart := Length(MemoResposta.Lines.Text);
end;
```

46.3 3. Código-Fonte e Registro na Paleta

O código completo do componente está disponível em `DlpAiPromptDispatcher.pas`.

O componente é registrado na paleta `'Delphi Engenharia Avancada'`, pronto para enriquecer sistemas ERP com processamento de linguagem natural.

47 MÓDULO 43: Parser de Respostas Estruturadas JSON (TDlpAiStructuredExtractor)

No desenvolvimento de sistemas corporativos em Delphi 12 Athens+ integrados a modelos generativos de IA, o maior obstáculo prático para a automação de rotinas (como classificação automática de títulos a pagar, roteamento de chamados de suporte e validação de documentos fiscais digitalizados via OCR) é a instabilidade sintática do JSON retornado pelos LLMs.

Neste módulo, implementamos o `TDlpAiStructuredExtractor`, um extrator resiliente que higieniza saídas brutas de IA, remove marcações Markdown, corrige imperfeições sintáticas e hidrata records fortemente tipados sem falhas de runtime.

47.1 1. Topologia de Limpeza e Validação

```
graph TD
    subgraph Entrada Não-Estruturada
        A[Resposta do LLM: Aqui está o resultado ```json ...] ⟶ B[CleanMarkdownFences]
    end

    subgraph Sanitização e Reparo Sintático
        B ⟶ C[Localiza primeiro '{' e último '}']
        C ⟶ D[RepairJsonSyntax: Remove Trailing Commas]
        D ⟶ E[Substitui Delimitadores Inválidos]
    end

    subgraph Hidratação Tipada Delphi
        E ⟶ F[ExtractField: String / Bool / Float com InvariantCulture]
        F ⟶ G[TDlpCustomerIntentRecord Preenchido]
        G ⟶ H[Integração Segura com Regras de Negócio e FireDAC]
    end
```

47.2 2. Robustez na Conversão de Ponto Flutuante

Divergências regionais de formatação entre o ponto decimal emitido pelo modelo generativo e a vírgula utilizada em sistemas operacionais configurados em Português do Brasil são neutralizadas pela utilização sistemática de `TFormatSettings.Invariant`:

```
function TDlpAiStructuredExtractor.ExtractFloatField(const Json, FieldName: string): Double;
var
  V: string;
begin
  V := Trim(ExtractField(Json, FieldName));
  V := StringReplace(V, ',', '.', [rfReplaceAll]);
  Result := StrToFloatDef(V, 0.0, TFormatSettings.Invariant);
end;
```

47.3 3. Código-Fonte e Registro na Paleta

O código integral encontra-se em `DlpAiStructuredExtractor.pas`.

O componente é registrado na paleta `'Delphi Engenharia Avancada'`, permitindo transformar textos gerados por inteligência artificial em estruturas tipadas seguras.

48 MÓDULO 44: Motor de Busca Vetorial e Embeddings (TDlpVectorEmbeddingStore)

A implementação de mecanismos de busca semântica em catálogos com centenas de milhares de produtos, manuais técnicos e bases de clientes em Delphi 12 Athens+ capacita sistemas comerciais a responder perguntas conceituais de clientes sem necessitar de bancos vetoriais em nuvem (como Pinecone, Weaviate ou Milvus), eliminando custos recorrentes em moeda estrangeira e latência de rede.

Neste módulo, implementamos o `TDlpVectorEmbeddingStore`, um banco de dados vetorial nativo em memória construído com `TArray<Single>`, provendo indexação vetorial, cálculo de similaridade por cosseno e persistência binária otimizada.

48.1 1. Arquitetura de Busca Semântica e RAG

```
graph TD
    subgraph Fase 1: Ingestão de Conhecimento
        A[Catálogo de Produtos / FAQ Corporativo] ⟶ B[Gera Embeddings 384d / 768d]
        B ⟶ C[TDlpVectorEmbeddingStore.AddDocument]
        C ⟶ D[Persistência em Arquivo Binário VECSTORE]
    end

    subgraph Fase 2: Consulta e Ranking
        E[Busca do Usuário: 'furadeira de impacto sem fio'] ⟶ F[Gera Query Embedding]
        F ⟶ G[TDlpVectorEmbeddingStore.SearchTopK QueryVector, 5]
        G ⟶ H[Cálculo Paralelizável de Cosine Similarity]
        H ⟶ I[Ranking Parcial Top-K Ordenado por Score]
        I ⟶ J[Exibição na Interface VCL/FMX]
    end
```

48.2 2. Álgebra Linear e Normalização de Módulo

A similaridade por cosseno quantifica a coincidência de direção entre os dois vetores multidimensionais, variando no intervalo [− 1.0, + 1.0], onde valores próximos a + 1.0 atestam afinidade semântica máxima:

```
function TDlpVectorEmbeddingStore.ComputeCosineSimilarity(const V1, V2: TDlpEmbeddingVector;
  Mag1, Mag2: Single): Single;
var
  Dot, Denominator: Single;
  i, Len: Integer;
begin
  Len := Length(V1);
  if (Len <> Length(V2)) or (Len = 0) then Exit(0.0);

  Dot := 0.0;
  for i := 0 to Len - 1 do
    Dot := Dot + (V1[i] * V2[i]);

  Denominator := Mag1 * Mag2;
  if Denominator > 0.000001 then
    Result := Dot / Denominator
  else
    Result := 0.0;
end;
```

48.3 3. Código-Fonte e Registro na Paleta

O código completo do componente está disponível em `DlpVectorEmbeddingStore.pas`.

O componente é registrado na paleta `'Delphi Engenharia Avancada'`, permitindo criação imediata de motores de recomendação e RAG em Delphi.

49 MÓDULO 45: Construtor Universal de Daemons e Serviços (TDlpSystemDaemonController)

A consolidação de arquiteturas de backend corporativas no Delphi 12 Athens+ exige que o mesmo binário ou projeto possa ser implantado como um Serviço do Windows gerenciado pelo Service Control Manager (SCM) e como um Daemon Linux supervisionado pelo Systemd, além de permitir testes rápidos no terminal local sem a burocracia de registrar o serviço no sistema operacional durante a fase de desenvolvimento.

Neste módulo, implementamos o `TDlpSystemDaemonController`, provendo despacho unificado de eventos de ciclo de vida (`OnStart`, `OnStop`, `OnPause`, `OnContinue`), controle de parâmetros de inicialização e suporte a logs estruturados.

49.1 1. Topologia de Instalação e Execução

```
graph TD
    A[Executável Inicia: ProcessCommandLine] ⟶ B{Argumentos de Linha de Comando?}
    B ⟶|--install ou -i| C[Registra Serviço no Windows SCM / Systemd]
    B ⟶|--uninstall ou -u| D[Remove Serviço do SCM / Desabilita no Systemd]
    B ⟶|--systemd-unit| E[Emite Arquivo .service para StdOut e Encerra]
    B ⟶|--debug ou -d| F[Executa em Modo Console Interativo]
    B ⟶|Sem Parâmetros / Modo Normal| G[StartService: Inicia em Segundo Plano]

    F ⟶ H[Loop de Espera com Interrupção por Teclado]
    G ⟶ I[Despacho de Eventos OnStart e OnStop]
```

49.2 2. Padrão de Integração em Projetos de Console / Service

No arquivo principal do projeto (.dpr), o controle de execução é simplificado para uma chamada declarativa:

```
program BackendDaemon;

{$APPTYPE CONSOLE}

uses
  System.SysUtils,
  DlpSystemDaemonController in 'units\DlpSystemDaemonController.pas';

var
  Daemon: TDlpSystemDaemonController;
begin
  Daemon := TDlpSystemDaemonController.Create(nil);
  try
    Daemon.ServiceName := 'DelphiEnterpriseService';
    Daemon.OnStart := procedure(Sender: TObject; var Handled: Boolean)
    begin
      Writeln('Serviço iniciado com sucesso.');
    end;
    Daemon.ProcessCommandLine;
  finally
    Daemon.Free;
  end;
end.
```

49.3 3. Código-Fonte e Registro na Paleta

O código completo do componente está disponível em `DlpSystemDaemonController.pas`.

O componente é registrado na paleta `'Delphi Engenharia Avancada'`, pronto para a construção de daemons de missão crítica.

50 MÓDULO 46: Sonda de Monitoramento de Sistema (TDlpSystemMetricsProbe)

O diagnóstico proativo de desempenho em sistemas corporativos desenvolvidos em Delphi 12 Athens+ previne degradações de velocidade decorrentes de fragmentação de memória ou concorrência excessiva de threads de background.

Neste módulo, implementamos o `TDlpSystemMetricsProbe`, gerando instantâneos periódicos com consumo de CPU, tamanho do conjunto de trabalho de memória (*Working Set*) e contagem de handles abertos, alimentando dashboards de saúde da aplicação.

50.1 1. Métricas Essenciais de Telemetria

```
graph TD
    subgraph Sonda Periódica: TDlpSystemMetricsProbe
        A[Timer Tick no Intervalo SamplingIntervalMs] ⟶ B[SampleNow]
        B ⟶ C[Computa Porcentagem de CPU do Processo]
        B ⟶ D[Lê WorkingSet e Private Bytes de Memória]
        B ⟶ E[Conta Handles e Threads Ativas]
        C ⟶ F[TDlpProcessMetricsSnapshot]
        D ⟶ F
        E ⟶ F
        F ⟶ G[OnMetricsSampled Event]
        G ⟶ H[Atualização de Indicadores Visuais de Status]
    end
```

50.2 2. Padrão de Integração em Dashboards de Monitoramento

O evento `OnMetricsSampled` entrega os valores já consolidados para consumo direto em componentes visuais (como os criados na Parte VIII):

```
procedure TfrmPrincipal.SystemMetricsProbeSampled(Sender: TObject;
  const ASnapshot: TDlpProcessMetricsSnapshot);
begin
  GaugeCpu.Value := ASnapshot.CpuUsagePercent;
  lblMemoria.Caption := Format('RAM: %.1f MB', [ASnapshot.WorkingSetBytes / (1024 * 1024)]);
  lblHandles.Caption := Format('Handles: %d', [ASnapshot.OsHandlesCount]);
end;
```

50.3 3. Código-Fonte e Registro na Paleta

O código completo do componente está disponível em `DlpSystemMetricsProbe.pas`.

O componente é registrado na paleta `'Delphi Engenharia Avancada'`, permitindo fácil adição de telemetria a qualquer projeto Delphi.

51 MÓDULO 47: Gerenciador de Pool de Conexões SQL (TDlpDatabaseConnectionPool)

O gerenciamento eficiente de conexões com bancos de dados relacionais em aplicações servidoras de alta demanda (como APIs Horse/DataSnap ou backends corporativos no Delphi 12 Athens+) exige uma camada de pooling dedicada para evitar a sobrecarga de conexões concorrentes físicas no FireDAC.

Neste módulo, implementamos o `TDlpDatabaseConnectionPool`, orquestrando instâncias de conexões protegidas por `System.SyncObjs.TCriticalSection`, garantindo limites mínimos e máximos configuráveis, tolerância a timeout de aquisição e expurgo preventivo de conexões inativas.

51.1 1. Topologia de Concorrência e Métricas do Pool

```
graph TD
    subgraph Threads de Aplicação
        T1[Worker Thread 1] ⟶|AcquireConnection| P[TDlpDatabaseConnectionPool]
        T2[Worker Thread 2] ⟶|AcquireConnection| P
        T3[Worker Thread 3] ⟶|AcquireConnection| P
    end

    subgraph Gerenciamento Thread-Safe com TCriticalSection
        P ⟶|Lock.Enter| M{Existe Conexão Ociosa?}
        M ⟶|Sim| O[Marca InUse e Retorna Instância]
        M ⟶|Não e Pool < Max| N[Instancia Nova Conexão Física]
        M ⟶|Não e Pool Cheio| W[Aguarda até ConnectionTimeoutMs]
        N ⟶ O
    end

    subgraph Retorno e Limpeza
        T1 ⟶|ReleaseConnection| R[Marca InUse = False]
        P ⟶|PurgeIdleConnections| K[Elimina Conexões Zumbis Excedentes]
    end
```

51.2 2. Padrão de Uso em Worker Threads

O padrão clássico `try ... finally` assegura que a conexão alugada retorne ao pool mesmo se ocorrerem exceções de banco de dados:

```
var
  ConnObj: TObject;
  Conn: TFDConnection;
begin
  if Pool.AcquireConnection(ConnObj, 3000) then
  begin
    Conn := TFDConnection(ConnObj);
    try
      // Executa consultas e atualizações SQL com segurança
      Conn.ExecSQL('UPDATE Parametros SET UltimoAcesso = NOW()');
    finally
      Pool.ReleaseConnection(ConnObj);
    end;
  end
  else
    raise Exception.Create('Pool de banco de dados esgotado. Tente novamente.');
end;
```

51.3 3. Código-Fonte e Registro na Paleta

O código integral encontra-se em `DlpDatabaseConnectionPool.pas`.

O componente é registrado na paleta `'Delphi Engenharia Avancada'`, oferecendo uma infraestrutura essencial para aplicações de backend e clientes pesados.

52 MÓDULO 48: Motor de Atualização Automática (TDlpAutoUpdateManager)

A manutenção contínua de ecossistemas corporativos no Delphi 12 Athens+ requer esteiras de distribuição que mantenham as frotas de estações de trabalho sempre alinhadas com as últimas alterações da legislação tributária e melhorias de software sem gerar chamados recorrentes para o Help Desk.

Neste módulo, implementamos o `TDlpAutoUpdateManager`, organizando o ciclo completo de verificação semântica de versão, download desacoplado, verificação de hash SHA-256 e substituição atômica com reinicialização transparente.

52.1 1. Topologia de Entrega de Patches

```
graph TD
    A[Inicialização / Timer Periódico] ⟶ B[TDlpAutoUpdateManager.CheckForUpdates]
    B ⟶ C[Consulta Manifest JSON Remoto]
    C ⟶ D[CompareSemVer: Versão Remota > Local?]
    D ⟶|Não| E[Aplicação Atualizada]
    D ⟶|Sim| F[Dispara OnUpdateAvailable]
    F ⟶ G[DownloadUpdateAsync com Notificação de Progresso]
    G ⟶ H[Verificação Criptográfica do Checksum SHA-256]
    H ⟶|Divergência| I[Alerta de Segurança e Descarte]
    H ⟶|Confere| J[OnUpdateReady: ApplyUpdateAndRestart]
    J ⟶ K[Gera Script Efêmero e Encerra Processo]
```

52.2 2. Comparação de Versões SemVer (Semantic Versioning)

A precedência de versões segue o padrão `MAJOR.MINOR.PATCH`:

```
function TDlpAutoUpdateManager.CompareSemVer(const V1, V2: string): Integer;
var
  P1, P2: TArray<string>;
  Maj1, Min1, Patch1, Maj2, Min2, Patch2: Integer;
begin
  P1 := V1.Split(['.']);
  P2 := V2.Split(['.']);
  // ... Parsing de inteiros e comparação hierárquica ...
end;
```

52.3 3. Código-Fonte e Registro na Paleta

O código completo do componente está disponível em `DlpAutoUpdateManager.pas`.

O componente é registrado na paleta `'Delphi Engenharia Avancada'`, pronto para modernizar os processos de entrega contínua em Delphi.

53 MÓDULO 49: Harness de Micro-Benchmarks de Código (TDlpCodeBenchmarkHarness)

A garantia de alta vazão e eficiência algorítmica no Delphi 12 Athens+ exige instrumentos científicos de medição de tempo de execução. Ao comparar abordagens concorrentes (como concatenação de strings com operador `+` versus `TStringBuilder`, ou serialização manual de memória versus RTTI estendida), variações sutis na ordem de nanossegundos decidem a escalabilidade da solução.

Neste módulo, implementamos o `TDlpCodeBenchmarkHarness`, utilizando os temporizadores de hardware de alta resolução da API do Windows (`QueryPerformanceCounter` / `QueryPerformanceFrequency`) e amostragem estatística repetida com fase de aquecimento prévio de caches.

53.1 1. Topologia de Micro-Profiling de Código

```
graph TD
    A[Cadastro de Alvos: RegisterTarget] ⟶ B[RunAllBenchmarks]
    B ⟶ C[Fase Warmup: 1.000 iterações de aquecimento de cache]
    C ⟶ D[QueryPerformanceCounter: Início da janela temporal]
    D ⟶ E[Loop de Medição: 100.000 iterações de teste]
    E ⟶ F[QueryPerformanceCounter: Término da janela temporal]
    F ⟶ G[Cálculo de Nanosegundos por Operação e Ops/s]
    G ⟶ H[Geração de Relatório Tabular Comparativo]
```

53.2 2. Exemplo de Uso: Benchmark Comparativo

```
var
  Harness: TDlpCodeBenchmarkHarness;
begin
  Harness := TDlpCodeBenchmarkHarness.Create(nil);
  try
    Harness.RegisterTarget('Concatenação com +', BenchmarkConcatSimples);
    Harness.RegisterTarget('TStringBuilder', BenchmarkStringBuilder);
    Harness.RegisterTarget('Buffer de Memória Direta', BenchmarkMemoryBuffer);

    Writeln(Harness.RunAllBenchmarks);
  finally
    Harness.Free;
  end;
end;
```

53.3 3. Código-Fonte e Registro na Paleta

O código completo do componente está disponível em `DlpCodeBenchmarkHarness.pas`.

O componente é registrado na paleta `'Delphi Engenharia Avancada'`, fornecendo uma suíte completa de micro-benchmarking para desenvolvedores exigentes.

54 MÓDULO 50: Pipeline de CI/CD Multiplataforma e Releases no GitHub (TDlpReleasePipeline)

A culminação do desenvolvimento profissional de componentes em Delphi 12 Athens+ é a automação da esteira de compilação, validação de compatibilidade multiplataforma (Windows 32/64 bits, Linux, Android e iOS) e empacotamento dos artefatos de distribuição para o ecossistema GetIt Package Manager e GitHub Releases.

Neste quinquagésimo módulo, implementamos o `TDlpReleasePipeline`, automatizando a geração de notas de lançamento (Changelog Markdown), o cômputo dos hashes criptográficos de cada binário e a estruturação do pipeline de integração contínua (CI) baseado no motor oficial Microsoft Build Engine (`msbuild`).

54.1 1. Topologia da Esteira DevOps Delphi

```
graph TD
    A[Git Push / Tag de Versão] ⟶ B[GitHub Actions Windows Runner]
    B ⟶ C[Passo 1: actions/checkout]
    C ⟶ D[Passo 2: Configuração de Ferramentas MSBuild]
    D ⟶ E[Passo 3: Compilação de DlpComponents_RT.dpk Win64]
    E ⟶ F[Passo 4: Compilação de DlpComponents_DT.dpk Win32]
    F ⟶ G[Passo 5: Execução da Suíte DUnitX / Testes de Componentes]
    G ⟶|Falha| H[Interrupção Imediata da Esteira e Alerta]
    G ⟶|Sucesso| I[Passo 6: TDlpReleasePipeline Inventaria Binários e Hashes]
    I ⟶ J[Passo 7: Criação de GitHub Release com Artefatos]
```

54.2 2. Automação de Compilação com MSBuild

A compilação em linha de comando de pacotes Delphi (`.dpk`) e projetos (`.dproj`) sem a interface visual da IDE utiliza o MSBuild parametrizado:

```
# Compilação do pacote Runtime para Windows 64-bit
msbuild Livro2_Delphi_VCL_FMX\code\packages\DlpComponents_RT.dpk /p:Config=Release
        /p:Platform=Win64

# Compilação do pacote Design-Time para a IDE (Win32)
msbuild Livro2_Delphi_VCL_FMX\code\packages\DlpComponents_DT.dpk /p:Config=Release
        /p:Platform=Win32
```

54.2.1 Definição Declarativa do Workflow (.github/workflows/ci_msbuild.yml)

```yaml
name: CI/CD Delphi MSBuild

on:
  push:
    branches: [ main ]
    tags: [ 'v*' ]
  pull_request:
    branches: [ main ]

jobs:
  build-windows:
    runs-on: windows-latest
    steps:
      - name: Checkout Repositório
        uses: actions/checkout@v4

      - name: Configurar MSBuild
        uses: microsoft/setup-msbuild@v2

      - name: Compilar Pacote Runtime (Win64)
        run: msbuild Livro2_Delphi_VCL_FMX\code\packages\DlpComponents_RT.dpk /p:Config=Release
        /p:Platform=Win64

      - name: Compilar Pacote Design-Time (Win32)
        run: msbuild Livro2_Delphi_VCL_FMX\code\packages\DlpComponents_DT.dpk /p:Config=Release
        /p:Platform=Win32
```

54.3 3. Código-Fonte e Registro na Paleta

O código completo do componente está disponível em `DlpReleasePipeline.pas`.

O componente é registrado na paleta `'Delphi Engenharia Avancada'`, coroando com chave de ouro a coleção definitiva de 50 módulos sobre engenharia de componentes no Delphi 12 Athens+.